



Saber SDK 用户手册

文档编号 UM0002

Rev 1.06

2020 年 2 月

北京原子机器人科技有限公司
www.atom-robotics.com

INFORMATION IN THIS DOCUMENT IS PROVIDED IN CONNECTION WITH ATOM ROBOTICS PRODUCTS. NO LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS IS GRANTED BY THIS DOCUMENT. EXCEPT AS PROVIDED IN ATOM'S TERMS AND CONDITIONS OF SALE FOR SUCH PRODUCTS, ATOM ASSUMES NO LIABILITY WHATSOEVER, AND ATOM DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY, RELATING TO SALE AND/OR USE OF ATOM PRODUCTS INCLUDING LIABILITY OR WARRANTIES RELATING TO FITNESS FOR A PARTICULAR PURPOSE, MERCHANTABILITY, OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT.

ATOM ROBOTICS may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." ATOM ROBOTICS reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

The ATOM ROBOTICS SDK may contain design defects or errors known as errata, which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Contact your local ATOM ROBOTICS sales office or your distributor to obtain the latest specifications and before placing your product order.

ATOM ROBOTICS, [include any ATOM ROBOTICS trademarks which are used in this document] and the ATOM ROBOTICS logo are trademarks or registered trademarks of ATOM ROBOTICS Co., Ltd. or its subsidiaries in CHINA and other countries.

*Other names and brands may be claimed as the property of others.

Copyright © 2019, ATOM ROBOTICS Co., Ltd. All rights reserved.

Content

1	关于本文	10
1.1	目标	10
1.2	目标用户	10
1.3	范围和局限性	10
1.4	缩写词、缩略语和专用术语	11
1.5	文档格式	11
2	SDK 概述	13
2.1	SDK 安装	13
2.2	SDK 结构	13
2.2.1	Examples 示例文件夹	14
2.2.2	Linux 文件夹	15
2.2.3	Windows 文件夹	15
2.2.4	ROS 文件夹	15
3	系统结构	17
3.1	概述	17
3.2	设备系统状态	18
3.2.1	WAKEUP 状态	18
3.2.2	CONFIG 状态	19
3.2.3	MEASURE 状态	19
3.2.4	SELFTEST 状态	19
3.2.5	MAG_CAL 状态	19
3.3	数据包的多样性	19
4	协议描述	22
4.1	通用消息结构	22
4.2	头部字段	23
4.2.1	Preamble1	23
4.2.2	Preamble2	23
4.2.3	MADDR	24
4.2.4	CID	24
4.2.5	MID	25
4.2.6	PL(有效载荷长度)	25
4.2.7	SLP(超长数据包)	25
4.3	PAYLOAD 有效载荷	26
4.4	Atom BCC(Atom 异或校验)	26
4.5	TAIL 尾部字段	26
4.6	Saber 数据配置结构	27
4.7	Saber 数据包结构	28
4.7.1	温度	29
4.7.2	原始数据(Acc/Gyro/Mag)	29
4.7.3	校准数据(Acc/Gyro/Mag)	30
4.7.4	Kalman 滤波数据(Acc/Gyro/Mag)	30
4.7.5	磁力偏差数据	31
4.7.6	气压计原始数据	31
4.7.7	Kalman 滤波的气压计数据	31
4.7.8	Gnss PVT 数据	31

4.7.9	Gnss Satellites 数据	32
4.7.10	四元数	32
4.7.11	欧拉角	33
4.7.12	余弦矩阵	33
4.7.13	线性加速度	34
4.7.14	速度	34
4.7.15	位移	34
4.7.16	升沉	34
4.7.17	高度	35
4.7.18	经度纬度	35
4.7.19	轮速计数据	35
4.7.20	数据包编号	35
4.7.21	操作系统参考时间	35
4.7.22	采样间隔时间	36
4.7.23	状态字	36
4.7.24	Saber 数据包 PID	36
5	API 接口	38
5.1	枚举定义	38
5.1.1	模式 ID	38
5.2	宏定义	38
5.2.1	Saber 类名称	38
5.2.2	操作类	39
5.2.3	设备信息类	40
5.2.4	传感器配置类	41
5.2.5	算法引擎类	42
5.2.6	通信配置类	44
5.2.7	主机配置类	44
5.2.8	固件升级类	45
5.3	ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200)	46
5.4	ATR_CloseCom(const char *openComNo)	46
5.5	ATR_Init()	47
5.6	ATR_RegisterCallback(CallbackFunction pFunction, unsigned char classID, unsigned char mID)	48
5.7	ATR_DeRegisterCallback(u8 classID, u8 mID)	48
5.8	ATR_WakeUpHostCmdAck ()	49
5.9	ATR_SwitchModeReq(char mode)	50
5.10	ATR_SystemResetReq()	51
5.11	ATR_ConfigurationWriteToFlashReq()	51
5.12	ATR_ConfigurationLoadFromFlashReq()	52
5.13	ATR_LoadFactoryConfigurationReq()	53
5.14	ATR_GetFWVersionReq()	53
5.15	ATR_GetProductVersionReq()	54
5.16	ATR_GetVendorInfoReq()	55
5.17	ATR_GetAllStatusReq()	56
5.18	ATR_GetDeviceInfoReq()	57
5.19	ATR_SetModuleIDReq(u8 id)	59
5.20	ATR_GetModuleIDReq()	59
5.21	ATR_SetAccelerometerScaleReq(u8 *acc, u8 dataLen)	60
5.22	ATR_GetAccelerometerScaleReq()	61
5.23	ATR_SetAccelerometerCoeffReq(u8 *acc, u8 dataLen)	62
5.24	ATR_GetAccelerometerCoeffReq()	63

5.25	ATR_SetGyroscopeScaleReq(u8 *gyro, u8 dataLen)	64
5.26	ATR_GetGyroscopeScaleReq()	65
5.27	ATR_SetMagnetometerCoeffReq(u8 *mag, u8 dataLen)	66
5.28	ATR_GetMagnetometerCoeffReq()	67
5.29	ATR_SetMagnetometerSensorCalibrateReq(u8 *onOrOff, u8 dataLen)	68
5.30	ATR_SetMux2Req(u8 *onOrOff, u8 dataLen)	69
5.31	ATR_GetMux2Req()	70
5.32	ATR_SetAcc1DEKFParamReq(u8 *KFP, u8 dataLen)	71
5.33	ATR_GetAcc1DEKFParamReq()	72
5.34	ATR_SetGyro1DEKFParamReq(u8 *KFP, u8 dataLen)	73
5.35	ATR_GetGyro1DEKFParamReq()	75
5.36	ATR_SetMag1DEKFParamReq(u8 *KFP, u8 dataLen)	76
5.37	ATR_GetMag1DEKFParamReq()	77
5.38	ATR_SetEngineFusionModeReq(u8 *setMode, u8 dataLen)	78
5.39	ATR_GetEngineFusionModeReq()	79
5.40	ATR_SetPacketUpdateRateReq(u8 *setRate, u8 dataLen)	80
5.41	ATR_GetPacketUpdateRateReq()	81
5.42	ATR_SetGyroThresholdReq(u8 *pdata, u8 dataLen)	82
5.43	ATR_GetGyroThresholdReq()	83
5.44	ATR_SetYawOffsetReq(u8 *data, u8 dataLen)	84
5.45	ATR_SetAttitudeOffsetReq(u8 *data, u8 dataLen)	85
5.46	ATR_SetReferenceOffsetReq(u8 *data, u8 dataLen)	86
5.47	ATR_ClearOffsetReq(u8 *data, u8 dataLen)	87
5.48	ATR_SetComPortParamReq(u8 *para, u8 dataLen)	87
5.49	ATR_GetComPortParamReq()	89
5.50	ATR_SetComPortParamHandshakeReq()	90
5.51	ATR_SetDrdyConfigReq(u8* pData, u8 dataLen)	91
5.52	ATR_GetDrdyConfigReq()	92
5.53	ATR_SetGpioConfigReq(u8* pData, u8 dataLen)	93
5.54	ATR_GetGpioConfigReq()	94
5.55	ATR_SetSyncOutConfigReq(u8 *pData, u8 dataLen)	95
5.56	ATR_GetSyncOutConfigReq()	97
5.57	ATR_SetDataPacketConfigReq(u8* pData, u8 dataLen)	97
5.58	ATR_GetDataPacketConfigReq()	98
5.59	ATR_RegisterDownloadCallback(DownloadCallbackFunction pFunction)	99
5.60	ATR_DownloadFirmware_new(char *filename)	100
5.61	ATR_GetVersion(char *version_Buf)	101
5.62	ATR_DebugSwitch(bool onOrOff)	101
5.63	ATR_GetScenarioReq()	102
5.64	ATR_SetScenarioReq(u8* pData, u8 dataLen)	103
6	底层通信协议接口	104
6.1	概述	104
6.2	操作类	104
6.2.1	WakeUpHost	104
6.2.2	WakeUpHostAck	104
6.2.3	SwitchToConfigMode	104
6.2.4	SwitchToConfigModeAck	105
6.2.5	SwitchToMeasureMode	105
6.2.6	SwitchToMeasureModeAck	105
6.2.7	SystemReset	105
6.2.8	SystemResetAck	105

6.2.9	ConfigurationWriteToFlash	105
6.2.10	ConfigurationWriteToFlashAck	106
6.2.11	ConfigurationLoadFromFlash	106
6.2.12	ConfigurationLoadFromFlashAck	106
6.2.13	LoadFactoryConfiguration	106
6.2.14	LoadFactoryConfigurationAck	106
6.3	设备信息类	107
6.3.1	GetFWVersion	107
6.3.2	RespFWVersion	107
6.3.3	GetProductVersion	107
6.3.4	RespProductVersion	107
6.3.5	GetVendorInfo	107
6.3.6	RespVendorInfo	108
6.3.7	GetAllStatus	108
6.3.8	RespAllStatus	108
6.3.9	GetDeviceInfo	110
6.3.10	RespDeviceInfo	110
6.3.11	SetModuleID	111
6.3.12	ModuleIDAck	111
6.3.13	GetModuleID	111
6.3.14	RespModuleID	112
6.4	传感器配置类	112
6.4.1	SetAccelerometerScale	112
6.4.2	AccelerometerScaleAck	112
6.4.3	GetAccelerometerScale	112
6.4.4	RespAccelerometerScale	113
6.4.5	SetAccelerometerCoeff	113
6.4.6	AccelerometerCoeffAck	113
6.4.7	GetAccelerometerCoeff	114
6.4.8	RespAccelerometerCoeff	114
6.4.9	SetGyroscopeScale	114
6.4.10	GyroscopeScaleAck	115
6.4.11	GetGyroscopeScale	115
6.4.12	RespGyroscopeScale	115
6.4.13	SetMagnetometerCoeff	116
6.4.14	MagnetometerCoeffAck	116
6.4.15	GetMagnetometerCoeff	116
6.4.16	RespMagnetometerCoeff	116
6.4.17	SetMagnetometerCalibrate	117
6.4.18	MagnetometerCalibrateAck	118
6.4.19	MagCalPacket	118
6.4.20	MagCalMessage	118
6.4.21	SetMux2	119
6.4.22	Mux2Ack	119
6.4.23	GetMux2	120
6.4.24	RespMux2	120
6.5	算法引擎类	121
6.5.1	SetACC1DEKFPParam	121
6.5.2	ACC1DEKFPParamAck	121
6.5.3	GetACC1DEKFPParam	121
6.5.4	RespACC1DEKFPParam	122
6.5.5	SetGYRO1DEKFPParam	122
6.5.6	GYRO1DEKFPParamAck	122
6.5.7	GetGYRO1DEKFPParam	123
6.5.8	RespGYRO1DEKFPParam	123

6.5.9	SetMAG1DEKFParam	123
6.5.10	MAG1DEKFParamAck	124
6.5.11	GetMAG1DEKFParam	124
6.5.12	RespMAG1DEKFParam	124
6.5.13	SetEngineFusionMode	124
6.5.14	EngineFusionModeAck	125
6.5.15	GetEngineFusionMode	125
6.5.16	RespEngineFusionMode	125
6.5.17	SetPacketUpdateRate	125
6.5.18	PacketUpdateRateAck	126
6.5.19	GetPacketUpdateRate	126
6.5.20	RespPacketUpdateRate	126
6.5.21	SetGyroThreshold	127
6.5.22	GyroThresholdACK	127
6.5.23	GetGyroThreshold	127
6.5.24	RespGyroThreshold	127
6.5.25	SetYawOffSet	128
6.5.26	SetYawOffSetAck	128
6.5.27	SetAttitudeOffSet	128
6.5.28	SetAttitudeOffSetAck	128
6.5.29	SetReferenceOffSet	128
6.5.30	SetReferenceOffSetAck	129
6.5.31	SetDistanceOffSet	129
6.5.32	SetDistanceOffSetAck	129
6.5.33	SetVelocityOffSet	129
6.5.34	SetVelocityOffSetAck	129
6.5.35	SetStartYaw	130
6.5.36	SetStartYawAck	130
6.5.37	ClearOffset	130
6.5.38	ClearOffsetAck	130
6.5.39	SetOdometerPara	130
6.5.40	SetOdometerParaAck	132
6.5.41	GetOdometerPara	132
6.5.42	RespOdometerPara	132
6.5.43	OdometerPacket	133
6.5.44	SetScenario	133
6.5.45	SetScenarioAck	134
6.5.46	GetScenario	134
6.5.47	RespGetScenario	134
6.5.48	SetCoordinatesMode	135
6.5.49	SetCoordinatesModeAck	136
6.5.50	GetCoordinatesMode	137
6.5.51	RespGetCoordinatesMode	137
6.5.52	SetSelfTestMode	137
6.5.53	SetSelfTestModeAck	138
6.5.54	GetSelfTestMode	139
6.5.55	RespGetSelfTestMode	139
6.6	通讯配置类	140
6.6.1	SetComPortParam	140
6.6.2	ComPortParamAck	141
6.6.3	GetComPortParam	142
6.6.4	RespComPortParam	142
6.6.5	SetComPortParamHandshake	143
6.6.6	ComPortParamHandshakeAck	143
6.7	主机配置类	143

6.7.1	SaberDatapacket.....	143
6.7.2	SetDrdyConfig	144
6.7.3	DrdyConfigAck	144
6.7.4	GetDrdyConfig	145
6.7.5	RespDrdyConfig	145
6.7.6	SetGpioConfig.....	145
6.7.7	GpioConfigAck	146
6.7.8	GetGpioConfig	146
6.7.9	RespGpioConfig.....	147
6.7.10	SetSyncOutConfig	147
6.7.11	SetSyncOutConfigAck	148
6.7.12	GetSyncOutConfig	148
6.7.13	GetSyncOutConfigAck	148
6.7.14	SetDataPacketConfig	149
6.7.15	DataPacketConfigAck	149
6.7.16	GetDataPacketConfig	149
6.7.17	RespDataPacketConfig	149
6.8	固件更新类	149
6.8.1	FirmwareUpdateReq	149
6.8.2	FirmwareUpdateReqAck	150
6.8.3	FirmwareUpdatePacket.....	150
6.8.4	FirmwareUpdatePacketAck.....	150
7	示例代码	152
7.1	概述	152
7.2	SDK AutoProtocolDemo	153
7.3	SDK MeasureDataDemo	154
7.4	SDK DownloadFirmware	155
7.5	Low-Level MeasureDataDemo 说明	157
7.5.1	Windows/STM32-Uart/Linux.....	157
7.5.2	STM32 IIC and SPI	167
7.6	Low-Level Download 底层下载说明	172
7.6.1	SwitchToConfigMode.....	172
7.6.2	FirmwareUpdateReq	173
7.6.3	FirmwareUpdatePacket.....	173
7.6.4	SystemReset	173
8	Saber 返回码	174
	历史版本.....	175

Figures

Figure 1 – Saber SDK 目录结构	13
Figure 2 – Saber 应用结构	17
Figure 3 – 系统状态转换流程图.....	18
Figure 4 – 通用消息结构	22
Figure 5 – 头结构	23
Figure 6 – CID 结构	24
Figure 7 – MID 结构	25
Figure 8 – 有效载荷和超长数据包结构	26
Figure 9 – Saber 数据配置结构	28

Figure 10 – Saber 数据包结构	29
Figure 11 – Saber 状态定义	109
Figure 12 – 初始化与自检恢复流程.....	110
Figure 13 – 坐标定义	132
Figure 14 – SDK AutoProtocolDemo 运行示例	154
Figure 15 – SDK MeasureDataDemo 运行示例.....	155
Figure 16 – SDK DownloadFirmware 在 Windows 下运行示例	157
Figure 17 – UART 例程流程图	158
Figure 18 – Wait Ack 流程图	159
Figure 19 – IIC and SPI 例程流程图.....	168
Figure 20 – IIC 发送 Message 流程	169
Figure 21 – IIC 获取 Measure 数据流程.....	170
Figure 22 – IIC and SPI with DRDY 发送 Message 流程	171
Figure 23 – IIC and SPI with DRDY 获取 Measure 数据流程.....	172

Tables

Table 1 – Saber C1/C2/C2G/C3/C3G/C4/C6/B6 数据包列表	21
Table 2 – GNSS PVT Data	32
Table 3 – Saber 数据包 PID	37
Table 4 – 类名称和类 ID	39
Table 5 – 操作类下的消息列表.....	40
Table 6 – 设备信息类下的消息列表	41
Table 7 – 传感器配置类下的消息列表	42
Table 8 – 算法引擎类下的消息列表	44
Table 9 – 通信配置类下的消息列表	44
Table 10 – 主机配置类下的消息列表.....	45
Table 11 – 固件升级类下的消息列表.....	46
Table 12 – 自检配置与状态表	138
Table 13 – SDK 例程矩阵.....	152
Table 14 – Low Level 例程矩阵	152
Table 15 – Saber 返回码	174
Table 16 – 历史版本	177

1 关于本文

1.1 目标

Saber C 系列是标准的工业级惯导产品，与此同时它集成了我们专有的算法引擎 BosonEngine™，对于工业用户来说，Saber C 系列是一款集高性能与低成本于一体的产品。

本手册专为来自北京原子机器人科技有限公司的 Saber 产品提供关于软件架构和 API 协议相关的产品说明。手册描述了使用 Atom 通信协议与 Saber 产品进行连接的方法和细节，并提供了来自 SDK(软件开发工具包)的示例代码。

1.2 目标用户

目标用户是来自于机器人产品领域或其他对惯性导航产品有工业级需求的软件设计人员和开发人员，如：VR/AR，机器人，无人机，运动跟踪，自动驾驶，物联网等。

本文档也可用于指导正在使用 ATOM 产品进行硬件设计和调试的工程师。

1.3 范围和局限性

从用户的角度来看本文可分为如下几部分：

- 针对功能和模块的基本介绍，以便更好地理解 Saber C 系列产品
- 演示软件结构以便让用户了解如何在工作模式下正确的配置引擎
- 列出所有 API 协议并给出相应的解释
- 介绍一种通过底层协议进行通信的方法
- 对软件开发工具包(SDK)进行描述并且附上示例代码加以解释
- 一些必要的硬件信息

本文总结了软件领域的所有技术方面问题，旨在让用户能够轻松高效地将 Saber 模组集成到各种应用场景中。如果有更多细节未涉及，请访问公司官网：www.atom-robotics.com，或者发邮件给我们：contact@atom-robotics.com

1.4 缩写词、缩略语和专用术语

专业术语	描述
INS	Inertial Navigation System
IMU	Inertial Measurement Unit
VRU	Vertical Reference Unit
AHRS	Attitude & Heading Reference System
VR	Virtual Reality
AR	Augmented Reality
UAV	Unmanned Aerial Vehicle
DCM	Direction Cosine Matrix
MISC	Miscellaneous
SDK	Software Development Kit
BosonEngine™	The name of Saber Engine
Hz	Hertz
DRDY	Data ready

1.5 文档格式

正文格式： 字体

文字引用： “1.5文档格式”

状态： **WAKEUP**

命令格式： ***WakeUpHost***



例程格式: `#define FULL_SCALE 32768`

特殊名称: BosonEngine™

2 SDK 概述

2.1 SDK 安装

Saber SDK是一套软件开发工具，专为在特定软件包、软件框架和硬件平台的开发环境下开发的开发人员提供便捷的技术指导，以便与Saber产品进行通信。

我们开发了一个名为“SaberSDK_Setup_xxx.exe”的专业SDK安装包。首先，用户在使用之前需要安装SDK，安装程序如下：

1. 打开SDK文件夹并双击可执行文件“SaberSDK_Release_Setup_xxx.exe”。
2. 点击“next”直到安装完成。

2.2 SDK 结构

提供的SDK是单个可执行文件。安装完成后，在安装路径下有一个名为“AtomRobotics”的文件夹，其中包含以下子文件夹：

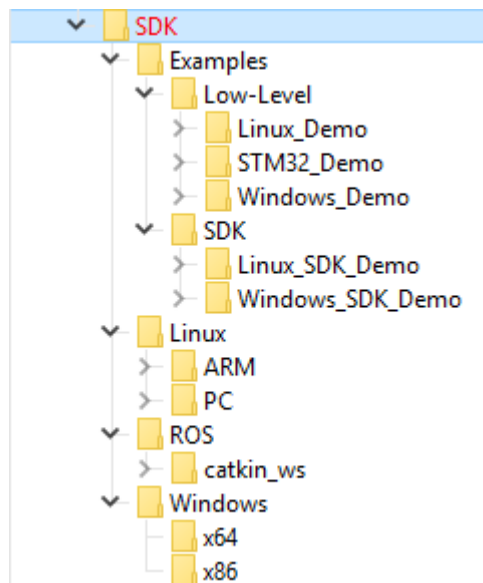


Figure 1 – Saber SDK 目录结构

2.2.1 Examples 示例文件夹

Exmaples 示例文件夹包含：

Low-Level文件夹：

- Linux_Demo: Linux平台Demo，由ubuntu 14.04系统编译和运行。包括：
 - o MeasureDataDemo文件夹中为Linux环境下获取测量数据例程。
 - o DownLoad_Firmware文件夹中为Linux环境下实现Atom Robotics模组更新FW的例程。
- Windows_Demo: Windows平台Demo，由VS2013环境编译运行，包括：
 - o MeasureDataDemo文件夹中为windows下获取模组的测量数据例程。
 - o DownLoad_Firmware文件夹中为windows下实现Atom Robotics模组更新FW的例程。
- STM32_Demo: STM32平台Demo，由IAR7或者8在Windows10系统上编译并在STM32F411环境下运行，包括：
 - o DownLoad_Firmware文件夹中为实现Atom Robotics模组更新FW的例程。
 - o MeasureDataDemo文件夹，分为下面几个工程：
 - STM32 Uart版，由IAR在Windows10系统上编译并在STM32F411环境下运行，演示了如何利用Uart去使用Atom Robotics的底层协议与设备通信。
 - STM32 IIC版，由IAR在Windows10系统上编译并在STM32F411环境下运行，此版本分为两个子版本，使用DRDY和不使用DRDY版本，演示了如何利用IIC去使用Atom Robotics的底层协议与设备通信。
 - STM32 SPI版，由IAR在Windows10系统上编译并在STM32F411环境下运行，此版本为使用DRDY的版本（SPI只能使用DRDY进行通信），演示了如何利用SPI去使用Atom Robotics的底层协议与设备通信。

SDK文件夹：

- Linux_SDK_Demo: Linux平台Demo，包括：

- AutoProtocolDemo文件夹中有X86, X64, ARM环境下, 调用SDK实现与Atom Robotics设备的协议通信例程。
- DownLoad_Firmware文件夹中有X86, X64, ARM环境下, 调用SDK实现为Atom Robotics模组更新的例程。
- Windows_SDK_Demo: Windows平台Demo, 包括:
 - AutoProtocolDemo文件夹中为windows下VS2013环境下, 调用SDK实现与Atom Robotics设备的协议通信例程。
 - DownLoad_Firmware文件夹中为windows下VS2013环境下, 调用SDK实现为Atom Robotics模组更新的例程。
 - MeasureDataDemo文件夹中为windows下VS2013环境下, 调用SDK实现与Atom Robotics设备的测量数据获取解析例程。

2.2.2 Linux 文件夹

Linux文件夹包括Linux环境下SDK的二进制库文件:

- PC-X86: Atom Robotics发布了基于32位桌面操作系统(Ubuntu 16.04)的Linux SDK, 经过测试, 支持GCC-7.3.0、GCC-5、GCC-5.4.0、GCC-4.8版本。
- PC-X64: Atom Robotics发布了基于64位桌面操作系统(Ubuntu 18.04)的Linux SDK, 同样支持GCC-7.3.0、GCC-5、GCC-5.4.0、GCC-4.8版本。
- ARM: Atom Robotics发布了基于ARM两种交叉编译环境(arm-none-linux-gnueabi-gcc、gcc-linaro-4.9.4-2017.1)的Linux SDK。

2.2.3 Windows 文件夹

Windows文件夹包括Windows环境下SDK的二进制库文件:

- X64: Atom Robotics发布了基于64位桌面操作系统的Windows SDK。
- X86: Atom Robotics发布了基于32位桌面操作系统的Windows SDK。

2.2.4 ROS 文件夹

ROS文件夹包含:

- catkin_ws文件夹是ROS和模组通信的例程。
 - ROS版本为Kinetic Kame，环境为 Ubuntu 16.04，64位操作系统
- ROS例程使用方法：
1. 打开新终端进入catkin_ws；
创建工作空间，将imu_publish中的源码放入对应工作空间中；详细步骤参考
ReadMe.txt；
运行命令catkin_make编译产生可执行文件“imu”；
编译结束后运行roscore。
 2. 打开新终端进入catkin_ws；
执行命令source devel/setup.bash；
给串口添加权限sudo chmod 666 /dev/ttyUSB0 (代码写的是ttyUSB0,如果需要更改，可在代码中更改)；
执行命令 rosrn imu_publish imu (imu_publish 是package的名字，imu是可执行文件名)。
 3. 打开新终端进入catkin_ws；
执行命令source devel/setup.bash； 执行命令rostopic echo /Imu_data 可以看见发布在topic的Imu_data。

注：ROS例程输出的Linear ACC单位是 m/s^2 ，输出的Calibrated Gyro单位是 rad/s 。

3 系统结构

3.1 概述

为了给用户提供最灵活的访问方式，Saber C系列产品提供三种硬件接口供选择：UART，SPI和IIC。UART接口可以通过SaberXplore应用程序或通过API进行配置，SPI和IIC接口可以通过API进行配置。

以下是应用管理器与 Saber 设备连接架构图。

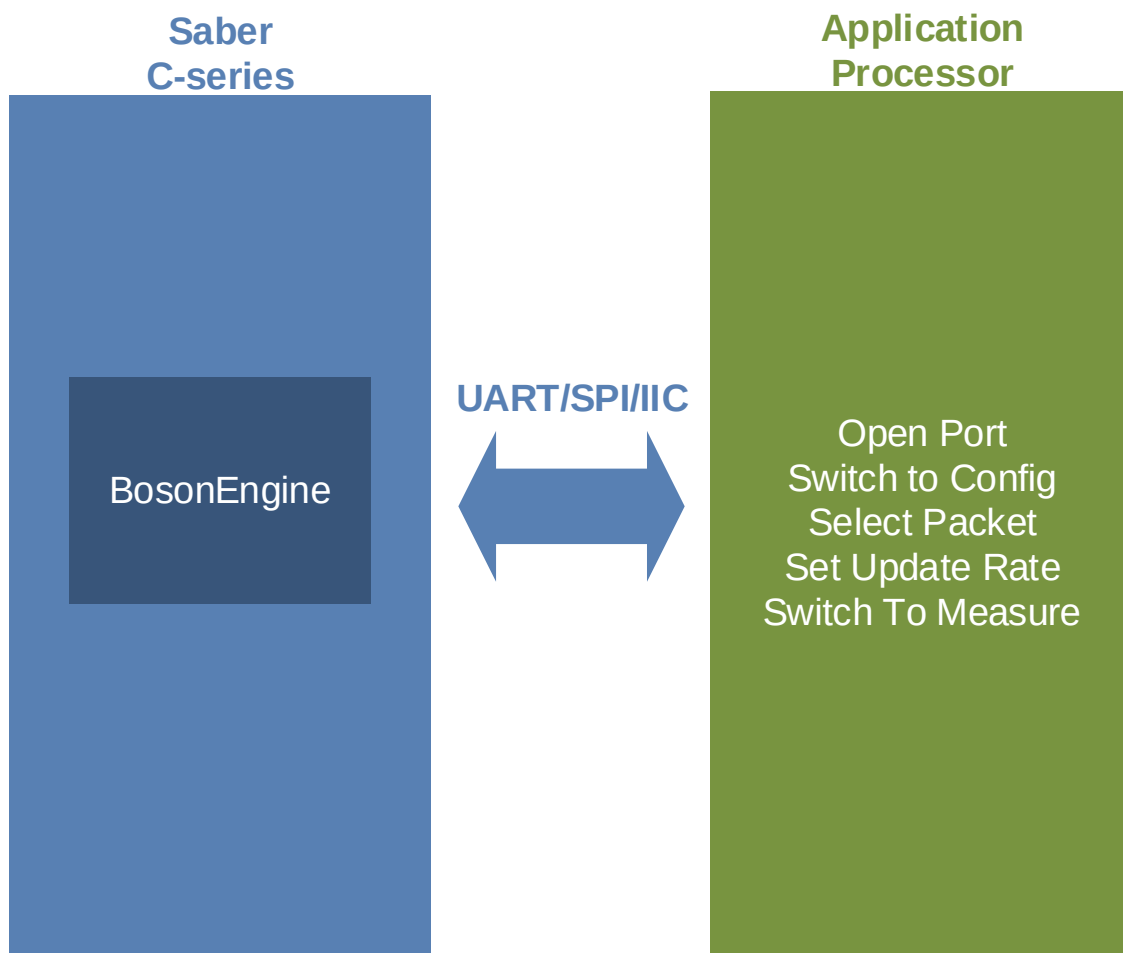


Figure 2 – Saber 应用结构

3.2 设备系统状态

Saber 定义了几种系统状态以方便用户从一种状态模式切换到另一种状态模式。Saber 总共包含 5 个系统状态：**WAKEUP**、**CONFIG**、**MEASURE**、**SELFTEST**、**MAG CAL**(磁力计校准)。“Figure 3”是 Saber 系统状态转换流程图，它显示了如何通过响应不同的命令从一个状态切换到另一个状态。

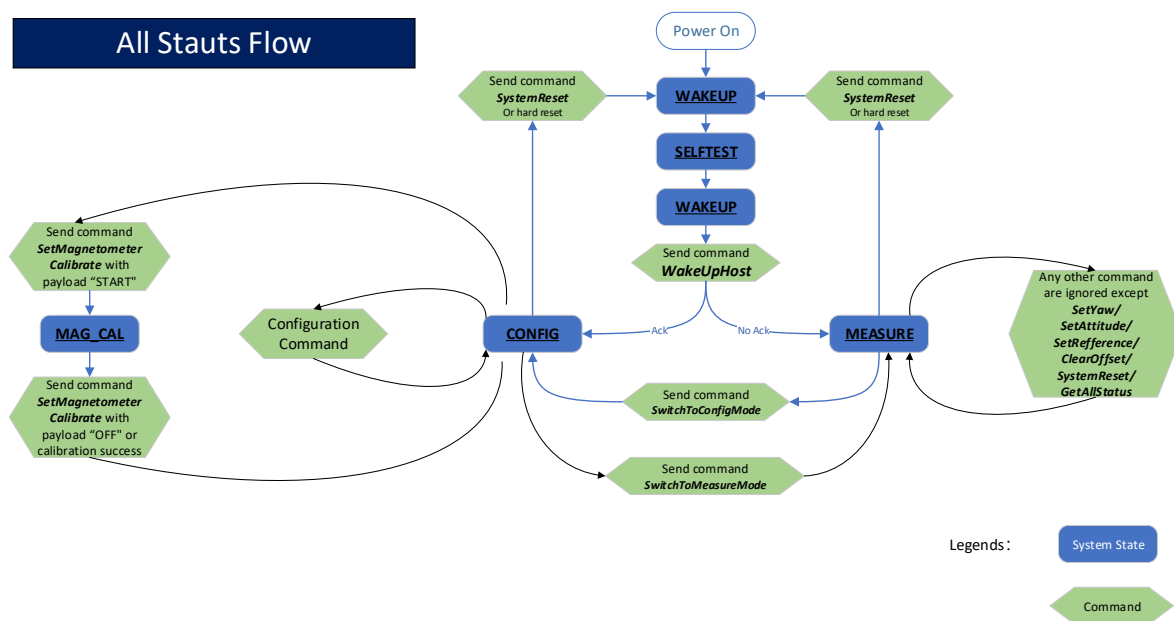


Figure 3 – 系统状态转换流程图

3.2.1 WAKEUP 状态

当Saber启动时，它将运行在**WAKEUP**状态。以下程序将在**WAKEUP**状态下执行：

- 硬件参数初始化。
- 变量参数初始化。
- 算法参数初始化。
- 操作系统初始化。
- 转入**SELFTEST**状态进行传感器自检测。
- 传感器参数初始化。
- 发送**WakeUpHost**命令。

3.2.2 CONFIG 状态

当Saber从主机接收到**SwitchToConfigMode**命令时，它将切换到**CONFIG**状态，在该状态下，Saber固件将处理由主机发送的命令，并向主机发送确认响应。

3.2.3 MEASURE 状态

当Saber从主机收到**SwitchToMeasureMode**命令时，它将执行以下操作：

- 从传感器获取原始数据。
- 根据参数配置处理原始数据(例如校准/卡尔曼滤波器设置)。
- 将传感器数据传送到Boson引擎进行数据融合。
- 准备数据并通过接口(UART, SPI或IIC)将数据发送到主机。

3.2.4 SELFTEST 状态

Saber在**WAKEUP**状态中完成参数初始化后，如果配置选择进行自检将会进入**SELFTEST**状态，进行加速度计、陀螺仪、磁力计的自检。自检成功完成后，回到**WAKEUP**状态。关于**SELFTEST**状态的逻辑，详见“6.5.52 SetSelfTestMode”章节。

3.2.5 MAG_CAL 状态

SetMagnetometerCalibrate命令控制Saber进入或退出**MAG_CAL**状态，进行磁力计的校准。在校准完成后Saber退出该状态。

3.3 数据包的多样性

Saber C系列产品内部集成了微处理器，并且能够在复杂模式(sophisticate)下运行BosonEngine™，同时可实现高达400Hz的数据更新速率，或者在快速模式(swift)下运行BosonEngine™，从而实现高达1000Hz的数据更新速率。传感器数据经过AKF (Atom卡尔曼滤波) 结构的引擎计算，并自动地利用外部输入数据进行算法估计和校正。该引擎提供了两种数据融合模式，以满足不同应用场景的需要。即使没有外部设备数据对于速度和距离的补偿，Saber产品(C2G/C3G/C4/C6)也可以通过BosonEngine™中的算法对速度和距离进行估计，所有的数据包都可以自由配置，其中包括：

- 温度数据;
- 原始传感器数据:
 - 加速度计X, Y, Z三轴传感器测量;
 - 陀螺仪X, Y, Z三轴传感器测量;
 - 磁力计X, Y, Z三轴传感器测量;
- 校准传感器数据:
 - 加速度计X, Y, Z三轴校准和补偿传感器测量;
 - 陀螺仪X, Y, Z三轴校准和补偿传感器测量;
 - 磁力计X, Y, Z三轴校准和补偿传感器测量;
- 卡尔曼滤波器的传感器数据:
 - 基于相应“校准传感器数据”的加速度计X, Y, Z三轴卡尔曼滤波数据;
 - 基于相应“校准传感器数据”的陀螺仪X, Y, Z三轴卡尔曼滤波数据;
 - 基于相应“校准传感器数据”的磁力计X, Y, Z三轴卡尔曼滤波数据;
- 磁力计偏差数据;
- 气压计数据:
 - 气压计原始数据;
 - 经过Kalman滤波的气压计数据;
- GNSS数据:
 - GNSS PVT数据;
 - Gns Satellite 数据;
- 姿态数据:
 - 翻滚角, 俯仰角, 偏航角;
 - 四元数;
 - DCM;
- 运动数据:
 - 线性加速度;
 - 速度;
 - 距离;
 - 升沉运动;
 - 经纬度;

- 海拔;
- 里程计;
- 其余数据:
 - 时间戳;
 - ΔT ;
 - 数据包编号;
 - 状态字。

下表显示了 Saber 支持的所有数据包。

Category Name	Section Name	Saber Section name	PID (Packet ID, 2 BYTES)	C1	C2	C2G	C3	C3G	C4	C6	B6
TEMPERATURE		SABER_TEMPERATURE_CLASS									
	Temperature	Saber_Temperature	0x8000/0x0000	●	●	●	●	●	●	●	●
ACCELERATION		SABER_ACCELERATION_CLASS									
	Raw Data Acc	Saber_Raw_Data_Acc	0x8400/0x0400	●	●	●	●	●	●	●	●
	Calibrated Acc	Saber_Cal_Acc	0x8800/0x0800	●	●	●	●	●	●	●	●
	Kalman Acc	Saber_Kal_Acc	0x8801/0x0801	●	●	●	●	●	●	●	●
GYROSCOPE		SABER_GYROSCOPE_CLASS									
	Raw Data Gyro	Saber_Raw_Data_Gyro	0x8401/0x0401	●	●	●	●	●	●	●	●
	Calibrated Gyro	Saber_Cal_Gyro	0x8C00/0x0C00	●	●	●	●	●	●	●	●
	Kalman Gyro	Saber_Kal_Gyro	0x8C01/0x0C01	●	●	●	●	●	●	●	●
MAGNETOMETER		SABER_MAGNETOMETER_CLASS									
	Raw Data Mag	Saber_Raw_Data_Mag	0x8402/0x0402				●	●	●	●	●
	Calibrated Mag	Saber_Cal_Mag	0x9000/0x1000				●	●	●	●	●
	Kalman Mag	Saber_Kal_Mag	0x9001/0x1001				●	●	●	●	●
	Magneto Deviation	Saber_Mag_Dev	0x9002/0x1002				●	●	●	●	●
BAROMETER		SABER_BAROMETER_CLASS									
	Raw Baro	Saber_Raw_Data_Baro	0x8403/0x0403						●	●	●
	Kalman Baro	Saber_Kal_Baro	0x9401/0x1401						●	●	●
GNSS		SABER_GNSS_CLASS									
	Gnss PVT Data	Saber_Gnss_PVT_Data	0xA000/0x2000							●	●
	Gnss Satellites Data	Saber_Gnss_Satellites_Data	0xA001/0x2001							●	●
ORIENTATION		SABER_ORIENTATION_CLASS									
	Quaternion	Saber_Quaternion	0xB000/0x3000		●	●	●	●	●	●	●
	Euler Angles	Saber_Euler	0xB001/0x3001		●	●	●	●	●	●	●
	Rotation Matrix	Saber_Rotation_Matrix	0xB002/0x3002		●	●	●	●	●	●	●
MOTION		SABER_MOTION_CLASS									
	Linear Acc	Saber_Linear_Acc	0xB800/0x3800			●		●	●	●	●
	Velocity	Saber_Velocity	0xB802/0x3802			●		●	●	●	●
	Distance	Saber_Distance	0xB403/0x3403			●		●	●	●	●
	Heave Motion	Saber_Heave_Motion	0xB803/0x3803						●	●	●
	Altitude Ellipsoid	Saber_Altitude_Ellipsoid	0xB400/0x3400							●	●
	LonLat	Saber_LonLat	0xB402/0x3402							●	●
	Odometer Data	Saber_Odometer_Data	0xB500/0x3500			●		●	●	●	●
MISC		SABER_MISC_CLASS									
	Packet Counter	Saber_Packet_Counter	0xC000/0x4000	●	●	●	●	●	●	●	●
	OS Reference Time Stamp	Saber_Os_Time	0xC002/0x4002	●	●	●	●	●	●	●	●
	Delta T	Saber_DeltaT	0xC006/0x4006	●	●	●	●	●	●	●	●
	Status Word	Saber_Status_Word	0xC400/0x4400	●	●	●	●	●	●	●	●

Table 1 – Saber C1/C2/C2G/C3/C3G/C4/C6/B6 数据包列表

4 协议描述

4.1 通用消息结构

该协议是一组用于在主机和设备(也称为模组)之间进行通信的消息。如果用户希望通过 IIC、SPI 和 UART 接口直接与 Saber 产品以底层协议进行通信，那么对底层协议的充分理解非常重要。

下面是通用协议的结构，它定义了主机和设备(模组)之间通信的详细字段，详细描述如下图。

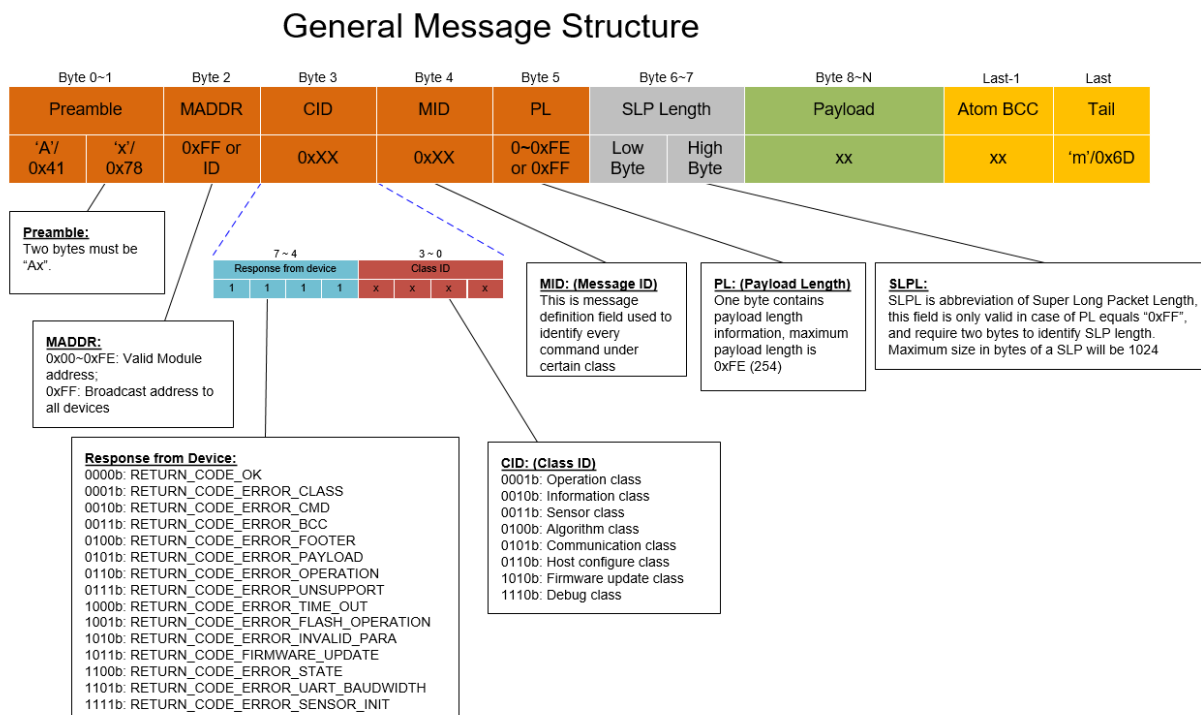


Figure 4 – 通用消息结构

4.2 头部字段

头一个字段在每个通用 Saber Message 中最多包含八个字节，它包括：**Preamble**，**MADDR**(Module ADDRESS)，**CID**(Class ID)，**MID**(Message ID)，**PL**(Payload Length)和 **SLP**(Super Length Packet)。

头区域的每个子字段详细说明如下，头结构如下图所示。

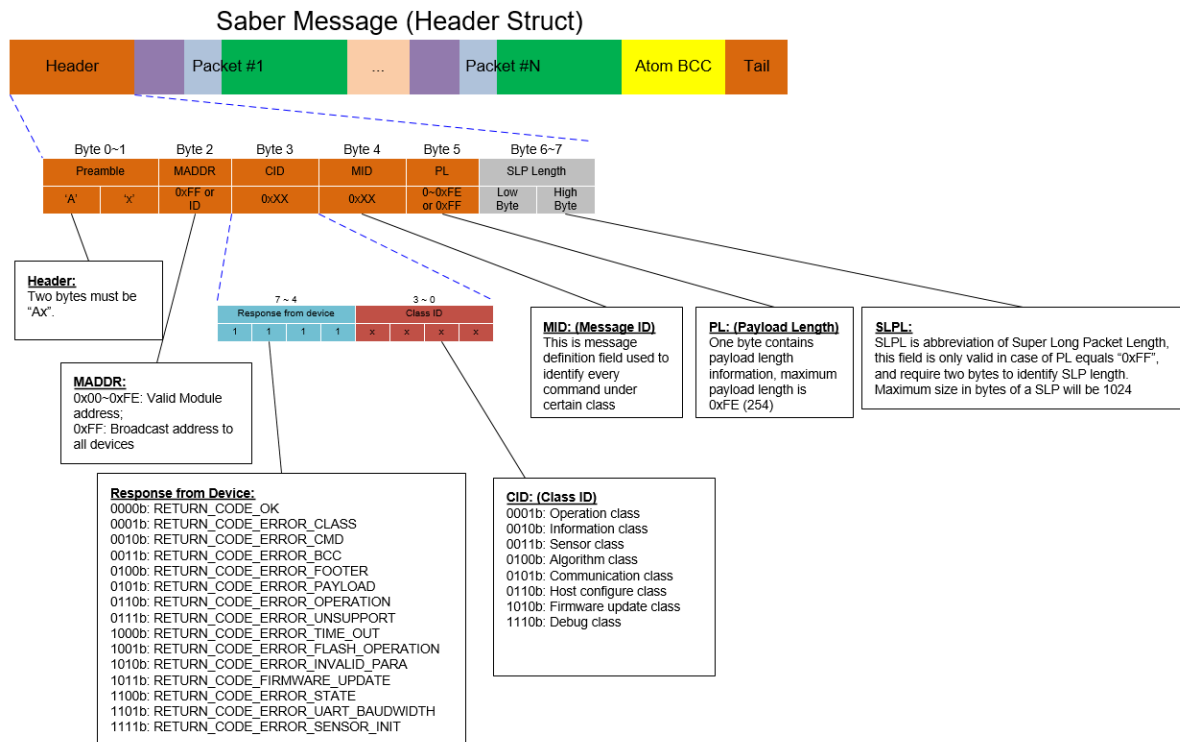


Figure 5 – 头结构

4.2.1 Preamble1

Preamble1 位于第一个字节，并且必须固定为“A”

4.2.2 Preamble2

Preamble2 位于第二个字节，并且必须固定为“x”

4.2.3 MADDR

模组地址标识符

在默认状态或硬件/软件复位的情况下，模组将自动从用于识别地址的 non-volatile flash 中检索模组地址信息，如果未设置有效地址，则在响应或用于广播信息时采用 0xFF。

综上所述，如果模组已经被设置了一个有效地址，其地址编号必须在 0-0xFE 之间。反过来，取自 flash 的数字用作主机发送或接收消息的标识符。

重要提示： 使用另一个命令 **SetModuleAddress** 来配置专用模组地址。

4.2.4 CID

Class ID 标识符，用于区分不同的消息组。

该字段用来在主机发送或设备发送情况下描述正确的类别 ID。

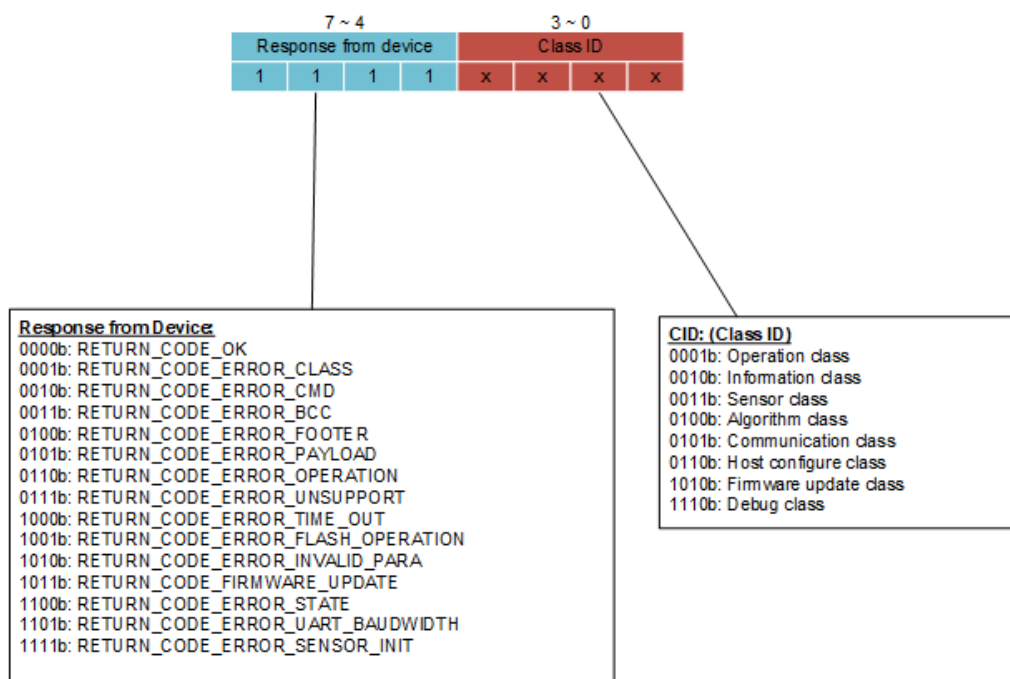


Figure 6 – CID 结构

在响应消息时用最高 4 位来标识 **ERROR** 状态，更多细节请参考“8 Saber 返回码”部分，如果最高 4 位为 0，则表示在相应操作中没有发现错误，其他情况下，则说明有错误发生。

4.2.5 MID

Message ID 标识符，用于描述特定类组下的不同命令。

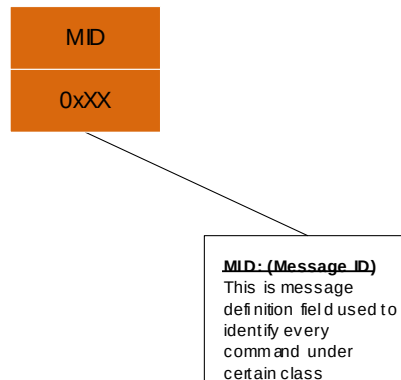


Figure 7 – MID 结构

4.2.6 PL(有效载荷长度)

有效载荷长度，该字段必须是 0-254(0xFE)内的值，而 255(0xFF)将被保留以用来支持超长数据包(SLP)，另一个字段“扩展长度”将会在有效载荷长度之后被添加。

4.2.7 SLP(超长数据包)

该字段占用两个字节，表示有效载荷数据的总长度，其值应在 255-1024 之间。SLP 是可选字段，只有在有效载荷长度等于 0xFF 时才有效，这意味着有效载荷字节超过了正常的数据包长度 0-254，那么它需要超长数据包在 255-1024 发送有效载荷。

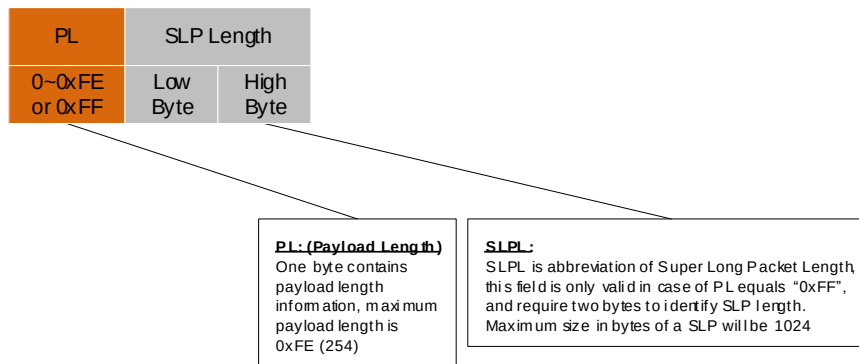


Figure 8 – 有效载荷和超长数据包结构

4.3 PAYLOAD 有效载荷

原则上，有效载荷取决于主机发送或接收的确切命令，该命令在每个 Saber 消息中由 CID 和 MID 标识。

例如，**ConnectDevice**(CID:0x01,MID:0x01)不需要有效载荷;而 **SETAccelerometerScale** (CID:0x03, MID:0x04)需要 1 字节的有效载荷来确定合适的量程。

本章需要讨论两个重要的命令来描述配置模组以及获取数据的基本方法。它们是 **SetDataPacketConfig** 和 **SaberDataPacket**。

用户可以通过命令 **SetDataPacketConfig** 来配置启用了哪些数据包，有关该命令的详细信息将在“4.6 Saber 数据配置结构”一节中介绍。命令 **SaberDataPacket** 的信息详见“4.7 Saber 数据包结构”。

4.4 Atom BCC(Atom 异或校验)

Saber 系列产品使用 Atom BCC 算法来计算 1 字节的 BCC。

详情请参阅示例代码。

```
u8 Atom_BCC(u8 *addr, u16 len)
{
    unsigned char *DataPoint;
    DataPoint = addr;
    unsigned char XorData = 0;
    unsigned short DataLength = len;

    while(DataLength --)
    {
        XorData = XorData ^ *DataPoint;
        DataPoint ++;
    }

    return XorData;
}
```

4.5 TAIL 尾部字段

与 Preamble 对应，固定字节‘m’/0x6D 被定义为 Saber 消息的终止码。

4.6 Saber 数据配置结构

Saber 数据配置结构是从主机发送到设备最重要的消息之一，它将指示设备中哪一个数据包被启用或禁用。在配置中，如果相应的数据包被启用，Saber C 系列模组就会考虑该数据包的计算。如果需要特定的 ODR(输出数据速率)输出用户选择的数据包，那么相应算法将会开始运行并通过专用硬件接口(UART, IIC 和 SPI)发送数据包。

每个 Saber 模组的消息配置都遵循通用消息结构，如“4.1 通用消息结构”所述。集成的 Saber 数据配置消息包含头部、单独数据包配置、Atom BCC(Atom 异或校验)和尾部。

单独数据包配置包含四个字节，较高的两个字节保留以供将来使用，较低的两个字节则用于配置。

配置单独数据包的较低两位字节的定义如下图所示，共定义 16 位：

- Bit 15 用于禁止相应数据包的启用。当该位设置为‘1’时，数据包启用；当该位复位为‘0’时，数据包被禁用。
- Bit 14-Bit 10 用于标识数据包的类别。专用数据包是被标记为类别 ID 和区段 ID 的组合。
- Bit 9-Bit 3 被保留以备将来使用。
- Bit 2-Bit 0 被用作区段 ID。

如何确定由 Category ID 和 Section ID 组合选择哪个数据包，详情请参考“4.7.24Saber 数据包 PID”。

下图为 Saber 的数据配置结构。

Saber Data Configure Structure

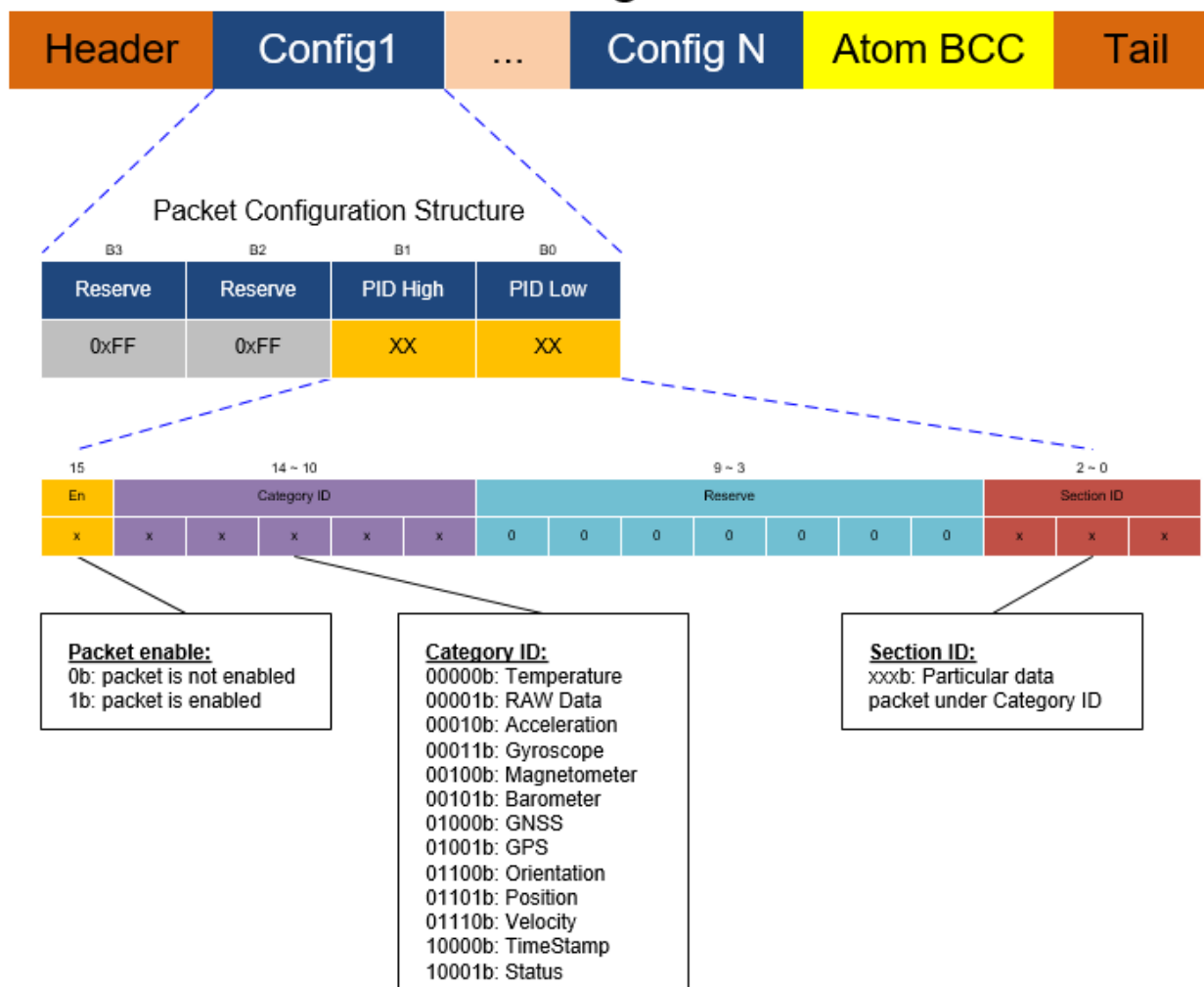


Figure 9 – Saber 数据配置结构

4.7 Saber 数据包结构

无论何时设备要进入 **MEASURE** 模式，Saber 数据包消息都会从设备发送到主机。与 Saber 数据配置消息一样，Saber 数据包消息包含头部、单独数据包、Atom BCC(Atom 异或校验)和尾部，详见“6.7.1SaberDatapacket”，唯一的区别是单独数据包具有可变的有效载荷长度和有效载荷数据，而数据配置在同一字段中只有 4 个字节。

数据包的前两个字节始终为 PID(与数据配置定义相同)，后跟 1 个字节的 PL(有效载荷长度)和 N 个字节的有有效负荷。Saber 数据包结构如下图。

数据通过 LSB 传输。

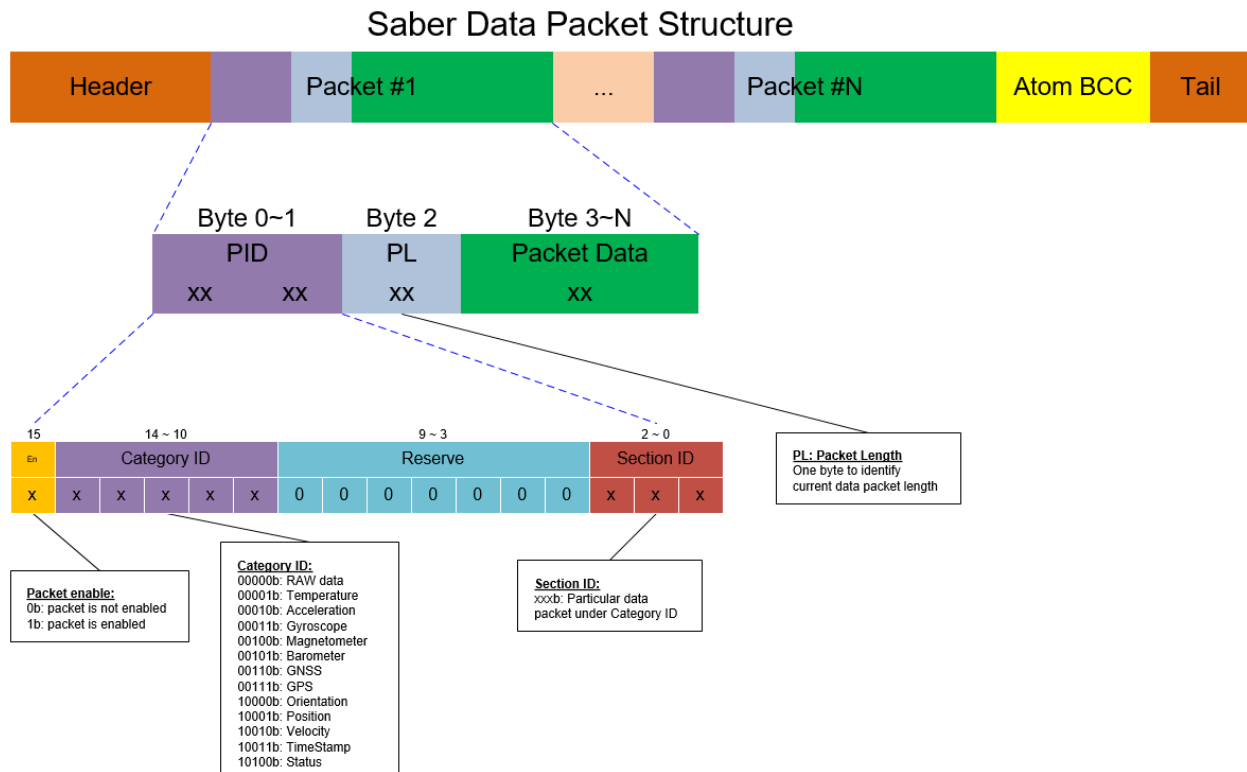
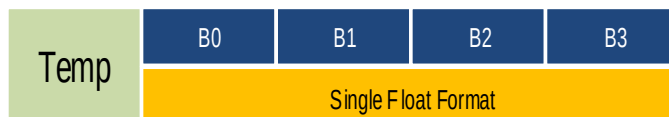


Figure 10 – Saber 数据包结构

4.7.1 温度

温度的数据包由 4 字节的单精度浮点数表示，单位摄氏度，PID 是 0x8000。



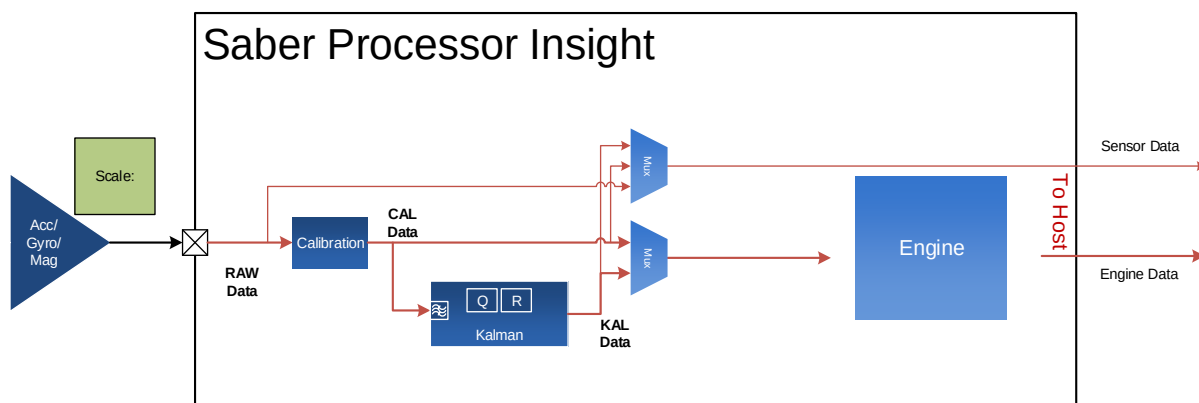
4.7.2 原始数据(Acc/Gyro/Mag)

Saber 系列模组根据配置可以输出原始数据、校准数据和 Kalman 滤波数据。这三种数据的主要区别如下图所示。

Raw Data 是传感器采样的原始数据；

CAL Data 是经过校准的数据；

KAL Data 是把 CAL Data 再进行 Kalman 滤波之后的数据。



像 ACC 或 Gyro 或 Mag 这样一个传感器的原始数据(X/Y/Z 轴)的数据包由 6 个字节组成, short 格式。加速度计单位 g, 陀螺仪单位 dps, 磁力计单位 mGauss。

Raw Data Acc 的 PID 是 0x8400, Raw Data Gyro 的 PID 是 0x8401, Raw Data Mag 的 PID 是 0x8402。数据是 short 类型。

RAW/X	RAW-X B0	RAW-X B1	RAW/Y	RAW-Y B2	RAW-Y B3	RAW/Z	RAW-Z B4	RAW-Z B5
	UShort Format			UShort Format			UShort Format	

4.7.3 校准数据(Acc/Gyro/Mag)

像 ACC 或 Gyro 或 Mag 这样一个传感器的校准数据(X/Y/Z 轴)的数据包由 12 个字节组成, float 格式。加速度计单位 g, 陀螺仪单位 dps, 磁力计单位 mGauss。

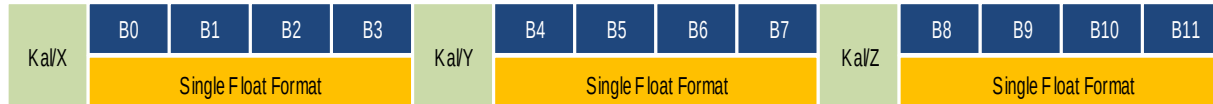
Calibrated Acc 的 PID 是 0x8800, Calibrated Gyro 的 PID 是 0x8C00, Calibrated Mag 的 PID 是 0x9000。

CalX	B0	B1	B2	B3	CalY	B4	B5	B6	B7	CalZ	B8	B9	B10	B11
	Single Float Format					Single Float Format					Single Float Format			

4.7.4 Kalman 滤波数据(Acc/Gyro/Mag)

像 ACC 或 Gyro 或 Mag 这样一个传感器的 Kalman 滤波数据(X/Y/Z 轴)的数据包由 12 个字节组成, float 格式。加速度计单位 g, 陀螺仪单位 dps, 磁力计单位 mGauss。

Kalman Acc 的 PID 是 0x8801, Kalman Gyro 的 PID 是 0x8C01, Kalman Mag 的 PID 是 0x9001。



4.7.5 磁力偏差数据

Magnetic Deviation 是 Saber 计算出的磁力受到干扰时的偏差数据, float 格式, 单位 mGauss, PID 是 0x9002。



4.7.6 气压计原始数据

Raw Baro 是气压计原始数据, 有符号整数, 单位帕斯卡, PID 是 0x8403。



4.7.7 Kalman 滤波的气压计数据

Kalman Baro 是经过 Kalman 滤波的气压计数据, 有符号整数, 单位帕斯卡, PID 是 0x9401。



4.7.8 Gnss PVT 数据

Gnss PVT 数据是带 GNSS 功能的 Saber 模组输出的 GNSS PVT 原始数据, PID 是 0xA000, 具体的协议请参考如下:

Byte Offset	Number Format	Scaling	Name	Unit	Description
0	U4	-	iTOW	ms	GPS time of week of the navigation epoch. See the description of iTOW for details.
4	U2	-	year	y	Year (UTC)
6	U1	-	month	month	Month, range 1..12 (UTC)
7	U1	-	day	d	Day of month, range 1..31 (UTC)
8	U1	-	hour	h	Hour of day, range 0..23 (UTC)
9	U1	-	min	min	Minute of hour, range 0..59 (UTC)
10	U1	-	sec	s	Seconds of minute, range 0..60 (UTC)
11	X1	-	valid	-	Validity flags (see graphic below)
12	U4	-	tAcc	ns	Time accuracy estimate (UTC)
16	I4	-	nano	ns	Fraction of second, range -1e9 .. 1e9 (UTC)
20	U1	-	fixType	-	GNSSfix Type: 0: no fix 1: dead reckoning only 2: 2D-fix 3: 3D-fix 4: GNSS + dead reckoning combined 5: time only fix
21	X1	-	flags	-	Fix status flags (see graphic below)
22	X1	-	flags2	-	Additional flags (see graphic below)
23	U1	-	numSV	-	Number of satellites used in Nav Solution
24	I4	1E-07	lon	deg	Longitude
28	I4	1E-07	lat	deg	Latitude
32	I4	-	height	mm	Height above ellipsoid
36	I4	-	hMSL	mm	Height above mean sea level
40	U4	-	hAcc	mm	Horizontal accuracy estimate
44	U4	-	vAcc	mm	Vertical accuracy estimate
48	I4	-	velN	mm/s	NED north velocity
52	I4	-	velE	mm/s	NED east velocity
56	I4	-	velD	mm/s	NED down velocity
60	I4	-	gSpeed	mm/s	Ground Speed (2-D)
64	I4	0.00001	headMot	deg	Heading of motion (2-D)
68	U4	-	sAcc	mm/s	Speed accuracy estimate
72	U4	0.00001	headAcc	deg	Heading accuracy estimate (both motion and vehicle)
76	U2	0.01	pDOP	-	Position DOP
78	U1[6]	-	reserve d1	-	Reserved
84	I4	0.00001	headVeh	deg	Heading of vehicle (2-D)
88	U1[4]	-	reserve d2	-	Reserved

Table 2 – GNSS PVT Data

4.7.9 Gnss Satellites 数据

Gnss Satellite 数据是 Saber 系列带 GNSS 或 RTK 功能输出的卫星数据，PID 是 0xA001。

4.7.10 四元数

四元数数据(Q0-Q3)的数据包由 16 个字节组成，float 格式，PID 是 0xB000:

Q0: 字节 B0-B3

Q1: 字节 B4-B7

Q2: 字节 B8-B11

Q3: 字节 B12-B15



4.7.11 欧拉角

欧拉角数据包由 16 字节组成，单位度，float 格式，其中 B12-B15 被保留，单位度，PID 是 0xB001:

翻滚角: B0-B3

俯仰角: B4-B7

偏航角: B8-B11



4.7.12 余弦矩阵

余弦矩阵由 36 个字节组成，总共 9 个 float 数据，表示的是导航坐标系到机体坐标系的转换矩阵，PID 是 0xB002，矩阵具体表示如下:

Matrix[0][0]: B0-B3

Matrix[0][1]: B4-B7

Matrix[0][2]: B8-B11

Matrix[1][0]: B12-B15

Matrix[1][1]: B16-B19

Matrix[1][2]: B20-B23

Matrix[2][0]: B24-B27

Matrix[2][1]: B28-B31

Matrix[2][2]: B32-B35

A11	B0	B1	B2	B3	A12	B4	B5	B6	B7	A13	B8	B9	B10	B11
	Single Float Format					Single Float Format					Single Float Format			
A21	B12	B13	B14	B15	A22	B16	B17	B18	B19	A23	B20	B21	B22	B23
	Single Float Format					Single Float Format					Single Float Format			
A31	B24	B25	B26	B27	A32	B28	B29	B30	B31	A33	B32	B33	B34	B35
	Single Float Format					Single Float Format					Single Float Format			

4.7.13 线性加速度

Saber_Linear_Acc 数据包由 12 个字节组成，单位 m/s^2 ，float 格式，PID 是 0xB800

Linear /X	B4	B5	B6	B7	Linear /Y	B8	B9	B10	B11	Linear /Z	B12	B13	B14	B15
	Single Float Format					Single Float Format					Single Float Format			

4.7.14 速度

Saber_Velocity 数据包由 12 个字节组成，单位 m/s ，float 格式，PID 是 0xB802

Velocity/X	B4	B5	B6	B7	Velocity/Y	B8	B9	B10	B11	Velocity/Z	B12	B13	B14	B15
	Single Float Format					Single Float Format					Single Float Format			

4.7.15 位移

Saber_Distance 数据包由 12 个字节组成，单位 m ，float 格式，PID 是 0xB403

Dis_X	B4	B5	B6	B7	Dis_Y	B8	B9	B10	B11	Dis_Z	B12	B13	B14	B15
	Single Float Format					Single Float Format					Single Float Format			

4.7.16 升沉

Saber_Heave_Motion 数据包由 4 个字节组成，单位 m ，float 格式，PID 是 0xB803

Heave	B0	B1	B2	B3
	Single Float Format			

4.7.17 高度

Saber_Altitude_Ellipsoid 数据包是 Saber C6 模组输出的定位信息(海拔), 单位是 m, 总共由 4 个字节组成, float 格式, PID 是 0xB400

	B0	B1	B2	B3
Altitude	Single Float Format			

4.7.18 经度纬度

Saber_LonLat 数据包是 Saber C6 模组输出的定位信息(经度和纬度), 单位度, 总共由 16 个字节组成, double 格式, PID 是 0xB402

Lon	B0	B1	B2	B3	B4	B5	B6	B7	Lat	B8	B9	B10	B11	B12	B13	B14	B15
	Single Float Format				Single Float Format					Single Float Format				Single Float Format			

4.7.19 轮速计数据

Saber_Odometer 数据包是 Saber 模组输出的轮速计信息, 总共由 16 个字节组成, 单位 m/s, double 格式, PID 是 0xB500

LS	B0	B1	B2	B3	RS	B4	B5	B6	B7	LD	B8	B9	B10	B11	RD	B12	B13	B14	B15
	Single Float Format					Single Float Format					Single Float Format					Single Float Format			

4.7.20 数据包编号

数据包编号由 4 个字节组成, 无符号整形, PID 是 0xC000

P.NO	B0	B1	B2	B3
	UInt Format			

4.7.21 操作系统参考时间

操作系统参考时间由 6 个字节组成, PID 是 0xC002

B0-B3 是无符号整形, 以毫秒为单位。

B4-B5 是无符号整形, 以微秒为单位。

ORT/ ms	B0	B1	B2	B3	ORT/ us	B4	B5
	UInt Format					Uchar	Uchar

4.7.22 采样间隔时间

Saber_DeltaT 是 Saber 输出的数据包的时间间隔由 4 个字节组成，单位毫秒，float 格式，PID 是 0xC006。

DT	B0	B1	B2	B3
Single Float Format				

4.7.23 状态字

Saber_Status_Word 是 Saber 输出的当前系统状态字，由 4 个字节组成，PID 是 0xC400。

具体的状态字的定义，请参考“6.3.7GetAllStatus”命令的回复。

Status	B0	B1	B2	B3
Single Float Format				

4.7.24 Saber 数据包 PID

Category Name	Section Name	Saber Section name	PID (Packet ID, 2 BYTEs)	C1	C2	C2G	C3	C3G	C4	C6	B6
TEMPERATURE		SABER_TEMPERATURE_CLASS									
	Temperature	Saber_Temperature	0x8000/0x0000	•	•	•	•	•	•	•	•
ACCELERATION		SABER_ACCELERATION_CLASS									
	Raw Data Acc	Saber_Raw_Data_Acc	0x8400/0x0400	•	•	•	•	•	•	•	•
	Calibrated Acc	Saber_Cal_Acc	0x8800/0x0800	•	•	•	•	•	•	•	•
	Kalman Acc	Saber_Kal_Acc	0x8801/0x0801	•	•	•	•	•	•	•	•
GYROSCOPE		SABER_GYROSCOPE_CLASS									
	Raw Data Gyro	Saber_Raw_Data_Gyro	0x8401/0x0401	•	•	•	•	•	•	•	•
	Calibrated Gyro	Saber_Cal_Gyro	0x8C00/0x0C00	•	•	•	•	•	•	•	•
	Kalman Gyro	Saber_Kal_Gyro	0x8C01/0x0C01	•	•	•	•	•	•	•	•
MAGNETOMETER		SABER_MAGNETOMETER_CLASS									
	Raw Data Mag	Saber_Raw_Data_Mag	0x8402/0x0402				•	•	•	•	
	Calibrated Mag	Saber_Cal_Mag	0x9000/0x1000				•	•	•	•	
	Kalman Mag	Saber_Kal_Mag	0x9001/0x1001				•	•	•	•	
	Magneto Deviation	Saber_Mag_Dev	0x9002/0x1002				•	•	•	•	
BAROMETER		SABER_BAROMETER_CLASS									
	Raw Baro	Saber_Raw_Data_Baro	0x8403/0x0403						•	•	
	Kalman Baro	Saber_Kal_Baro	0x9401/0x1401						•	•	
GNSS		SABER_GNSS_CLASS									
	Gnss PVT Data	Saber_Gnss_PVT_Data	0xA000/0x2000							•	•
	Gnss Satellites Data	Saber_Gnss_Satellites_Data	0xA001/0x2001							•	•
ORIENTATION		SABER_ORIENTATION_CLASS									
	Quaternion	Saber_Quaternion	0xB000/0x3000		•	•	•	•	•	•	
	Euler Angles	Saber_Euler	0xB001/0x3001		•	•	•	•	•	•	•
	Rotation Matrix	Saber_Rotation_Matrix	0xB002/0x3002		•	•	•	•	•	•	
MOTION		SABER_MOTION_CLASS									
	Linear Acc	Saber_Linear_Acc	0xB800/0x3800			•		•	•	•	
	Velocity	Saber_Velocity	0xB802/0x3802			•		•	•	•	•
	Distance	Saber_Distance	0xB403/0x3403			•		•	•	•	
	Heave Motion	Saber_Heave_Motion	0xB803/0x3803						•	•	
	Altitude Ellipsoid	Saber_Altitude_Ellipsoid	0xB400/0x3400							•	•
	LonLat	Saber_LonLat	0xB402/0x3402							•	•
	Odometer Data	Saber_Odometer_Data	0xB500/0x3500			•		•	•	•	•
		SABER_MISC_CLASS									
	Packet Counter	Saber_Packet_Counter	0xC000/0x4000	•	•	•	•	•	•	•	•
MISC	OS Reference Time Stamp	Saber_Os_Time	0xC002/0x4002	•	•	•	•	•	•	•	•
	Delta T	Saber_DeltaT	0xC006/0x4006	•	•	•	•	•	•	•	•
	Status Word	Saber_Status_Word	0xC400/0x4400	•	•	•	•	•	•	•	•

Table 3 – Saber 数据包 PID

5 API 接口

5.1 枚举定义

5.1.1 模式 ID

通过以下数据结构，列举了 Saber 产品的工作模式：

```
enum mode
{
    WAKEUP_MODE,
    CONFIG_MODE,
    MEASURE_MODE,
    SELFTEST_MODE,
    MAG_CAL_MODE
};
```

5.2 宏定义

下面给出了所有 Saber 消息 CID(Class ID)和 MID(Message ID)的详细定义，详情请参见“Figure 4 – 通用消息结构”和“4.2 头部字段”了解更多关于 CID 和 MID 的信息。CID 和 MID 是用户需要知道的基本信息，下面的配置由主机给出，它展示了不同消息组的含义和该组下的每个命令。

5.2.1 Saber 类名称

Saber 支持 8 类操作，如下表所示：

- 操作类，该类提供了控制模组的基本途径。
- 设备信息类，该类提供了获取设备信息的方法。

- 传感器配置类，该类主要用于配置传感器，如加速度计、陀螺仪、磁力计、气压计等。
- 算法引擎类，该类主要用于通过配置引擎的工作模式和参数来控制引擎。
- 通信配置类，该类主要提供了一些命令来配置通信接口，例如 Uart, IIC 和 SPI。
- 主机控制类，该类用于配置主机处理器的其余项信号，例如数据准备就绪，信号同步，GPIO 等。
- 固件升级类，该类用于提供更新固件的方法。
- 调试类，该类仅用于内部调试。

详情请参阅下表以获得类 ID 的赋值。

类名称	类 ID
#define CLASS_ID_OPERATION_CMD	0x01
#define CLASS_ID_DEVICE_INFO	0x02
#define CLASS_ID_SENSOR_CONFIG	0x03
#define CLASS_ID_ALGORITHMENGINE	0x04
#define CLASS_ID_COMMUNICATION_CONFIG	0x05
#define CLASS_ID_HOSTCONTROL_CMD	0x06
#define CLASS_ID_FIRMWARE_UPDATE	0x0A
#define CLASS_ID_DEBUG	0x0E

Table 4 –类名称和类 ID

5.2.2 操作类

下表列出了操作类下的所有消息，用户可以根据表格自由发送消息以达到相应的目的，我们也提供相应 API 做上述事情，详细 API 细节请参考“5.3 ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200”到“5.64 ATR_SetScenarioReq(u8* pData, u8 dataLen”章节。

消息名称	消息 ID
#define CMD_ID_WAKEUP_HOST	0x01
#define CMD_ID_SWITCH_TO_CFG_MODE	0x02
#define CMD_ID_SWITCH_TO_MEASURE_MODE	0x03
#define CMD_ID_SYS_RESET	0x04
#define CMD_ID_CFG_WRITE_TO_FLASH	0x08
#define CMD_ID_CFG_LOAD_FROM_FLASH	0x09
#define CMD_ID_CFG_LOAD_FACTORY	0x0A

Table 5 – 操作类下的消息列表

5.2.3 设备信息类

下表列出了设备信息类下的所有消息，用户可以根据表格自由发送消息以达到相应的目的，我们也提供相应 API 做上述事情，详细 API 细节请参考“5.3 ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200”到“5.64 ATR_SetScenarioReq(u8* pData, u8 dataLen”章节。

消息名称	消息 ID
#define CMD_ID_GET_FW_VERSION	0x01
#define CMD_ID_GET_PRODUCT_VERSION	0x03
#define CMD_ID_GET_VENDOR_INFO	0x04
#define CMD_ID_GET_ALL_STATUS	0x06
#define CMD_ID_GET_DEVICE_INFO	0x07
#define CMD_ID_SET_MODULEID	0x0C

#define CMD_ID_GET_MODULEID	0x0D
------------------------------------	------

Table 6 – 设备信息类下的消息列表

5.2.4 传感器配置类

下表列出了传感器配置类下的所有消息，用户可以根据表格自由发送消息以达到相应的目的，我们也提供相应 API 做上述事情，详细 API 细节请参考“5.3 ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200”到“5.64 ATR_SetScenarioReq(u8* pData, u8 dataLen”章节。

消息名称	消息 ID
#define CMD_ID_SET_ACC_SCALE	0x01
#define CMD_ID_GET_ACC_SCALE	0x02
#define CMD_ID_SET_ACC_COEFF	0x03
#define CMD_ID_GET_ACC_COEFF	0x04
#define CMD_ID_SET_GYRO_SCALE	0x06
#define CMD_ID_GET_GYRO_SCALE	0x07
#define CMD_ID_SET_MAG_COEFF	0x0D
#define CMD_ID_GET_MAG_COEFF	0x0E
#define CMD_ID_SET_MAG_CALIBRATION	0x0F
#define CMD_ID_SET_MUX2	0x12
#define CMD_ID_GET_MUX2	0x13
#define CMD_ID_MAG_CAL_PAC (Note)	0xB0
#define CMD_ID_MAG_CAL_MSG (Note)	0xB1

Table 7 – 传感器配置类下的消息列表

注意：虽然没有与宏定义 `CMD_ID_MAG_CAL_PAC` 和 `CMD_ID_MAG_CAL_MSG` 相对应的 API，但是此宏定义也会在磁力计校准的场景中使用。

在磁力计校准时，Saber 将发送消息“6.4.19MagCalPacket”和“6.4.20MagCalMessage”给主机。

用户可以通过以下代码去设置以上两个消息的回调函数：

```
ATR_RegisterCallback(MagCalPacketCallback, CLASS_ID_SENSOR_CONFIG, CMD_ID_MAG_CAL_PAC);
```

```
ATR_RegisterCallback(MagCalMessageCallback, CLASS_ID_SENSOR_CONFIG, CMD_ID_MAG_CAL_MSG);
```

5.2.5 算法引擎类

下表列出了算法引擎类下的所有消息，用户可以根据表格自由发送消息以达到相应的目的，我们也提供相应 API 做上述事情，详细 API 细节请参考“5.3 ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200”到“5.64 ATR_SetScenarioReq(u8* pData, u8 dataLen”章节。

消息名称	消息 ID
<code>#define CMD_ID_SET_ACC_1DEKF_PARAM</code>	0x03
<code>#define CMD_ID_GET_ACC_1DEKF_PARAM</code>	0x04
<code>#define CMD_ID_SET_GYRO_1DEKF_PARAM</code>	0x05
<code>#define CMD_ID_GET_GYRO_1DEKF_PARAM</code>	0x06
<code>#define CMD_ID_SET_MAG_1DEKF_PARAM</code>	0x07
<code>#define CMD_ID_GET_MAG_1DEKF_PARAM</code>	0x08
<code>#define CMD_ID_SET_ENGINE_FUSION_MODE</code>	0x09
<code>#define CMD_ID_GET_ENGINE_FUSION_MODE</code>	0x0A
<code>#define CMD_ID_SET_PACKET_UPDATE_RATE</code>	0x10

#define CMD_ID_GET_PACKET_UPDATE_RATE	0x11
#define CMD_ID_SET_GYRO_THRESHOLD	0x22
#define CMD_ID_GET_GYRO_THRESHOLD	0x23
#define CMD_ID_SET_YAW_OFFSET	0x30
#define CMD_ID_SET_ATTITUDE_OFFSET	0x31
#define CMD_ID_SET_REFERENCE_OFFSET	0x32
#define CMD_ID_SET_DISTANCE_OFFSET	0x33
#define CMD_ID_SET_VELOCITY_OFFSET	0x34
#define CMD_ID_SET_START_YAW	0x35
#define CMD_ID_CLEAR_OFFSET	0x40
#define CMD_ID_SET_ODOMETER_PARA	0x41
#define CMD_ID_GET_ODOMETER_PARA	0x42
#define CMD_ID_ODOMETER_PACKET	0x43
#define CMD_ID_SET_SCENARIO	0x50
#define CMD_ID_GET_SCENARIO	0x51
#define CMD_ID_SET_COORDINATES_MODE	0x52
#define CMD_ID_GET_COORDINATES_MODE	0x53
#define CMD_ID_SET_SELFTEST_MODE	0x54
#define CMD_ID_GET_SELFTEST_MODE	0x55

Table 8 – 算法引擎类下的消息列表

5.2.6 通信配置类

下表列出了通信配置类下的所有消息，用户可以根据表格自由发送消息以达到相应的目的，我们也提供相应 API 做上述事情，详细 API 细节请参考“5.3 ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200”到“5.64 ATR_SetScenarioReq(u8* pData, u8 dataLen”章节。

消息名称	消息 ID
#define CMD_ID_SET_COMPORT_PARAM	0x01
#define CMD_ID_GET_COMPORT_PARAM	0x02
#define CMD_ID_COMPORT_PARAM_HANDSHAKE	0x10

Table 9 – 通信配置类下的消息列表

5.2.7 主机配置类

下表列出了主机配置类下的所有消息，用户可以根据表格自由发送消息以达到相应的目的，我们也提供相应 API 做上述事情详细 API 细节请参考“5.3 ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200”到“5.64 ATR_SetScenarioReq(u8* pData, u8 dataLen”章节。

消息名称	消息 ID
#define CMD_ID_SABER_DATA_PACKET (Note)	0x81
#define CMD_ID_SET_CURRENT_ROLLPITCH_OFFSET	0x02
#define CMD_ID_GET_CURRENT_ROLLPITCH_OFFSET	0x03
#define CMD_ID_SET_DRDY_CONFIG	0x06
#define CMD_ID_GET_DRDY_CONFIG	0x07

#define CMD_ID_SET_GPIO_CONFIG	0x08
#define CMD_ID_GET_GPIO_CONFIG	0x09
#define CMD_ID_SET_DATA_PAC_CFG	0x0A
#define CMD_ID_GET_DATA_PAC_CFG	0x0B
#define CMD_ID_SET_SYNCOUT_CFG	0x0C
#define CMD_ID_GET_SYNCOUT_CFG	0x0D

Table 10 – 主机配置类下的消息列表

注意：虽然没有与宏定义 `CMD_ID_SABER_DATA_PACKET` 相对应的 API，但是此宏定义也会在获取数据的场景中使用。

当 Saber 处于 **MEASURE** 模式时，它将自动发送消息“6.7.1 SaberDatapacket”，用户可以通过以下代码去设置上述消息的回调函数：

```
ATR_RegisterCallback(SaberDataPacketCallback, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_SABER_DATA_PACKET);
```

5.2.8 固件升级类

下表列出了固件升级类下的所有消息，用户可以根据表格自由发送消息以达到相应的目的，我们也提供相应 API 做上述事情，详细 API 细节请参考“5.3 `ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200)`”到“5.64 `ATR_SetScenarioReq(u8* pData, u8 dataLen)`”章节。

消息名称	消息 ID
#define CMD_ID_FIRMWARE_UPDATE_REQ	0x01
#define CMD_ID_FIRMWARE_UPDATE_PACKET	0x02

Table 11 – 固件升级来下的消息列表

5.3 ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate = 115200)

函数名称	ATR_OpenCom
函数原型	ATR_OpenCom(const char *ucComNo, unsigned long ulBaudrate)
行为描述	打开 Com 口。
输入参数	“ucComNo”是指向数据的指针； “ulBaudrate”是波特率； “ucComNo”的值是 Com 口编号。 “ulBaudrate”的值可以有如下选择： 9600, 38400, 57600, 115200, 230400, 460800, 921600, 1500000
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 RETURN_CODE_ERROR_xxx。
先决条件	无
备注	例如： <pre>unsigned char com = 12; unsigned long BaudRate = 115200; ATR_OpenCom(&com, BaudRate);</pre>
注册回调	无

5.4 ATR_CloseCom(const char *openComNo)

函数名称	ATR_CloseCom
------	--------------

函数原型	ATR_CloseCom(const char *openComNo)
行为描述	关闭 Com 口。
输入参数	“openComNo”是 Com 口编号。
输出参数	无
返回值	无
先决条件	无
备注	例如: <code>unsigned char com = 12;</code> <code>ATR_CloseCom(&com);</code>
注册回调	无

5.5 ATR_Init()

函数名称	ATR_Init
函数原型	ATR_Init()
行为描述	初始化 SDK。
输入参数	无
输出参数	无
返回值	无
先决条件	无
备注	例如: <code>ATR_Init();</code>
注册回调	无

5.6 ATR_RegisterCallback(CallbackFunction pFunction, unsigned char classID, unsigned char mID)

函数名称	ATR_RegisterCallback
函数原型	ATR_RegisterCallback(CallbackFunction pFunction, unsigned char classID, unsigned char mID)
行为描述	注册回调函数。
输入参数	CallbackFunction pFunction = user define function classID = CLASS_ID_XXXX, please reference “4.2.4CID” mID = CMD_ID_XXXX, please reference “4.2.5MID”
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 RETURN_CODE_ERROR_REGISTER。
先决条件	无
备注	当收到 Saber 发送的 ACK 消息时候回调 pFunction 函数 例如: <pre>ATR_RegisterCallback(SwitchTOConfigCallBack, CLASS_ID_OPERATION_CMD, CMD_ID_SWITCH_TO_CFG_MODE); Void SwitchTOConfigCallBack(u8 classID,u8 mID,u8 errorCode,u8* pData,u16 datalen) { //User code }</pre>

5.7 ATR_DeRegisterCallback(u8 classID, u8 mID)

函数名称	ATR_DeRegisterCallback
------	------------------------

函数原型	ATR_DeRegisterCallback(u8 classID, u8 mID)
行为描述	取消注册回调函数。
输入参数	classID = CLASS_ID_XXXX, please reference “4.2.4CID” mID = CMD_ID_XXXX, please reference “4.2.5MID”
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 RETURN_CODE_ERROR_REGISTER。
先决条件	无
备注	取消注册回调函数。 例如： <code>ATR_DeRegisterCallback (CLASS_ID_OPERATION_CMD, CMD_ID_SWITCH_TO_CFG_MODE) ;</code>

5.8 ATR_WakeUpHostCmdAck ()

函数名称	ATR_WakeUpHostCmdAck
函数原型	ATR_WakeUpHostCmdAck ()
行为描述	响应从设备发送的 WakeUpHost 命令。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无

备注	例如: <code>ATR_WakeUpHostCmdAck();</code>
注册回调	<code>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_OPERATION_CMD, CMD_ID_WAKEUP_HOST);</code>

5.9 ATR_SwitchModeReq(char mode)

函数名称	ATR_SwitchModeReq
函数原型	ATR_SwitchModeReq(char mode)
行为描述	请求 Saber 进入特定模式。
输入参数	“mode” can be CONFIG_MODE or MEASURE_MODE. “mode” value can be explained as below: CONFIG_MODE = 3; MEASURE_MODE = 4
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如: <code>unsigned char Config_mode = 3; ATR_SwitchModeReq(Config_mode);</code>
注册回调	<code>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_OPERATION_CMD, CMD_ID_SWITCH_TO_CFG_MODE);</code>

	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_OPERATION_CMD, CMD_ID_SWITCH_TO_MEASURE_MODE);</pre>
--	--

5.10 ATR_SystemResetReq()

函数名称	ATR_SystemResetReq
函数原型	ATR_SystemResetReq()
行为描述	请求 Saber 进入软重置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如： ATR_SystemResetReq();
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_OPERATION_CMD, CMD_ID_SYS_RESET);</pre>

5.11 ATR_ConfigurationWriteToFlashReq()

函数名称	ATR_ConfigurationWriteToFlashReq
函数原型	ATR_ConfigurationWriteToFlashReq()
行为描述	Saber 会将用户配置保存到 flash 中。
输入参数	无

输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如： <code>ATR_ConfigurationWriteToFlashReq();</code>
注册回调	<code>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_OPERATION_CMD, CMD_ID_CFG_WRITE_TO_FLASH);</code>

5.12 ATR_ConfigurationLoadFromFlashReq()

函数名称	ATR_ConfigurationLoadFromFlashReq
函数原型	ATR_ConfigurationLoadFromFlashReq()
行为描述	Saber 将从 flash 中检索用户配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如： <code>ATR_ConfigurationLoadFromFlashReq();</code>
注册回调	<code>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_OPERATION_CMD, CMD_ID_CFG_LOAD_FROM_FLASH);</code>

5.13 ATR_LoadFactoryConfigurationReq()

函数名称	ATR_LoadFactoryConfigurationReq
函数原型	ATR_LoadFactoryConfigurationReq()
行为描述	恢复出厂设置，所有配置恢复为默认选项。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如： <code>ATR_LoadFactoryConfigurationReq();</code>
注册回调	<code>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_OPERATION_CMD, CMD_ID_CFG_LOAD_FACTORY);</code>

5.14 ATR_GetFWVersionReq()

函数名称	ATR_GetFWVersionReq
函数原型	ATR_GetFWVersionReq()
行为描述	Saber 将发送固件版本信息。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。

先决条件	无
备注	ATR_GetFWVersion()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(fwVersionResponseMsgCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_SW_VERSION); void fwVersionResponseMsgCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_DEVICE_INFO, CMD_ID_GET_FW_VERSION);</pre>

5.15 ATR_GetProductVersionReq()

函数名称	ATR_GetProductVersionReq
函数原型	ATR_GetProductVersionReq()
行为描述	Saber 将发送产品版本信息。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK， 否则返回 Error， 详情请参阅返回码。

先决条件	无
备注	ATR_GetProductVersionReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(ProductVersionCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_PRODUCT_ID); void ProductVersionCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_DEVICE_INFO, CMD_ID_GET_PRODUCT_VERSION);</pre>

5.16 ATR_GetVendorInfoReq()

函数名称	ATR_GetVendorInfoReq
函数原型	ATR_GetVendorInfoReq()
行为描述	Saber 将发送供应商信息。
输入参数	无
输出参数	无

返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetVendorInfoReq()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(VendorInfoCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_VENDER_INFO); void VenderInfoCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_DEVICE_INFO, CMD_ID_GET_VENDER_INFO);

5.17 ATR_GetAllStatusReq()

函数名称	ATR_GetAllStatusReq
函数原型	ATR_GetAllStatusReq()
行为描述	Saber 将发送所有状态信息。
输入参数	无

输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK, 否则返回 Error, 详情请参阅返回码。
先决条件	无
备注	ATR_GetAllStatusReq()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetAllStatusCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_ALL_STATUS); void AllStatusCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_DEVICE_INFO, CMD_ID_GET_ALL_STATUS);

5.18 ATR_GetDeviceInfoReq()

函数名称	ATR_GetDeviceInfoReq
函数原型	ATR_GetDeviceInfoReq()
行为描述	Saber 将发送所有设备信息。

输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK， 否则返回 Error，详情请参阅返回码。
先决条件	无
备注	实际上该 API 使得用户可以获取所有信息，包括：状态信息，固件版本，产品版本，产品 ID，供应商信息和引擎版本。 ATR_GetDeviceInfoReq()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetDeviceInfoCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_DEVICE_INFO); void GetDeviceInfoCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_DEVICE_INFO, CMD_ID_GET_DEVICE_INFO);

5.19 ATR_SetModuleIDReq(u8 id)

函数名称	ATR_SetModuleIDReq
函数原型	ATR_SetModuleIDReq(u8 id)
行为描述	Saber 将对模组进行编号配置。
输入参数	u8 “id” “id” can be any number.
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	可以让用户通过专用模组 ID 序列化多个 Saber 模组。 例如： <pre>unsigned char id = 12; ATR_SetModuleIDReq(id);</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_DEVICE_INFO, CMD_ID_SET_MODULEID);

5.20 ATR_GetModuleIDReq()

函数名称	ATR_GetModuleIDReq
函数原型	ATR_GetModuleIDReq()
行为描述	Sabe 将返回模组 ID。
输入参数	无
输出参数	无

返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetModuleIDReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(moduleIDMsgCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_MODULEID); void moduleIDMsgCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_DEVICE_INFO, CMD_ID_GET_MODULEID);

5.21 ATR_SetAccelerometerScaleReq(u8 *acc, u8 dataLen)

函数名称	ATR_SetAccelerometerScaleReq
函数原型	ATR_SetAccelerometerScaleReq (u8 *acc, u8 dataLen)
行为描述	Saber 将根据参数“acc”的内容配置加速度计的量程。
输入参数	u8 *acc= The pointer which points to data; u8 dataLen = The length of data;

	“Acc” value can be explained as below: <table border="1"> <thead> <tr> <th>Selection</th><th>Acc Scale</th></tr> </thead> <tbody> <tr> <td>0</td><td>±2G</td></tr> <tr> <td>1</td><td>±4G</td></tr> <tr> <td>2</td><td>±8G</td></tr> <tr> <td>3</td><td>±16G</td></tr> </tbody> </table>	Selection	Acc Scale	0	±2G	1	±4G	2	±8G	3	±16G
Selection	Acc Scale										
0	±2G										
1	±4G										
2	±8G										
3	±16G										
输出参数	无										
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。										
先决条件	无										
备注	例如: <pre>unsigned char acc_scale = 3; unsigned char datalen = 1; ATR_SetAccelerometerScaleReq(&acc_scale, datalen);</pre>										
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_SET_ACC_SCALE);</pre>										

5.22 ATR_GetAccelerometerScaleReq()

函数名称	ATR_GetAccelerometerScaleReq
函数原型	ATR_GetAccelerometerScaleReq()
行为描述	Saber 将返回加速度计的量程。
输入参数	无;
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无

备注	ATR_GetAccelerometerScaleReq()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetAccelerometerScaleCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_ACC_SCALE); void GetAccelerometerScaleCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_GET_ACC_SCALE);</pre>

5.23 ATR_SetAccelerometerCoeffReq(u8 *acc, u8 dataLen)

函数名称	ATR_SetAccelerometerCoeffReq
函数原型	ATR_SetAccelerometerCoeffReq(u8 *acc, u8 dataLen)
行为描述	Saber 将配置加速度计校准系数。
输入参数	“acc” = The pointer which points to data; “dataLen” = The length of data; “ulBaudrate” value can be explained as below: <pre>Typedef struct {</pre>

	<pre>Float Bx, Sx, Exy, Exz, By, Eyx, Sy, Eyz, Bz, Ezx, Ezy, Sz; }Coeff_ttypedef</pre>
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	此 API 应该谨慎使用，因为它会进一步刷新校准系数进而影响加速度计的准确性，除非用户精准理解系数和确保参数正确性，否则我们不建议用户使用该 API。 例如： <pre>float acc_coeff[12] = {1,1,1,1,1,1,1,1,1,1,1,1}; ATR_SetAccelerometerCoeffReq((u8 *)acc_coeff,48);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_SET_ACC_COEFF);</pre>

5.24 ATR_GetAccelerometerCoeffReq()

函数名称	ATR_GetAccelerometerCoeffReq
函数原型	ATR_GetAccelerometerCoeffReq()
行为描述	收到此消息后，Saber 将返回加速度计校准系数。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无

备注	ATR_GetAccelerometerCoeffReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetAccelerometerCoeffCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_ACC_COEFF); void GetAccelerometerCoeffCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_GET_ACC_COEFF);

5.25 ATR_SetGyroscopeScaleReq(u8 *gyro, u8 dataLen)

函数名称	ATR_SetGyroscopeScaleReq
函数原型	ATR_SetGyroscopeScaleReq(u8 *gyro, u8 dataLen)
行为描述	收到此消息后，Saber 将根据下表配置陀螺仪的量程。
输入参数	“gyro” is pointer which points to data; “dataLen” is the length of data “gyro” value can be explained as below:

	Selection	Gyro Scale	Remark
	0	±250	Unit:°/sec
	1	±500	Unit:°/sec
	2	±1000	Unit:°/sec
	3	±2000	Unit:°/sec
输出参数	无		
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。		
先决条件	无		
备注	例如： <pre>unsigned char gyro_scale = 2; ATR_SetGyroscopeScaleReq(&gyro_scale,1);</pre>		
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_SET_GYRO_SCALE);</pre>		

5.26 ATR_GetGyroscopeScaleReq()

函数名称	ATR_GetGyroscopeScaleReq		
函数原型	ATR_GetGyroscopeScaleReq()		
行为描述	收到此信息后，Saber 将按照下表返回陀螺仪的量程。		
	Selection	Gyro Scale	Remark
	0	±250	Unit:°/sec
	1	±500	Unit:°/sec
	2	±1000	Unit:°/sec
	3	±2000	Unit:°/sec
输入参数	无		

输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetGyroscopeScaleReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetGyroscopeScaleCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_GYRO_SCALE); void GetGyroscopeScaleCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_GET_GYRO_SCALE);

5.27 ATR_SetMagnetometerCoeffReq(u8 *mag, u8 dataLen)

函数名称	ATR_SetMagnetometerCoeffReq
函数原型	ATR_SetMagnetometerCoeffReq(u8 *mag, u8 dataLen)

行为描述	Saber 将配置磁力计校准系数
输入参数	“mag” is pointer which points to data; “dataLen” is the length of data “mag” value can be explained as below: <pre>float G[3][3]; float H0[3];</pre>
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	此 API 应该谨慎使用，因为它会进一步刷新校准系数进而影响磁力计的准确性，除非用户精准理解系数和确保参数正确性，否则我们不建议用户使用该 API。 例如: <pre>Float mag_coeff[12] = {1,1,1,2,2,2,3,3,3,1,1,1} ATR_SetMagnetometerCoeffReq(mag_coeff,48);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_SET_MAG_COEFF);</pre>

5.28 ATR_GetMagnetometerCoeffReq()

函数名称	ATR_GetMagnetometerCoeffReq
函数原型	ATR_GetMagnetometerCoeffReq()
行为描述	Saber 将返回磁力计校准系数。
输入参数	无
输出参数	无

返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetMagnetometerCoeffReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetMagnetometerCoeffCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_MAG_COEFF); void GetMagnetometerCoeffCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_GET_MAG_COEFF);

5.29 ATR_SetMagnetometerSensorCalibrateReq(u8 *onOrOff, u8 dataLen)

函数名称	ATR_SetMagnetometerSensorCalibrateReq
函数原型	ATR_SetMagnetometerSensorCalibrateReq(u8 *mag, u8 dataLen)
行为描述	根据“onOrOff”，Saber 将对磁力计进行校准。

输入参数	“onOrOff” is pointer which points to data; “dataLen” is the length of data. “onOrOff” value can be explained as below: 0x09: EIGHT_SHAPED_CALIBRATE_START, 0x0A: SIX_FACE_CALIBRATE_START, 0x0B: CALIBRATE_STOP, 0x0C: TAKE_EFFECT
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK, 否则返回 Error, 详情请参阅返回码。
先决条件	无
备注	例如: <pre>unsigned char mag_cal = 1; ATR_SetMagnetometerSensorCalibrateReq(&mag_cal, 1);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_SET_MAG_CALIBRATION);</pre>

5.30 ATR_SetMux2Req(u8 *onOrOff, u8 dataLen)

函数名称	ATR_SetMux2Req
函数原型	ATR_SetMux2Req(u8 *onOrOff, u8 dataLen)
行为描述	Saber 将决定是校准数据还是卡尔曼数据输入 Boson 引擎。
输入参数	“onOrOff” is pointer which points to data; “dataLen” is the length of data “onOrOff” value can be explained as below:

	First byte is reserved. Second byte (bit0-1 for Acc, bit 2-3 for Gyro, bit4-5 for Mag, others are reserved.) Please refer to “6.4.21SetMux2”
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	用户必须给模组发送正确的配置，如果错误，有效载荷将不会生效。加速度计，陀螺仪和磁力计可以单独设置。 例如： <pre>unsigned char mux2[2] = {0x11,0x3b}; ATR_SetMux2Req(mux2,2);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_SET_MUX2);</pre>

5.31 ATR_GetMux2Req()

函数名称	ATR_GetMux2Req
函数原型	ATR_GETMux2Req()
行为描述	Saber 将返回当前配置，显示是校准数据还是卡尔曼数据输入 Boson 引擎。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetMux2Req ()的响应是命令 ACK 的有效载荷。

	用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetMux2Callback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_MUX2); void GetMux2Callback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_SENSOR_CONFIG, CMD_ID_GET_MUX2);</pre>

5.32 ATR_SetAcc1DEKFParamReq(u8 *KFP, u8 dataLen)

函数名称	ATR_SetAcc1DEKFParamReq
函数原型	ATR_SetAcc1DEKFParamReq(u8 *KFP, u8 dataLen)
行为描述	Saber 将配置外部卡尔曼滤波器参数以满足不同的场景需求。
输入参数	“KFP” is pointer which points to data; “dataLen” is the length of data “KFP” value can be explained as below: <pre>u8 reserve[4]; float Kalman_Q; float Kalman_R;</pre>

输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	如果用户倾向于实现流畅的数据，建议使用高比例的 R/Q，而如果用户趋向于实现快速响应，则建议使用低比例的 R/Q。默认情况下，Q 值为 0.05，R 值为 5。 例如： <pre> unsigned char KFP[12]; KFP[0] = 0x11; KFP[1] = 0x11; KFP[2] = 0x11; KFP[3] = 0x11; KFP[4] = 0xCD; KFP[5] = 0xCC; KFP[6] = 0x4C; KFP[7] = 0x3D; KFP[8] = 0x00; KFP[9] = 0x00; KFP[10] = 0x48; KFP[11] = 0x42; ATR_SetAcc1DEKFParamReq(KFP, 12); </pre>
注册回调	<pre> ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_ACC_1DEKF_PARAM); </pre>

5.33 ATR_GetAcc1DEKFParamReq()

函数名称	ATR_GetAcc1DEKFParamReq
函数原型	ATR_GetAcc1DEKFParamReq()
行为描述	Saber 将返回外部卡尔曼滤波器参数的配置。
输入参数	无

输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK, 否则返回 Error, 详情请参阅返回码。
先决条件	无
备注	ATR_GetAcc1DEKFParamReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetAcc1DEKFParamCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_ACC_1DEKF_PARAM); void GetAcc1DEKFParamCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_GET_ACC_1DEKF_PARAM);</pre>

5.34 ATR_SetGyro1DEKFParamReq(u8 *KFP, u8 dataLen)

函数名称	ATR_SetGyro1DEKFParamReq
函数原型	ATR_SetGyro1DEKFParamReq(u8 *KFP, u8 dataLen)

行为描述	Saber 将配置外部卡尔曼滤波器参数以满足不同的场景需求。
输入参数	“KFP” is pointer which points to data; “dataLen” is the length of data “KFP” value can be explained as below: <pre>u8 reserve[4]; float Kalman_Q; float Kalman_R;</pre>
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	如果用户倾向于实现流畅的数据，建议使用高比例的 R/Q，而如果用户趋向于实现快速响应，则建议使用低比例的 R/Q。默认情况下，Q 值为 0.1，R 值为 1。 例如: <pre>unsigned char KFP[12]; KFP[0] = 0x11; KFP[1] = 0x11; KFP[2] = 0x11; KFP[3] = 0x11; KFP[4] = 0xCD; KFP[5] = 0xCC; KFP[6] = 0x4C; KFP[7] = 0x3D; KFP[8] = 0x00; KFP[9] = 0x00; KFP[10] = 0x48; KFP[11] = 0x42; ATR_SetAcc1DEKFParamReq(KFP, 12);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_GYRO_1DEKF_PARAM);</pre>

5.35 ATR_GetGyro1DEKFParamReq()

函数名称	ATR_GetGyro1DEKFParamReq
函数原型	ATR_GetGyro1DEKFParamReq()
行为描述	Saber 将返回外部卡尔曼滤波器参数的配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetGyro1DEKFParamReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback (GetGyro1DEKFParamCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_GYRO_1DEKF_PARAM); void GetGyro1DEKFParamCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>

注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_GET_GYRO_1DEKF_PARAM);</pre>
------	--

5.36 ATR_SetMag1DEKFParamReq(u8 *KFP, u8 dataLen)

函数名称	ATR_SetMag1DEKFParamReq
函数原型	ATR_SetMag1DEKFParamReq(u8 *KFP, u8 dataLen)
行为描述	Saber 将配置外部卡尔曼滤波器参数以满足不同的场景需求。
输入参数	“KFP” is pointer which points to data; “dataLen” is the length of data “KFP” value can be explained as below: <pre>U8 reserve[4]; Float Kalman_Q; Float Kalman_R;</pre>
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	如果用户倾向于实现流畅的数据，建议使用高比例的 R/Q，而如果用户趋向于实现快速响应，则建议使用低比例的 R/Q。默认情况下，Q 值为 0.05，R 值为 50。 例如: <pre>unsigned char KFP[12]; KFP[0] = 0x11; KFP[1] = 0x11; KFP[2] = 0x11; KFP[3] = 0x11; KFP[4] = 0xCD; KFP[5] = 0xCC;</pre>

	<pre> KFP[6] = 0x4C; KFP[7] = 0x3D; KFP[8] = 0x00; KFP[9] = 0x00; KFP[10] = 0x48; KFP[11] = 0x42; ATR_SetAcc1DEKFParamReq(KFP, 12); </pre>
注册回调	<pre> ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_MAG_1DEKF_PARAM); </pre>

5.37 ATR_GetMag1DEKFParamReq()

函数名称	ATR_GetMag1DEKFParamReq
函数原型	ATR_GetMag1DEKFParamReq()
行为描述	Saber 将返回外部卡尔曼滤波器参数的配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetMag1DEKFParamReq ()的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre> RegisterCallback(GetMag1DEKFParamCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_MAG_1DEKF_PARAM); void GetMag1DEKFParamCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) </pre>

	<pre> { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... } </pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_GET_MAG_1DEKF_PARAM);

5.38 ATR_SetEngineFusionModeReq(u8 *setMode, u8 dataLen)

函数名称	ATR_SetEngineFusionModeReq
函数原型	ATR_SetEngineFusionModeReq(u8 *setMode, u8 dataLen)
行为描述	Saber 将决定在引擎中选择哪种数据融合模式。
输入参数	“setMode” is pointer which points to data; “dataLen” is the length of data “setMode” value can be explained as below: <pre> enum { ATOM_EKF_SOPHISTICATE = 0, ATOM_EKF_SOPHISTICATE_MAG, ATOM_CF, ATOM_CF_MAG }; </pre>
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无

备注	例如: <pre>unsigned char setMode = 0; ATR_SetEngineFusionModeReq(&setMode, 1);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_ENGINE_FUSION_MODE);</pre>

5.39 ATR_GetEngineFusionModeReq()

函数名称	ATR_GetEngineFusionModeReq
函数原型	ATR_GetEngineFusionModeReq()
行为描述	Saber 将返回引擎模式的配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetEngineFusionModeReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetEngineFusionModeCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_ENGINE_FUSION_MODE); void GetEngineFusionModeCallback (u8 classID, u8 mID, u8 errorCode, u8* pData, u16 dataLen) { if (errorCode == RETURN_CODE_OK) {</pre>

	<pre> printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... } </pre>
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_GET_ENGINE_FUSION_MODE);

5.40 ATR_SetPacketUpdateRateReq(u8 *setRate, u8 dataLen)

函数名称	ATR_SetPacketUpdateRateReq
函数原型	ATR_SetPacketUpdateRateReq(u8 *setRate, u8 dataLen)
行为描述	Saber 将配置输出数据速率(ODR)。
输入参数	“setRate” is pointer which points to data; “dataLen” is the length of data “setRate” value can be explained as below: Valid value for Swift Mode: 0, 50, 100, 200, 400, 500, 800, 1000 Valid value for Sophisticate Mode: 0, 50, 100, 200, 400
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK, 否则返回 Error, 详情请参阅返回码。
先决条件	无

备注	如果引擎模式设置为复杂模式，则最大 ODR 为 400Hz，如果引擎模式设置为快速模式，则最大 ODR 为 1000Hz。如果用户想实现最低功耗，则可以使用 0Hz 来关闭输出数据。 例如： <pre>unsigned short setRate = 500; ATR_SetPacketUpdateRateReq((u8*)(&setRate), datalen);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_PACKET_UPDATE_RATE);</pre>

5.41 ATR_GetPacketUpdateRateReq()

函数名称	ATR_GetPacketUpdateRateReq
函数原型	ATR_GetPacketUpdateRateReq()
行为描述	Saber 将返回 ODR 的配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetPacketUpdateRateReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如：

	<pre>RegisterCallback(GetPacketUpdateRateCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_PACKET_UPDATE_RATE); void GetPacketUpdateRateCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_GET_PACKET_UPDATE_RATE);</pre>

5.42 ATR_SetGyroThresholdReq(u8 *pdata, u8 dataLen)

函数名称	ATR_SetGyroThresholdReq
函数原型	ATR_SetGyroThresholdReq(u8 *pdata, u8 dataLen)
行为描述	当陀螺仪进入数据融合引擎时，Saber 将为陀螺仪数据设置一个特殊的阈值。
输入参数	“pdata” is pointer which points to data; “dataLen” is the length of data “pdata” value can be explained as below: 4bytes for x, 4bytes for y, 4bytes for z, 1byte for enable, 3bytes reserve.
输出参数	无

返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	如果陀螺仪数据(无论是 CAL 数据还是 KAL 数据)超过阈值或小于负阈值，那么数据会被传输到引擎，否则将 0 传输到引擎。在 Saber 稳定的情况下，一旦用户启用此功能，用户将看到偏航角度保持冻结。这在一些应用场景中是十分有用的。 例如: <pre> unsigned char sendData[16]; float gyroThresValue[3] = { 2.0, 2.0, 2.0 }; memset(sendData, 0x00, 16 * sizeof(unsigned char)); //Enable/Disbale byte ThreValue[0] = 1; //Float data memcpy(&sendData[4], gyroThresValue, 3 * sizeof(float)); ATR_SETGyroThresholdReq(sendData, 16); </pre>
注册回调	<pre> ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_GYRO_THRESHOLD); </pre>

5.43 ATR_GetGyroThresholdReq()

函数名称	ATR_GetGyroThresholdReq
函数原型	ATR_GetGyroThresholdReq()
行为描述	Saber 将返回陀螺仪的阈值。
输入参数	无
输出参数	无

返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetGyroThresholdReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetGyroThresholdCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_GYRO_THRESHOLD); void GetGyroThresholdCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_GET_GYRO_THRESHOLD);</pre>

5.44 ATR_SetYawOffsetReq(u8 *data, u8 datalen)

函数名称	ATR_SetYawOffsetReq
函数原型	ATR_SetYawOffsetReq(u8 *data, u8 dataLen)

行为描述	Saber 将重置偏航角为零。设置偏航角初始位为当前位。
输入参数	u8 *data is null; u8 dataLen is 0
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如： ATR_SetYawOffsetReq(NULL, 0);
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_YAW_OFFSET);

5.45 ATR_SetAttitudeOffsetReq(u8 *data, u8 dataLen)

函数名称	ATR_SetAttitudeOffsetReq
函数原型	ATR_SetAttitudeOffsetReq(u8 *data, u8 dataLen)
行为描述	Saber 将重置翻滚角，俯仰角和偏航角为零。设置欧拉角的初始位为当前位。
输入参数	u8 *data is null; u8 dataLen is 0
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无

备注	例如: <pre>ATR_SetAttitudeOffsetReq(NULL, 0);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_ATTITUDE_OFFSET);</pre>

5.46 ATR_SetReferenceOffsetReq(u8 *data, u8 dataLen)

函数名称	ATR_SetReferenceOffsetReq
函数原型	ATR_SetReferenceOffsetReq(u8 *data, u8 dataLen)
行为描述	Saber 将重置翻滚角和俯仰角为零。设置欧拉角的初始位为当前位。
输入参数	u8 *data is null; u8 dataLen is 0
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如: <pre>ATR_SetReferenceOffsetReq(NULL, 0);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_REFERENCE_OFFSET);</pre>

5.47 ATR_ClearOffsetReq (u8 *data, u8 dataLen)

函数名称	ATR_ClearOffsetReq
函数原型	ATR_ClearOffsetReq(u8 *data, u8 dataLen)
行为描述	清空所有对于初始位的设置。
输入参数	“data” is 1; “dataLen” is 1 “pdata” value can be explained as below: 1: clear last offset
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如: <pre>unsigned char data = 1; ATR_ClearOffsetReq(&data, 1);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_CLEAR_OFFSET);</pre>

5.48 ATR_SetComPortParamReq(u8 *para, u8 datalen)

函数名称	ATR_SetComPortParamReq
函数原型	ATR_SetComPortParamReq(u8 *para, u8 datalen)
行为描述	根据“para”，Saber 将配置 Uart 接口。
输入参数	“para[0]” is baud rate; “para[1]” is data bit length;

	“para[2]” is stop bit length; “para[3]” is parity; “para[4]” is flow control; “datalen” = 5; “para[0]” value can be explained as below: <table> <tr><td>BAUDRATE_9600</td><td>0x01</td></tr> <tr><td>BAUDRATE_38400</td><td>0x02</td></tr> <tr><td>BAUDRATE_57600</td><td>0x03</td></tr> <tr><td>BAUDRATE_115200</td><td>0x04</td></tr> <tr><td>BAUDRATE_230400</td><td>0x05</td></tr> <tr><td>BAUDRATE_460800</td><td>0x06</td></tr> <tr><td>BAUDRATE_921600</td><td>0x07</td></tr> <tr><td>BAUDRATE_1500000</td><td>0x08</td></tr> </table> “para[1]” value can be explained as below: It is mandatory to be 8, means 8 bits data. “para[2]” value can be explained as below: <table> <tr><td>STOP_BIT_1</td><td>0x01</td></tr> <tr><td>STOP_BIT_2</td><td>0x02</td></tr> </table> “para[3]” value can be explained as below: <table> <tr><td>PARITY_NONE</td><td>0x01</td></tr> <tr><td>PARITY_EVEN</td><td>0x02</td></tr> <tr><td>PARITY_ODD</td><td>0x03</td></tr> </table> “para[4]” value can be explained as below: <table> <tr><td>FLOWCONTROL_NONE</td><td>0x01</td></tr> <tr><td>FLOWCONTROL_RTS_CTS</td><td>0x02</td></tr> </table>	BAUDRATE_9600	0x01	BAUDRATE_38400	0x02	BAUDRATE_57600	0x03	BAUDRATE_115200	0x04	BAUDRATE_230400	0x05	BAUDRATE_460800	0x06	BAUDRATE_921600	0x07	BAUDRATE_1500000	0x08	STOP_BIT_1	0x01	STOP_BIT_2	0x02	PARITY_NONE	0x01	PARITY_EVEN	0x02	PARITY_ODD	0x03	FLOWCONTROL_NONE	0x01	FLOWCONTROL_RTS_CTS	0x02
BAUDRATE_9600	0x01																														
BAUDRATE_38400	0x02																														
BAUDRATE_57600	0x03																														
BAUDRATE_115200	0x04																														
BAUDRATE_230400	0x05																														
BAUDRATE_460800	0x06																														
BAUDRATE_921600	0x07																														
BAUDRATE_1500000	0x08																														
STOP_BIT_1	0x01																														
STOP_BIT_2	0x02																														
PARITY_NONE	0x01																														
PARITY_EVEN	0x02																														
PARITY_ODD	0x03																														
FLOWCONTROL_NONE	0x01																														
FLOWCONTROL_RTS_CTS	0x02																														
输出参数	无																														
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。																														
先决条件	无																														
备注	例如:																														

	<pre> unsigned char para[5]; Para[0] = 0x04; Para[1] = 0x01; Para[2] = 0x01; Para[3] = 0x01; Para[4] = 0x01; ATR_SetComPortParamReq(Para, 5); </pre> 需要在使用该 API 之后的 3s 内使用 ATR_SetComPortParamHandshakeReq 来进行握手，确认波特率改变成功，否则设备将会恢复为原波特率。
注册回调	<pre> ATR_RegisterCallback(xxx_callback_function, CLASS_ID_COMMUNICATION_CONFIG, CMD_ID_SET_COMPORT_PARAM); </pre>

5.49 ATR_GetComPortParamReq()

函数名称	ATR_GetComPortParamReq
函数原型	ATR_GetComPortParamReq()
行为描述	Saber 将返回 uart 接口配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetComPortParamReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如： <pre> RegisterCallback(GetComPortParamCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_COMPORT_PARAM); </pre>

	<pre>void GetComPortParamCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_COMMUNICATION_CONFIG, CMD_ID_GET_COMPORT_PARAM);</pre>

5.50 ATR_SetComPortParamHandshakeReq()

函数名称	ATR_SetComPortParamHandshakeReq
函数原型	ATR_SetComPortParamHandshakeReq()
行为描述	该 API 应用于更改波特率情况下与主机通信的实际场景中。
输入参数	None
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	用户想更改波特率时，需要从 Host 发送带有先前波特率的第一个 API ATR_SetComPortParamReq ，然后使用带有新波特率的第二个 API

	ATR_SetComPortParamHandshakeReq 作为与 Host 的通信握手。一旦主机和 Saber 成功握手，就意味着波特率已成功改变。更多细节将在单独的程序备注中给出。 例如： <pre>ATR_SetComPortParamReq();</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_COMMUNICATION_CONFIG, CMD_ID_SET_COMPORT_PARAM_HANDSHAKE);</pre>

5.51 ATR_SetDrdyConfigReq(u8* pData, u8 dataLen)

函数名称	ATR_SetDrdyConfigReq										
函数原型	ATR_SetDrdyConfigReq(u8 *pData, u8 dataLen)										
行为描述	Saber 将打开或关闭数据准备(DRDY)功能。										
输入参数	“pData” is pointer which points to data; “dataLen” is the length of data; “pData” value can be explained as below: <table border="1"> <thead> <tr> <th>Bit Num</th><th>规格说明</th></tr> </thead> <tbody> <tr> <td>Bit 15</td><td>0:DRDY disable 1:DRDY enable</td></tr> <tr> <td>Bit 14:2</td><td>Reserve</td></tr> <tr> <td>Bit 1</td><td>OTYPE:Output type of DRDY pin 0:Push/pull 1:Open drain</td></tr> <tr> <td>Bit 0</td><td>POL:Polarity of DRDY signal 0:Active high 1: Active low</td></tr> </tbody> </table>	Bit Num	规格说明	Bit 15	0:DRDY disable 1:DRDY enable	Bit 14:2	Reserve	Bit 1	OTYPE:Output type of DRDY pin 0:Push/pull 1:Open drain	Bit 0	POL:Polarity of DRDY signal 0:Active high 1: Active low
Bit Num	规格说明										
Bit 15	0:DRDY disable 1:DRDY enable										
Bit 14:2	Reserve										
Bit 1	OTYPE:Output type of DRDY pin 0:Push/pull 1:Open drain										
Bit 0	POL:Polarity of DRDY signal 0:Active high 1: Active low										
输出参数	无										

返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	在 IIC 或 SPI 通信期间 DRDY 非常有用，它可以在 Saber 缓冲器准备好数据时通知主机。 例如： <pre>unsigned short drdy = 0; drdy = ((1 << 15) (1 << 1) (1 << 0)); ATR_SetDrdyConfigReq((u8 *)drdy, 2);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_SET_DRDY_CONFIG);</pre>

5.52 ATR_GetDrdyConfigReq()

函数名称	ATR_GetDrdyConfigReq
函数原型	ATR_GetDrdyConfigReq()
行为描述	Saber 将返回 DRDY 的配置。
输入参数	无;
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetDrdyConfigReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。

	例如: <pre>RegisterCallback(GetDrdyConfigCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_DRDY_CONFIG); void GetDrdyConfigCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode == RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_GET_DRDY_CONFIG);</pre>

5.53 ATR_SetGpioConfigReq(u8* pData, u8 dataLen)

函数名称	ATR_SetGpioConfigReq									
函数原型	ATR_SetGpioConfigReq(u8* pData, u8 dataLen)									
行为描述	Saber 将配置 GPIO 功能。									
输入参数	“pData” is pointer which points to data; “dataLen” is the length of data; “pData” value can be explained as below: <table><tr><th>Bit Num</th><th>规格说明</th><th>模式</th></tr><tr><td>Bit 15:3</td><td>Reserve</td><td></td></tr><tr><td>Bit 2</td><td>Data transmission</td><td>Real time toggle</td></tr></table>	Bit Num	规格说明	模式	Bit 15:3	Reserve		Bit 2	Data transmission	Real time toggle
Bit Num	规格说明	模式								
Bit 15:3	Reserve									
Bit 2	Data transmission	Real time toggle								

	Bit 1	Breath shining	模式 1	
	Bit 0	POL:GPIO Pin Function Select		
输出参数	无			
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。			
先决条件	无			
备注	Bit 0 用于选择 GPIO 模式。 如果选择 LED 模式，则 Bit 1 或 Bit 2 将决定相应的 LED 闪光灯功能。一旦呼吸灯启动，LED 将以每秒一次的模式 1 波形闪烁。在 LED 模式下，GPIO 被配置为输出 IO 口。 例如： <pre>unsigned char gpio[2] = {0x06,0x00}; ATR_SetGpioConfigReq(gpio,2);</pre>			
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_SET_GPIO_CONFIG);</pre>			

5.54 ATR_GetGpioConfigReq()

函数名称	ATR_GetGpioConfigReq
函数原型	ATR_GetGpioConfigReq()
行为描述	Saber 将返回 GPIO 配置。
输入参数	无;
输出参数	无

返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetGpioConfigReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetGpioConfigCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_GPIO_CONFIG); void GetGpioConfigCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_GET_GPIO_CONFIG);</pre>

5.55 ATR_SetSyncOutConfigReq(u8 *pData, u8 dataLen)

函数名称	ATR_SetSyncOutConfigReq
函数原型	ATR_SetSyncOutConfigReq(u8 *pData, u8 dataLen)
行为描述	设置同步信号输出配置。

输入参数	“pData” is pointer which points to data; “dataLen” is the length of data “pData” value can be explained as below: <table> <tr> <th>Byte Num</th><th>规格说明</th></tr> <tr> <td>Byte0</td><td>bit0: Enable(1) or Disable(0) SynCout function bit1-7:reserved</td></tr> <tr> <td>Byte1</td><td>Polarity :The polarity of the sync pulse.One of:Rising Edge(0x01),Falling Edge(0x02);Positive Pulse (0x03), Negative Pulse (0x04).</td></tr> <tr> <td>Byte2-byte3</td><td>Offset:Offset from event to pulse generation (100μs units, range [-30000..+30000]).</td></tr> <tr> <td>Byte4-byte5</td><td>Skip Factor: The number of events to skip after taking the action before taking action again.</td></tr> <tr> <td>Byte6-byte7</td><td>PulseWidth: The width of the generated pulse in 100μs units.</td></tr> </table>	Byte Num	规格说明	Byte0	bit0: Enable(1) or Disable(0) SynCout function bit1-7:reserved	Byte1	Polarity :The polarity of the sync pulse.One of:Rising Edge(0x01),Falling Edge(0x02);Positive Pulse (0x03), Negative Pulse (0x04).	Byte2-byte3	Offset:Offset from event to pulse generation (100μs units, range [-30000..+30000]).	Byte4-byte5	Skip Factor: The number of events to skip after taking the action before taking action again.	Byte6-byte7	PulseWidth: The width of the generated pulse in 100μs units.
Byte Num	规格说明												
Byte0	bit0: Enable(1) or Disable(0) SynCout function bit1-7:reserved												
Byte1	Polarity :The polarity of the sync pulse.One of:Rising Edge(0x01),Falling Edge(0x02);Positive Pulse (0x03), Negative Pulse (0x04).												
Byte2-byte3	Offset:Offset from event to pulse generation (100μs units, range [-30000..+30000]).												
Byte4-byte5	Skip Factor: The number of events to skip after taking the action before taking action again.												
Byte6-byte7	PulseWidth: The width of the generated pulse in 100μs units.												
输出参数	无												
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。												
先决条件	无												
备注	Payload Length = 8												
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_SET_SYNC_OUT_CFG);												

5.56 ATR_GetSyncOutConfigReq()

函数名称	ATR_GetSyncOutConfigReq
函数原型	ATR_GetSyncOutConfigReq()
行为描述	获取同步信号输出的配置
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	无
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_GET_SYNC_OUT_CFG);

5.57 ATR_SetDataPacketConfigReq(u8* pData, u8 dataLen)

函数名称	ATR_SetDataPacketConfigReq
函数原型	ATR_SetDataPacketConfigReq(u8* pData, u8 dataLen)
行为描述	设置 Saber 数据包，请参考示例代码或用户手册。
输入参数	“pData” is pointer which points to data; “dataLen” is the length of data “pData” please refer to “4.6Saber 数据配置结构”

输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	Payload Length = 4*N
注册回调	ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_SET_DATA_PAC_CFG);

5.58 ATR_GetDataPacketConfigReq()

函数名称	ATR_GetDataPacketConfigReq
函数原型	ATR_GetDataPacketConfigReq()
行为描述	Saber 将返回数据包配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	ATR_GetDataPacketConfigReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如：

	<pre>RegisterCallback(GetDataPacketConfigCallback, CLASS_ID_DEVICE_INFO, CMD_ID_GET_DATA_PAC_CFG); void GetDataPacketConfigCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode = RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_HOSTCONTROL_CMD, CMD_ID_GET_DATA_PAC_CFG);</pre>

5.59 ATR_RegisterDownLoadCallback(DownloadCallbackFunction pFunction)

函数名称	ATR_RegisterDownLoadCallback
函数原型	ATR_RegisterDownLoadCallback(DownloadCallbackFunction pFunction)
行为描述	注册 firmware 下载的回调函数
输入参数	DownloadCallbackFunction pFunction = user defined function
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK, 否则返回 RETURN_CODE_ERROR_DOWNLOAD,详情请参阅返回码。
先决条件	无

备注	当下载 firmware 完成后，会调用 pFunction 函数，通过 pFunction 的函数参数来判断是否下载成功。 Example: <pre>void DownloadFWCallback(unsigned char para) { if (para == RETURN_CODE_OK) //User code else if (para == RETURN_CODE_ERROR_DOWNLOAD) //User code } ATR_RegisterDownLoadCallback(DownloadFWCallback); ATR_DownloadFirmware("xxx.bin");</pre>
注册回调	无

5.60 ATR_DownloadFirmware_new(char *filename)

函数名称	ATR_DownloadFirmware_new
函数原型	ATR_DownloadFirmware_new(char *filename)
行为描述	固件更新
输入参数	Firmware.bin path
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如: <pre>ATR_DownloadFirmware_new(C:\XXX\XX\XX.Bin);</pre>
注册回调	无

5.61 ATR_GetVersion(char *version_Buf)

函数名称	ATR_GetVersion
函数原型	ATR_GetVersion(char *version_Buf)
行为描述	获得 SDK 版本。
输入参数	无
输出参数	Version_Buf is Used to accept version numbers
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无
备注	例如： <pre>unsigned char sdk_version[10] = {0}; ATR_GetVersion(sdk_version);</pre>
注册回调	无

5.62 ATR_DebugSwitch(bool onOrOff)

函数名称	ATR_DebugSwitch
函数原型	ATR_DebugSwitch(bool onOrOff)
行为描述	关闭或打开打印日志信息。
输入参数	onOrOff is true/false
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码。
先决条件	无

备注	例如: <code>ATR_DebugSwitch(true);</code>
注册回调	无

5.63 ATR_GetScenarioReq()

函数名称	ATR_GetScenarioReq
函数原型	ATR_GetScenarioReq()
行为描述	Saber 将反馈 2D/3D 场景配置。
输入参数	无
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码“8 Saber 返回码”。
先决条件	无
备注	ATR_GetScenarioReq 的响应是命令 ACK 的有效载荷。 用户应该注册一个回调函数来处理响应消息。 例如: <pre>RegisterCallback(GetScenarioCallback, CLASS_ID_ALGORITHMENGINE, CMD_ID_GET_SCENARIO); void GetScenarioCallback (u8 classID, u8 mID, u8 errorCode,u8* pData, u16 dataLen) { if (errorCode == RETURN_CODE_OK) { printf("The code is right!"); } else { printf("The errorCode is %d!",errorCode); } }</pre>

	<pre>printf("This is class %d, msgID %d.\n", classID, mID); //process pData here.... }</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENG, CMD_ID_GET_SCENARIO);</pre>

5.64 ATR_SetScenarioReq(u8* pData, u8 dataLen)

函数名称	ATR_SetScenarioReq
函数原型	ATR_SetScenarioReq (u8 *pData, u8 dataLen)
行为描述	收到此消息后，Saber 将配置 2D/3D 模式。
输入参数	“pData” is pointer which points to data; “dataLen” is the length of data “pData” value can be explained as below: <pre>0: SCENARIO_MODE_3D 1: SCENARIO_MODE_2D</pre>
输出参数	无
返回值	如果一切正常则返回 RETURN_CODE_OK，否则返回 Error，详情请参阅返回码“8 Saber 返回码”。
先决条件	无
备注	例如: <pre>unsigned char _mode = 1; ATR_SetScenarioReq(&_mode, 1);</pre>
注册回调	<pre>ATR_RegisterCallback(xxx_callback_function, CLASS_ID_ALGORITHMENGINE, CMD_ID_SET_SCENARIO);</pre>

6 底层通信协议接口

6.1 概述

要实现完全控制、最大灵活性和/或具有严格的实时性能要求，实现直接与 **Saber** 接口进行连接通信。**Saber** 的低功耗嵌入式固件可执行所有计算或校准操作，用户只需使用 **Atom** 通信协议以轮询模式或流模式从主机接口检索数据即可。详情请参考示例代码。

所有在主机和设备之间传输的消息将在下面这一节中进行描述。

6.2 操作类

6.2.1 WakeUpHost

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x01	0			0x6d	DEVICE	HOST

设备复位后，它会自动发送 **WakeUpHost** 消息。在设备发送消息 1 秒内，主机可以发送 **WakeUpHostAck** 消息，允许设备进入 **CONFIG** 模式；如果设备没有收到来自主机的确认，它就会进入 **MEASURE** 模式来发送数据包。

6.2.2 WakeUpHostAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x81	0			0x6d	HOST	DEVICE

该消息是对 **WakeUpHost** 消息的确认。

6.2.3 SwitchToConfigMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x02	0			0x6d	HOST	DEVICE

当设备收到此消息，它将更改为 **CONFIG** 模式。有关 **CONFIG** 模式的详细信息，请参阅“3.2 设备系统状态”章节。

6.2.4 SwitchToConfigModeAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x82	0			0x6d	DEVICE	HOST

该消息是对 **SwitchToConfigMode** 的确认。

6.2.5 SwitchToMeasureMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x03	0			0x6d	HOST	DEVICE

当设备收到此消息，它将更改为 **MEASURE** 模式。有关 **MEASURE** 模式的详细信息，请参阅“3.2 设备系统状态”章节。

6.2.6 SwitchToMeasureModeAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x83	0			0x6d	DEVICE	HOST

该消息是对 **SwitchToMeasureMode** 的确认。

6.2.7 SystemReset

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x04	0			0x6d	HOST	DEVICE

请求 Saber 进入软复位。

6.2.8 SystemResetAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x84	0			0x6d	DEVICE	HOST

此消息是重置前对 **SystemReset** 的确认。

6.2.9 ConfigurationWriteToFlash

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x08	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将用户配置保存到 flash 中。

6.2.10 ConfigurationWriteToFlashAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x88	0			0x6d	DEVICE	HOST

此消息是对 **ConfigurationWriteToFlash** 的确认。

6.2.11 ConfigurationLoadFromFlash

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x09	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将从 flash 中检索用户配置。

6.2.12 ConfigurationLoadFromFlashAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x89	0			0x6d	DEVICE	HOST

此消息是对 **ConfigurationLoadFromFlash** 的确认。

6.2.13 LoadFactoryConfiguration

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x0A	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将重新加载工厂配置。

6.2.14 LoadFactoryConfigurationAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x01	0x8A	0			0x6d	DEVICE	HOST

此消息是对 **LoadFactoryConfiguratioin** 的回复。

6.3 设备信息类

6.3.1 GetFWVersion

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x01	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将发送固件版本信息。

6.3.2 RespFWVersion

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x81	32			0x6d	DEVICE	HOST

对 **GetFWVersion** 命令的回应。

有效载荷是一个类似于“xxx.xxx.xxx.20161107.172509.xxxx”的字符串，它包含 32 个字节。

6.3.3 GetProductVersion

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x03	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将发送产品版本信息。

6.3.4 RespProductVersion

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x83	32			0x6d	DEVICE	HOST

对 **GetProductVersion** 命令的回应。

6.3.5 GetVendorInfo

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x04	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将发送供应商信息。

6.3.6 RespVendorInfo

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x84	32			0x6d	DEVICE	HOST

对 **GetVendorInfo** 命令的回应。

有效载荷是一个字符串：AtomRobotics。

6.3.7 GetAllStatus

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x06	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将发送所有状态信息，这使得用户了解了 Saber 的当前状态。

6.3.8 RespAllStatus

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x86	4			0x6d	DEVICE	HOST

对 **GetAllStatus** 命令的回应。

有效载荷描述如下：(以下的状态字定义仅适用于 Saber Firmware 00.07.620 (含 620)之后的版本，如果使用的时旧版本的 firmware，请参考前一个版本的 SDK user manual)

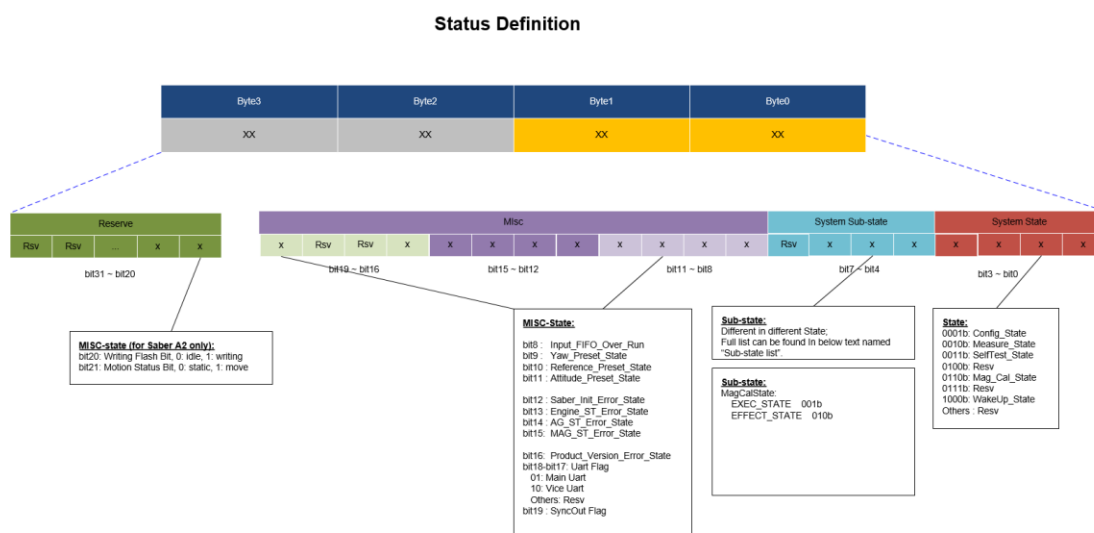


Figure 11 – Saber 状态定义

Bit0~Bit3 代表当前系统状态;

Bit4~Bit6 代表系统子状态, Bit7 预留。

Bit8~Bit19 代表一些其他状态, Saber 进入某状态时, 相应位将被置位。

System_state:

CONFIG_State: 用户可以配置 Saber 的状态

MEASURE_State: 与主机进行测量数据传输的状态

SELFTEST_State: Saber 自我检测状态

MAG_CAL_State: 磁力计校准状态

WAKEUP_State: 系统上电后的唤醒状态

System_Sub_State:

MAG_CAL_State:

EXEC_STATE 001b 执行校准状态

EFFECT_STATE 010b 生效状态

Misc_State:

Input_Overrun_Flag: 用户输入超出上限

Yaw_Preset_State: 命令 **SetYawOffset** 被触发

Reference_Preset_State: 命令 **SetReferenceOffset** 被触发

Attitude_Preset_State: 命令 **SetAttitudeOffset** 被触发

Sensor_Init_Error_State: 传感器初始化错误比特位, 1 为错误, 0 为正常。当发生错误时, 系统会进入 **CONFIG** 模式, 此模式下 Saber 只可以响应 **GetAllStatus/SystemReset** 两个命令。用户可以通过失败恢复流程让 Saber 重新初始化恢复正常。

Engine_ST_Error_State: 算法自检状态比特位, 1 为自检失败, 0 为自检成功, 当发生错误时, 系统会进入 **CONFIG** 模式, 此模式下 Saber 只可以响应 **GetAllStatus/SystemReset** 两个命令。用户可以通过失败恢复流程让 Saber 重新初始化恢复正常。

AG_ST_Error_State: 加速度计/陀螺仪自检状态比特位, 1 为自检失败, 0 为自检成功, 当发生错误时, 系统会进入 **CONFIG** 模式, 此模式下 Saber 只可以响应 **GetAllStatus/SystemReset** 两个命令。用户可以通过失败恢复流程让 Saber 重新初始化恢复正常。

Mag_ST_Error_State: 磁力计自检状态比特位，1 为自检失败，0 为自检成功，当发生错误时，系统会记录状态位信息，但是会继续运行。

Product_Version_Error_State: 这一状态比特位用于检测运行的 **firmware** 是否和硬件不匹配，1 为匹配失败，0 为匹配成功，当发生错误时，系统会记录状态位信息，同时进入错误处理状态。此时用户需要重新更新与硬件相匹配的 **firmware** 文件，系统才会恢复正常。

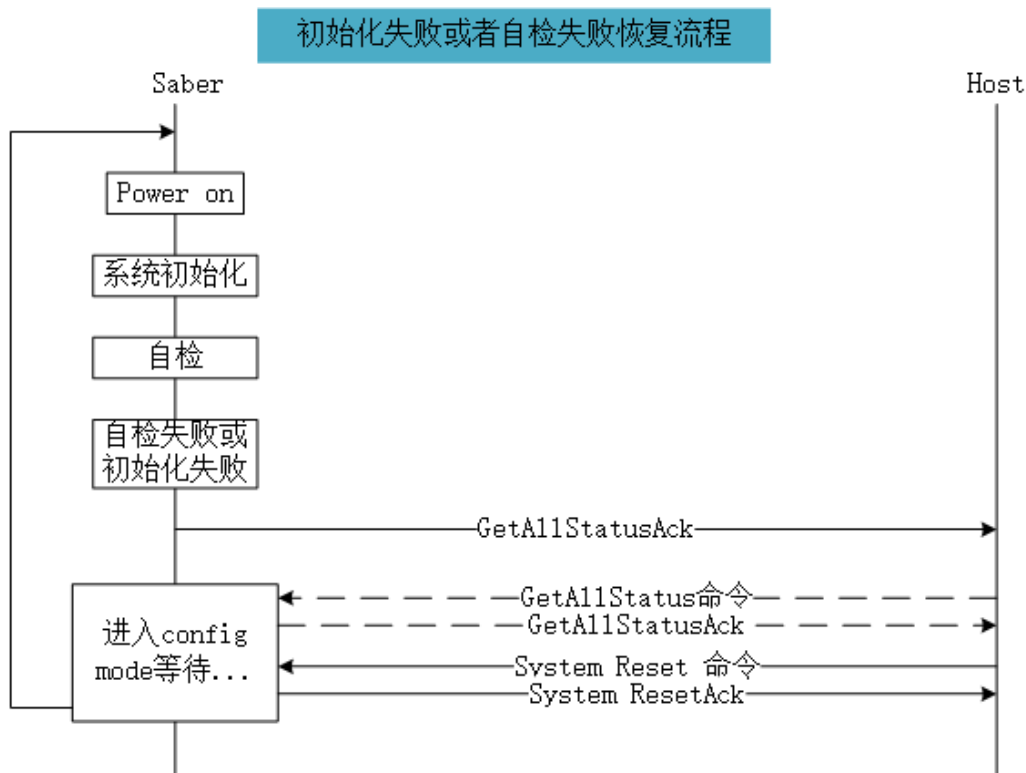


Figure 12 – 初始化与自检恢复流程

6.3.9 GetDeviceInfo

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x07	0			0x6d	HOST	DEVICE

当手打此消息，Saber 会发送所有设备信息，此消息可以让用户获取所有信息，包括：状态信息，固件版本，产品版本，产品 ID，供应商信息和引擎版本。

6.3.10 RespDeviceInfo

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x87	182			0x6d	DEVICE	HOST

对 **GetDeviceInfo** 命令的回应。

有效载荷描述如下：

```
u32 status;
u8 Fwversion[32];
u8 ProductVersion[32];
u8 ProductID[32];
u8 VenderInfo[32];
u8 EngineVersion[50];
```

6.3.11 SetModuleID

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x0C	1			0x6d	HOST	DEVICE

当收到此消息，Saber 会对模块进行编号配置，这可以让用户通过专用模组 ID 序列化多个 Saber 模组。

有效载荷可以是任意数字。

6.3.12 ModuleIDAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x8C	1			0x6d	DEVICE	HOST

此消息是对 **SetModuleID** 的确认。

有效载荷是用户设置的数字。

6.3.13 GetModuleID

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x0D	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将回复模组 ID。

6.3.14 RespModuleID

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x02	0x8D	1			0x6d	DEVICE	HOST

对 **GetModuleID** 命令的回应。

有效载荷是用户设置的数字。

6.4 传感器配置类

6.4.1 SetAccelerometerScale

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x01	1			0x6d	HOST	DEVICE

收到此消息，Saber 将根据消息的有效载荷配置加速度计的量程。

有效载荷值描述如下：

Selection	Acc Scale
0	±2G
1	±4G
2	±8G
3	±16G

6.4.2 AccelerometerScaleAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x81	1			0x6d	DEVICE	HOST

此消息是对 **SetAccelerometerScale** 的确认。

有效载荷与 **SetAccelerometerScale** 消息相同。

6.4.3 GetAccelerometerScale

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x02	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将回复加速度计量程。

6.4.4 RespAccelerometerScale

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x82	1			0x6d	DEVICE	HOST

对 **GetAccelerometerScale** 命令的回应。

有效载荷描述如下：

Selection	Acc Scale
0	±2G
1	±4G
2	±8G
3	±16G

6.4.5 SetAccelerometerCoeff

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x03	48			0x6d	HOST	DEVICE

当收到此消息，Saber 将配置加速度计校准系数。请注意，使用此消息应非常谨慎，由于它会进一步刷新校准系数以影响加速度计的准确性，因此除非用户精准理解系数并保证参数的正确性，否则我们不建议用户发送该消息。

有效载荷值可以是：

校准系数矩阵

有效载荷:[Bx, Sx, Exy, Exz, By, Eyx, Sy, Eyz, Bz, Ezx, Ezy, Sz]

全部元素均为浮点型

校准公式如下：

$$\begin{pmatrix} X' \\ Y' \\ Z' \end{pmatrix} = \begin{pmatrix} Sx & Exy & Exz \\ Eyx & Sy & Eyz \\ Ezx & Ezy & Sz \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} + \begin{pmatrix} Bx \\ By \\ Bz \end{pmatrix}$$

X,Y,Z代表原始数据。X',Y',Z'代表校准后数据。Sx,Sy,Sz,Exy,Exz,Eyx,Eyz,Ezx,Ezy 是非线性校准系数，Bx,By,Bz 是线性校准系数。

6.4.6 AccelerometerCoeffAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x83	48			0x6d	DEVICE	HOST

此消息是对 **SETAccelerometerCoeff** 的确认。

有效载荷与请求消息相同。

6.4.7 GetAccelerometerCoeff

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x04	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将回复加速度计校准系数。

6.4.8 RespAccelerometerCoeff

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x84	48			0x6d	DEVICE	HOST

对 **GetAccelerometerCoeff** 命令的回应。

有效载荷值可以是：

校准系数矩阵

有效载荷:[Bx, Sx, Exy, Exz, By, Eyx, Sy, Eyz, Bz, Ezx, Ezy, Sz]

全部元素均为浮点型

校准公式如下：

$$\begin{pmatrix} X' \\ Y' \\ Z' \end{pmatrix} = \begin{pmatrix} Sx & Exy & Exz \\ Eyx & Sy & Eyz \\ Ezx & Ezy & Sz \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} + \begin{pmatrix} Bx \\ By \\ Bz \end{pmatrix}$$

X,Y,Z代表原始数据。X',Y',Z'代表校准后数据。Sx,Sy,Sz,Exy,Exz,Eyx,Eyz,Ezx,Ezy 是非线性校准系数，Bx,By,Bz 是线性校准系数。

6.4.9 SetGyroscopeScale

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x06	1			0x6d	HOST	DEVICE

当收到此消息，Saber 将根据下表对陀螺仪的量程进行配置。

有效载荷描述如下：

Selection	Gyro Scale	Remark
-----------	------------	--------

0	±250	Unit:°/sec
1	±500	Unit:°/sec
2	±1000	Unit:°/sec
3	±2000	Unit:°/sec

6.4.10 GyroscopeScaleAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x86	1			0x6d	DEVICE	HOST

此消息是对 **SetGyroscopeScale** 的确认。

有效载荷与请求消息相同。

6.4.11 GetGyroscopeScale

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x07	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将根据下表回复陀螺仪的量程。

有效载荷描述如下：

Selection	Gyro Scale	Remark
0	±250	Unit:°/sec
1	±500	Unit:°/sec
2	±1000	Unit:°/sec
3	±2000	Unit:°/sec

6.4.12 RespGyroscopeScale

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x87	1			0x6d	DEVICE	HOST

对 **GetGyroscopeScale** 命令的回应。

有效载荷与请求消息相同。

6.4.13 SetMagnetometerCoeff

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message Checksum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x0D	48			0x6d	HOST	DEVICE

收到此消息时，Saber 将配置磁力计校准系数。请注意，使用此消息应非常谨慎，由于它会进一步刷新校准系数以影响磁力计的准确性，因此除非用户精准理解系数并保证参数的正确性，否则我们不建议用户发送该消息。

有效载荷值可以是：

校准参数：

有效载荷:float G[3][3],float H0[3];

校准公式如下：

$$\begin{pmatrix} X' \\ Y' \\ Z' \end{pmatrix} = \begin{pmatrix} G[0][0] & G[0][1] & G[0][2] \\ G[1][0] & G[1][1] & G[1][2] \\ G[2][0] & G[2][1] & G[2][2] \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} + \begin{pmatrix} H0[0] \\ H0[1] \\ H0[2] \end{pmatrix}$$

X,Y,Z代表原始数据。X',Y',Z'代表校准后数据。G[3][3]是非线性校准系数，H0 是线性校准系数。

6.4.14 MagnetometerCoeffAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message Checksum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x8D	48			0x6d	DEVICE	HOST

此消息是对 **SetMagnetometerCoeff** 的确认。

有效载荷与请求消息相同。

6.4.15 GetMagnetometerCoeff

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message Checksum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x0E	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回磁力计校准系数。

6.4.16 RespMagnetometerCoeff

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message Checksum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x8E	48			0x6d	DEVICE	HOST

对 **GetMagnetometerCoeff** 命令的回应。

有效载荷值可以是：

校准参数：

有效载荷:float G[3][3],float H0[3];

校准公式如下：

$$\begin{pmatrix} X' \\ Y' \\ Z' \end{pmatrix} = \begin{pmatrix} G[0][0] & G[0][1] & G[0][2] \\ G[1][0] & G[1][1] & G[1][2] \\ G[2][0] & G[2][1] & G[2][2] \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} + \begin{pmatrix} H0[0] \\ H0[1] \\ H0[2] \end{pmatrix}$$

X,Y,Z代表原始数据。X',Y',Z'代表校准后数据。G[3][3]是非线性校准系数，H0 是线性校准系数。

6.4.17 SetMagnetometerCalibrate

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x0F	1			0x6d	HOST	DEVICE

当收到此消息，Saber 将根据有效载荷对磁力计进行校准。校准程序如下：

1. 用户需要将模组放置在正确的位置，然后发送带有效载荷参数 1 的命令。
2. 用户需要旋转模组直到它完成感应数据。详情请参阅 Saber Xplore 的相同程序。
3. 一旦准备好所有的必要数据，Saber 将自动计算磁力计校准矩阵并记录到存储器中。
4. 用户需要发送带有效载荷参数 8 的命令将校准系数有效写入 RAM。需要注意的是，必须发送命令“6.2.9 ConfigurationWriteToFlash”将校准参数写入 flash，否则在下次复位期间校准参数将丢失。

有效载荷数据用来描述运行校准时的详细含义，它可以是以下值：

0x09: EIGHT_SHAPED_CALIBRATE_START,

0x0A: SIX_FACE_CALIBRATE_START,

0x0B: CALIBRATE_STOP,

0x0C: TAKE_EFFECT

6.4.18 MagnetometerCalibrateAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x8F	1			0x6d	DEVICE	HOST

此消息是对 **SetMagnetometerCalibrate** 的确认。

有效载荷描述如下：

0x09: EIGHT_SHAPED_CALIBRATE_START,

0x0A: SIX_FACE_CALIBRATE_START,

0x0B: CALIBRATE_STOP,

0x0C: TAKE_EFFECT

6.4.19 MagCalPacket

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0xB0	32			0x6d	HOST	DEVICE

用户进行磁校准时，会收到此信息，该消息将传送一组数据到主机，包括:数字编号、经卡尔曼滤波后的磁力计数据和当前欧拉角。

有效载荷描述如下：

short num, unsigned char reserve[2], float magkal_x, float magkal_y, float magkal_z, float roll, float yaw, float pitch.reserve 共 32 字节。

数字编号是理解校准算法是否获取正确数据的关键。总的来说，Saber 需要 24 个点来校准。如果数字编号是 0，就意味着这个包不满足磁校准采样标准，所以这个包将被 Saber 放弃，但是该包仍然可以反映当前 Saber 的状态和方向。如果数字为非零(0<数字<24)，则意味着这个包对应 24 个点中的一个，那么它将用于校准磁力计。Saber 计算校准系数，直到所有 24 个点都准备好，用户要触发校准计算，需要消息

SetMagnetometerCalibrate，并且有效载荷等于 1 即可。

6.4.20 MagCalMessage

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0xB1	4			0x6d	HOST	DEVICE

用户进行磁校准时，会收到此信息，该消息的内容显示当前磁校准所处状态。

有效载荷描述如下：

START	0x00
COMPLETE	0x01
FAIL	0x02
ABORT	0x03
REDO	0x04

6.4.21 SetMux2

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x12	2			0x6d	HOST	DEVICE

当收到此消息，Saber 将决定是校准数据还是卡尔曼数据输入 Boson 引擎，注意：用户必须将正确的配置发送给模组，如果错误，有效载荷将不会生效。加速度计，陀螺仪和磁力计可以单独设置。

有效载荷描述如下：

保留第一个字节，第二个字节中(Acc 占用 bit0-1, Gyro 占用 bit2-3, Mag 占用 bit4-5, 其他位保留)。

“CAL”表示数据已经通过 Saber 算法进行校准和补偿，“KAL”表示数据是从“CAL”数据基础上经卡尔曼滤波器导出。卡尔曼滤波器参数可以进行微调，详情请参阅“6.5.1 SetACC1DEKFParam”,“6.5.5 SetGYRO1DEKFParam”和“6.5.9SetMAG1DEKFParam”。

Style	reserve	reserve	Mag(bit5)	Mag(bit4)	Gyro(bit3)	Gyro(bit2)	Acc(bit1)	Acc(bit0)
NONE			0	0	0	0	0	0
Default(CAL)			0	1	0	1	0	1
CAL			1	0	1	0	1	0
KAL			1	1	1	1	1	1

6.4.22 Mux2Ack

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x92	2			0x6d	DEVICE	HOST

此消息是对 **SetMUX2** 的确认。

有效载荷描述如下：

保留第一个字节，第二个字节中(Acc 占用 bit0-1, Gyro 占用 bit2-3, Mag 占用 bit4-5, 其他位保留)。

“CAL”表示数据已经通过 **saber** 算法进行校准和补偿，“KAL”表示数据是从“CAL”数据基础上经卡尔曼滤波器导出。卡尔曼滤波器参数可以进行微调，详情请参阅“6.5.1 SetACC1DEKFParam”,“6.5.5 SetGYRO1DEKFParam”和“6.5.9SetMAG1DEKFParam”。

Style	reserve	reserve	Mag(bit5)	Mag(bit4)	Gyro(bit3)	Gyro(bit2)	Acc(bit1)	Acc(bit0)
NONE			0	0	0	0	0	0
Default(CAL)			0	1	0	1	0	1
CAL			1	0	1	0	1	0
KAL			1	1	1	1	1	1

6.4.23 GetMux2

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x13	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将回复当前配置，显示出是校准数据还是卡尔曼数据进入了 Boson 引擎。

6.4.24 RespMux2

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x03	0x93	2			0x6d	DEVICE	HOST

对 **GetMux2** 命令的回应。

有效载荷描述如下：

保留第一个字节，第二个字节中(Acc 占用 bit0-1, Gyro 占用 bit2-3, Mag 占用 bit4-5, 其他位保留)。

“CAL”表示数据已经通过 **saber** 算法进行校准和补偿，“KAL”表示数据是从“CAL”数据基础上经卡尔曼滤波器导出。卡尔曼滤波器参数可以进行微调，详情请参阅“6.5.1 SetACC1DEKFParam”,“6.5.5 SetGYRO1DEKFParam”和“6.5.9SetMAG1DEKFParam”。

Style	reserve	reserve	Mag(bit5)	Mag(bit4)	Gyro(bit3)	Gyro(bit2)	Acc(bit1)	Acc(bit0)
NONE			0	0	0	0	0	0
Default(CAL)			0	1	0	1	0	1
CAL			1	0	1	0	1	0
KAL			1	1	1	1	1	1

6.5 算法引擎类

6.5.1 SetACC1DEKFParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x03	12			0x6d	HOST	DEVICE

当收到此消息，Saber 将配置外部卡尔曼滤波器参数以满足不同的应用场景需求。如果用户倾向于实现流畅的数据，建议使用高比例的 R/Q，而如果用户趋向于实现快速响应，则建议使用低比例的 R/Q。默认情况下，Q 值为 0.05，R 值为 5。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.2 ACC1DEKFParamAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x83	12			0x6d	DEVICE	HOST

此消息是对 **SetACC1DEKFParam** 的确认。

有效载荷描述如下：

有效载荷: reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.3 GetACC1DEKFParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x04	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回外部卡尔曼滤波器参数配置。

6.5.4 RespACC1DEKFPParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x84	12			0x6d	DEVICE	HOST

是对 **GetACC1DEKFPParam** 命令的回应。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.5 SetGYRO1DEKFPParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x05	12			0x6d	HOST	DEVICE

当收到此消息，Saber 将配置外部卡尔曼滤波器参数以满足不同的应用场景需求。如果用户倾向于实现流畅的数据，建议使用高比例的 R/Q，而如果用户趋向于快速响应，则建议使用低比例的 R/Q。默认情况下，Q 值为 0.1，R 值为 1。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.6 GYRO1DEKFPParamAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x85	12			0x6d	DEVICE	HOST

此消息是对 **SetGYRO1DEKFPParam** 命令的确认。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.7 GetGYRO1DEKFParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x06	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回外部卡尔曼滤波器的参数配置。

6.5.8 RespGYRO1DEKFParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x86	12			0x6d	DEVICE	HOST

对 **GetGYRO1DEKFParam** 命令的回应。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.9 SetMAG1DEKFParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x07	12			0x6d	HOST	DEVICE

当收到此消息，Saber 将配置外部卡尔曼滤波器参数以满足不同的应用场景需求。如果用户倾向于实现流畅的数据，建议使用高比例的 R/Q，而如果用户趋向于实现快速响应，则建议使用低比例的 R/Q。默认情况下，Q 值为 0.05，R 值为 50。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.10 MAG1DEKFPParamAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x87	12			0x6d	DEVICE	HOST

此消息是对 **SetMAG1DEKFPParam** 的确认。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.11 GetMAG1DEKFPParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x08	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回外部卡尔曼滤波器的参数配置。

6.5.12 RespMAG1DEKFPParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x88	12			0x6d	DEVICE	HOST

对 **GetMAG1DEKFPParam** 命令的回应。

有效载荷描述如下：

有效载荷: u8 reserve[4],float Kalman_Q,float Kalman_R;

Kalman_Q (系统噪声协方差)

Kalman_R (观测噪声的协方差)

6.5.13 SetEngineFusionMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x09	1			0x6d	HOST	DEVICE

当收到此消息，Saber 将决定在引擎中选择哪种数据融合模式。

有效载荷描述如下：

```
enum
{
    ATOM_EKF_SOPHISTICATE = 0,
    ATOM_EKF_SOPHISTICATE_MAG,
    ATOM_CF,
    ATOM_CF_MAG
};
```

6.5.14 EngineFusionModeAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x89	1			0x6d	DEVICE	HOST

此消息是对 **SetEngineFusionMode** 的确认。

有效载荷与请求消息相同。

6.5.15 GetEngineFusionMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x0A	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将回复引擎模式的配置。

6.5.16 RespEngineFusionMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x8A	0			0x6d	DEVICE	HOST

对 **GetEngineFusionMode** 命令的回应。

6.5.17 SetPacketUpdateRate

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x10	2			0x6d	HOST	DEVICE

当收到此消息，Saber 将配置输出数据速率(ODR)，如果引擎模式设置为复杂，则最大 ODR 为 400Hz，如果引擎模式设置为快速，则最大 ODR 为 1000Hz。如果用户想实现最低功耗，则可以使用 0Hz 来关闭输出数据。

有效载荷是请求得到的更新速率，该值可以是下面的值之一：

0, 50, 100, 200, 400, 500, 800, 1000 Hz.

例如：

unsigned short payload = 500;

6.5.18 PacketUpdateRateAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x90	2			0x6d	DEVICE	HOST

此消息是对 **SetPacketUpdateRate** 的确认。

有效载荷是请求得到的更新速率，该值可以是下面的值之一：

0, 50, 100, 200, 400, 500, 800, 1000 Hz.

例如：

unsigned short payload = 500;

6.5.19 GetPacketUpdateRate

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x11	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回 ODR 的配置。

6.5.20 RespPacketUpdateRate

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x91	2			0x6d	DEVICE	HOST

对 **GetPacketUpdateRate** 命令的回应。

有效载荷是请求得到的更新速率，该值可以是下面的值之一：

0, 50, 100, 200, 400, 500, 800, 1000 Hz.

例如:

unsigned short payload = 500;

6.5.21 SetGyroThreshold

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x22	16			0x6d	HOST	DEVICE

当收到此消息，当陀螺仪进入引擎进行数据融合时，Saber 将为陀螺仪设置一个特殊的阈值。如果陀螺仪数据(无论是 CAL 数据还是 KAL 数据)超过阈值或小于负阈值，那么数据会被传输到引擎，否则将 0 传输到引擎。在 Saber 稳定的情况下，一旦用户启用此功能，用户将看到偏航角度保持冻结。这在一些应用场景中是十分有用的。

有效载荷描述如下:

有效载荷: u8 enable u8 reserve u8 reserve u8 reserve float x float y float z.

6.5.22 GyroThresholdACK

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xA2	16			0x6d	DEVICE	HOST

此消息是对 **SetGyroThreshold** 的确认。

有效载荷描述如下:

x 轴数据占用 4 字节, y 轴数据占用 4 字节, z 轴数据占用 4 字节,使能占用 1 字节,3 字节保留。

6.5.23 GetGyroThreshold

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x23	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回陀螺仪的阈值。

6.5.24 RespGyroThreshold

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xA3	16			0x6d	DEVICE	HOST

对 **GetGyroThreshold** 命令的回应。

有效载荷描述如下：

x 轴数据占用 4 字节，y 轴数据占用 4 字节，z 轴数据占用 4 字节,使能占用 1 字节,3 字节保留。

6.5.25 SetYawOffset

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x30	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将偏航角度重置为零。要恢复偏航角为初始上电方向为零的状态，请发送消息“6.5.37ClearOffset”。

6.5.26 SetYawOffsetAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xB0	0			0x6d	DEVICE	HOST

此消息是对 **SetYawOffset** 的确认。

6.5.27 SetAttitudeOffset

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x31	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将重置翻滚角，俯仰角和偏航角为零。要恢复欧拉角为初始上电方向为零的状态，请发送消息“6.5.37 ClearOffset”。

6.5.28 SetAttitudeOffsetAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xB1	0			0x6d	DEVICE	HOST

此消息是对 **SetAttitudeOffset** 的回应。

6.5.29 SetReferenceOffset

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x32	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将重置翻滚角和俯仰角为零，要恢复欧拉角为初始上电方向为零的状态，请发送消息“6.5.37 ClearOffset”。

6.5.30 SetReferenceOffsetAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xB2	0			0x6d	DEVICE	HOST

此消息是对 **SetReferenceOffset** 的确认。

6.5.31 SetDistanceOffset

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x33	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将重置位移信息。

6.5.32 SetDistanceOffsetAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xB3	0			0x6d	DEVICE	HOST

此消息是对 **SetDistanceOffset** 的确认。

6.5.33 SetVelocityOffset

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x34	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将重置速度信息。

6.5.34 SetVelocityOffsetAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xB4	0			0x6d	DEVICE	HOST

此消息是对 **SetVelocityOffset** 的确认。

6.5.35 SetStartYaw

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x35	4			0x6d	HOST	DEVICE

当收到此消息，Saber 会将初始偏航角设置为 Payload 所表示的值。

6.5.36 SetStartYawAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xB5	1			0x6d	DEVICE	HOST

此消息是对 **SetStartYaw** 的确认，Payload 代表该命令的执行结果，1 代表失败，0 代表成功。

6.5.37 ClearOffset

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x40	1			0x6d	HOST	DEVICE

当收到此消息，Saber 将清空所有消息“6.5.25 SetYawOffSet”、“6.5.27SetAttitudeOffSet”和“6.5.29 SetReferenceOffSet”。

有效载荷描述如下：

1:clear

6.5.38 ClearOffsetAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xC0	1			0x6d	DEVICE	HOST

此消息是对 **ClearOffset** 的确认。

有效载荷描述如下：

1:clear

6.5.39 SetOdometerPara

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
---------------	-------	----------	------------	----------------	---------	------------------	------------------	--------	-------------

0x41 0x78	0xFF or ID	0x04	0x41	20			0x6d	HOST	DEVICE
-----------	------------	------	------	----	--	--	------	------	--------

当收到此消息，Saber 将 Odometer 的配置参数写入内存，如果用户使用 **ConfigurationWriteToFlash** 命令，配置参数将会写入 flash，掉电不丢失，有效载荷描述如下：

In LSB format.

Byte0-3: 后轮直径 D，float 格式，单位 m。

Byte4-7: 后轮距 L，两个后轮之间的距离，float 格式，单位 m。

Byte8-11: 编码器分辨率，也就是后轮转一圈输出多少个方波，unsigned int 格式。

Byte12-15: 以两个后轮中心连线为 X 轴，选两个后轮连线的中心点为原点，连接线的垂线为 Y 轴建立坐标（参考下图）。Saber 模组中心距离原点的位置在 X 轴的坐标，float 格式，单位 m。

Byte16-19: 以两个后轮中心连线为 X 轴，选两个后轮连线的中心点为原点，连接线的垂线为 Y 轴建立坐标。Saber 模组中心距离原点的位置在 Y 轴的坐标，float 格式，单位 m。

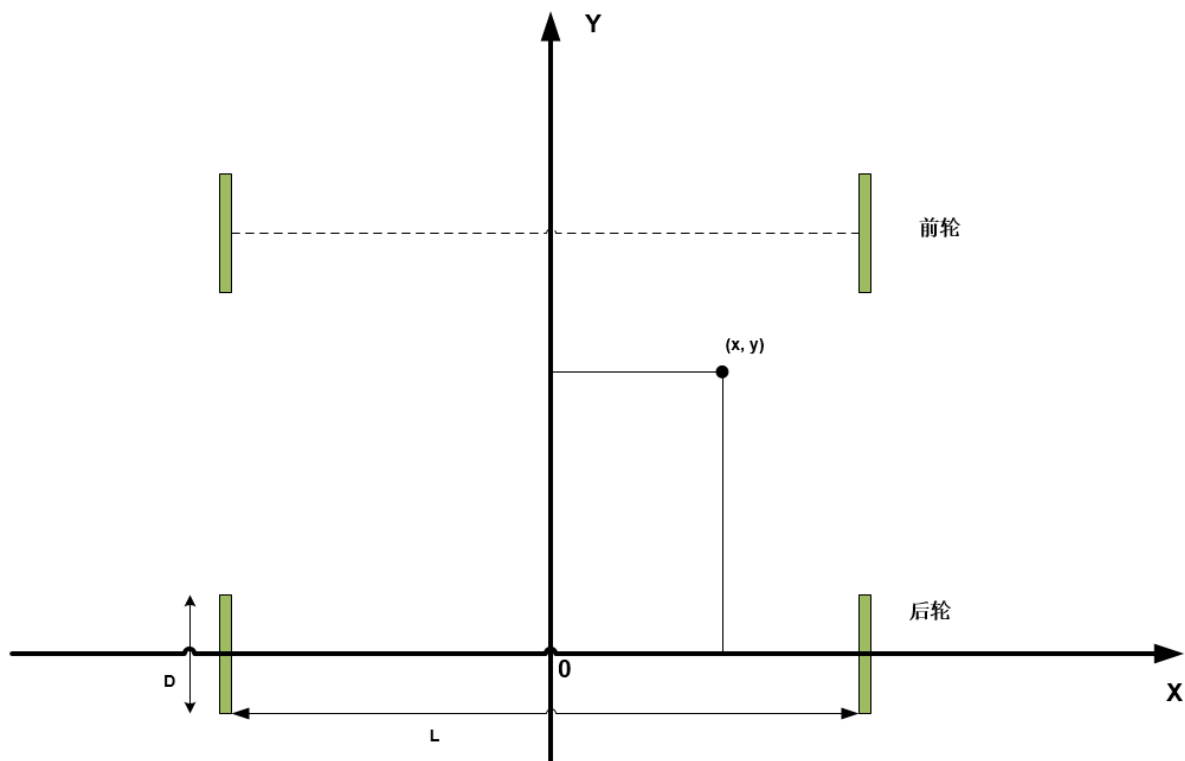


Figure 13 - 坐标定义

6.5.40 SetOdometerParaAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xC1	0			0x6d	DEVICE	HOST

此消息是对 **SetOdometerPara** 的确认。

6.5.41 GetOdometerPara

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x42	0			0x6d	HOST	DEVICE

当收到此消息收到此消息，Saber 将返回 Odometer 的配置参数。

6.5.42 RespOdometerPara

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xC2	20			0x6d	DEVICE	HOST

消息是对 **GetOdometerPara** 的确认。

有效载荷描述如下：

In LSB format.

Byte0-3: 轮子直径, in float, 单位 m。

Byte4-7: 轮距, 两个轮子之间的距离, in float, 单位 m。

Byte8-11: 编码器分辨率, 也就是轮子转一圈输出多少个方波。Unsigned int。

Byte12-15: 以两个轮子中心连线为 X 轴, 选两个轮子连线的中心点为原点, 连接线的垂线为 Y 轴建立坐标。Saber 模组中心距离原点的位置在 X 轴的坐标, float 格式, 单位 m。

Byte16-19: 以两个轮子中心连线为 X 轴, 选两个轮子连线的中心点为原点, 连接线的垂线为 Y 轴建立坐标。Saber 模组中心距离原点的位置在 Y 轴的坐标, float 格式, 单位 m。

6.5.43 OdometerPacket

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x43	16			0x6d	HOST	DEVICE

当收到此消息, Saber 将返回 Odometer 的数据。

有效载荷描述如下：

In LSB format

Byte0-3: Odometer 给出的左轮速度, float 格式, 单位 m/s

Byte4-7: Odometer 给出的右轮速度, float 格式, 单位 m/s

Byte8-15: Reserve

6.5.44 SetScenario

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x50	1			0x6d	HOST	DEVICE

当收到此命令时，Saber 将 Engine 根据工作场景配置的不同将引擎配置为不同的数据融合模式。

有效的载荷如下：

0: SCENARIO_MODE_3D，这是空间 3 维的运动场景(X/Y/Z)，引擎数据融合算法会针对 3D 运动进行优化。

1: SCENARIO_MODE_2D，这是平面 2 维的运动场景(X/Y)，数据融合引擎会根据 2D 运动进行优化。请注意，3D 场景在当 Saber 配置为 2D 场景时候也可以被支持，只是精度会差于将 Saber 配置为 3D 场景。

6.5.45 SetScenarioAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xD0	1			0x6d	DEVICE	HOST

此消息是对 **SetScenario** 的确认。

有效的载荷如下：

0: SCENARIO_MODE_3D，这是空间 3 维的运动场景。

1: SCENARIO_MODE_2D，这是平面 2 维的运动场景。

6.5.46 GetScenario

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x51	0			0x6d	HOST	DEVICE

收到此消息，Saber 将返回 Engine 的场景配置。

6.5.47 RespGetScenario

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xD1	1			0x6d	DEVICE	HOST

消息是对 **GetScenario** 的确认。

有效的载荷如下：

0: SCENARIO_MODE_3D，这是空间 3 维的运动场景。

1: SCENARIO_MODE_2D，这是平面 2 维的运动场景。

6.5.48 SetCoordinatesMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x52	9			0x6d	HOST	DEVICE

当收到此命令时，Saber 将设置坐标系转换矩阵 P。Saber 固有坐标系通过这个矩阵 P 的旋转，产生用户坐标系，Saber 输出的传感器数据将以用户坐标系为参考输出。

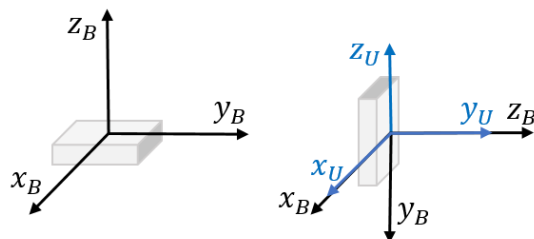
有效的载荷如下：

Payload 字节偏移	数据格式	名称	可用取值
0	char	p_{00}	0(0x00), -1(0xFF), 1(0x01)
1	char	p_{01}	0(0x00), -1(0xFF), 1(0x01)
2	char	p_{02}	0(0x00), -1(0xFF), 1(0x01)
3	char	p_{10}	0(0x00), -1(0xFF), 1(0x01)
4	char	p_{11}	0(0x00), -1(0xFF), 1(0x01)
5	char	p_{12}	0(0x00), -1(0xFF), 1(0x01)
6	char	p_{20}	0(0x00), -1(0xFF), 1(0x01)
7	char	p_{21}	0(0x00), -1(0xFF), 1(0x01)
8	char	p_{22}	0(0x00), -1(0xFF), 1(0x01)

坐标系旋转公式为：

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_U = P \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix}_B = \begin{bmatrix} p_{00} & p_{01} & p_{02} \\ p_{10} & p_{11} & p_{12} \\ p_{20} & p_{21} & p_{22} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix}_B$$

其中， $[x \ y \ z]_B$ 为 Saber 固有坐标系下的任意一组数据（例如加速度计），为了得到用户坐标系下的对应数据 $[x \ y \ z]_U$ （例如加速度计），需要设置坐标系转换矩阵 P。



例如，在上图中，Saber 固有坐标系和用户坐标系对应的矩阵为：

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix}$$

则对应的有效载荷为 “0x01 0x00 0x00 0x00 0x00 0x01 0x00 0xFF 0x00”。

值得注意的是，转换矩阵 P 应该为正交矩阵。目前，Saber 坐标系变换中，坐标系绕每个轴旋转的角度仅支持 0，90，180 和 270 度。因此，矩阵 P 中的每个元素的有效取值，只能为 0，1 或-1。矩阵 P 默认为单位矩阵，即 Saber 坐标系与用户坐标系默认重合。

6.5.49 SetCoordinatesModeAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xD2	9			0x6d	DEVICE	HOST

此消息是对 **SetCoordinatesMode** 的确认。

有效的载荷如下：

Payload 字节偏移	数据格式	名称	可用取值
0	char	p_{00}	0(0x00), -1(0xFF), 1(0x01)
1	char	p_{01}	0(0x00), -1(0xFF), 1(0x01)
2	char	p_{02}	0(0x00), -1(0xFF), 1(0x01)
3	char	p_{10}	0(0x00), -1(0xFF), 1(0x01)
4	char	p_{11}	0(0x00), -1(0xFF), 1(0x01)
5	char	p_{12}	0(0x00), -1(0xFF), 1(0x01)
6	char	p_{20}	0(0x00), -1(0xFF), 1(0x01)
7	char	p_{21}	0(0x00), -1(0xFF), 1(0x01)
8	char	p_{22}	0(0x00), -1(0xFF), 1(0x01)

6.5.50 GetCoordinatesMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x53	0			0x6d	HOST	DEVICE

收到此消息，Saber 将返回坐标系的配置。

6.5.51 RespGetCoordinatesMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xD3	9			0x6d	DEVICE	HOST

消息是对 **GetCoordinatesMode** 的确认。

有效的载荷如下：

Payload 字节偏移	数据格式	名称	可用取值
0	char	p_{00}	0(0x00), -1(0xFF), 1(0x01)
1	char	p_{01}	0(0x00), -1(0xFF), 1(0x01)
2	char	p_{02}	0(0x00), -1(0xFF), 1(0x01)
3	char	p_{10}	0(0x00), -1(0xFF), 1(0x01)
4	char	p_{11}	0(0x00), -1(0xFF), 1(0x01)
5	char	p_{12}	0(0x00), -1(0xFF), 1(0x01)
6	char	p_{20}	0(0x00), -1(0xFF), 1(0x01)
7	char	p_{21}	0(0x00), -1(0xFF), 1(0x01)
8	char	p_{22}	0(0x00), -1(0xFF), 1(0x01)

6.5.52 SetSelfTestMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x54	1			0x6d	HOST	DEVICE

当收到此命令时，Saber 将配置自检工作模式。

有效的载荷如下：

Bit [0]: ST_Enable_Bit

1: 开启自检。0: 关闭自检。

Bit [4]: ST_Output_Bit

1: 自检失败时继续输出。

0: 自检失败时，数据不输出。

其他 bit，保留位。

默认值为：0x01

	ST_Enable_Bit 0: 不自检 1: 自检	ST_Output_Bit 1: 输出 0: 不输出	自检结果	描述	响应命令支持
不自检	0	0	x	不自检 不输出 无效状态	
	0	1	x	不自检 输出 低精度数据	全部 MEASURE mode 可响应的命令
自检	1	0	自检通过	自检通过 不输出 无效状态	
	1	0	自检失败	自检不过 不输出	只支持以下命令： SystemReset GetAllStatus SetSelfTestMode GetSelfTestMode ConfigurationWriteToFlash
	1	1	自检通过	自检通过 输出 高精度数据	全部 MEASURE mode 可响应的命令
	1	1	自检失败	自检不过 输出 低精度数据	全部 MEASURE mode 可响应的命令

Table 12 – 自检配置与状态表

关于自检结果信息参考“6.3.7 GetAllStatus”。

6.5.53 SetSelfTestModeAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xD4	1			0x6d	DEVICE	HOST

此消息是对 **SetSelfTestMode** 的确认。

有效的载荷如下：

Bit [0]:

1: 开启自检。0: 关闭自检。

Bit [4]:

1: 自检失败时继续输出。

0: 自检失败时，数据不输出。

其他 bit，保留位。

默认值为：0x01

6.5.54 GetSelfTestMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0x55	0			0x6d	HOST	DEVICE

收到此消息，Saber 将返回自检的配置。

6.5.55 RespGetSelfTestMode

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x04	0xD5	1			0x6d	DEVICE	HOST

消息是对 **GetSelfTestMode** 的确认。

有效的载荷如下：

Bit [0]:

1: 开启自检。0: 关闭自检。

Bit [4]:

1: 自检失败时继续输出。

0: 自检失败时，数据不输出。

其他 bit，保留位。

默认值为：0x01

6.6 通讯配置类

6.6.1 SetComPortParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x05	0x01	5			0x6d	HOST	DEVICE

当收到此消息，Saber 将根据有效载荷配置串口，需要在发送该信息之后的 3s 内发送 **SetComPortParamHandshake** 来进行握手，确认波特率改变成功，否则设备将会恢复为原波特率。。

有效载荷描述如下：

波特率(U8)+字长(U8)+停止位(U8)+奇偶校验(U8)+流量控制(U8)

波特率数值可以是以下值：

BAUDRATE_9600	0x01
BAUDRATE_38400	0x02
BAUDRATE_57600	0x03
BAUDRATE_115200	0x04
BAUDRATE_230400	0x05
BAUDRATE_460800	0x06
BAUDRATE_921600	0x07
BAUDRATE_1500000	0x08

字长的描述如下：

它是强制性的 8 位，代表 8 位数据。

停止位数值可以是以下值：

STOP_BIT_1	0x01
STOP_BIT_2	0x02

奇偶校验位描述如下：

PARITY_无 0x01

PARITY_EVEN 0x02

PARITY_ODD 0x03

流控数值可以是如下数值：

FLOWCONTROL_无 0x01

FLOWCONTROL_RTS_CTS 0x02

6.6.2 ComPortParamAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x05	0x81	5			0x6d	DEVICE	HOST

此消息是对 **SETComPortParam** 的确认。

有效载荷描述如下：

波特率(U8)+字长(U8)+停止位(U8)+奇偶校验(U8)+流量控制(U8)

波特率数值可以是以下值：

BAUDRATE_9600 0x01

BAUDRATE_38400 0x02

BAUDRATE_57600 0x03

BAUDRATE_115200 0x04

BAUDRATE_230400 0x05

BAUDRATE_460800 0x06

BAUDRATE_921600 0x07

BAUDRATE_1500000 0x08

字长的描述如下：

它是强制性的 8 位，代表 8 位数据。

停止位数值可以是以下值：

STOP_BIT_1 0x01

STOP_BIT_2 0x02

奇偶校验位描述如下：

PARITY_无 0x01

PARITY_EVEN 0x02

PARITY_ODD 0x03

流控数值可以是如下数值:

FLOWCONTROL_无 0x01

FLOWCONTROL_RTS_CTS 0x02

6.6.3 GetComPortParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x05	0x02	0			0x6d	HOST	DEVICE

当收到此消息, Saber 将返回串口配置。

6.6.4 RespComPortParam

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x05	0x82	5			0x6d	DEVICE	HOST

对 **GetComPortParam** 命令的回应。

有效载荷描述如下:

波特率(U8)+字长(U8)+停止位(U8)+奇偶校验(U8)+流量控制(U8)

波特率数值可以是以下值:

BAUDRATE_9600 0x01

BAUDRATE_38400 0x02

BAUDRATE_57600 0x03

BAUDRATE_115200 0x04

BAUDRATE_230400 0x05

BAUDRATE_460800 0x06

BAUDRATE_921600 0x07

BAUDRATE_1500000 0x08

字长的描述如下:

它是强制性的 8 位, 代表 8 位数据。

停止位数值可以是以下值：

STOP_BIT_1 0x01

STOP_BIT_2 0x02

奇偶校验位描述如下：

PARITY_无 0x01

PARITY_EVEN 0x02

PARITY_ODD 0x03

流控数值可以是如下数值：

FLOWCONTROL_无 0x01

FLOWCONTROL_RTS_CTS 0x02

6.6.5 SetComPortParamHandshake

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x05	0x10	0			0x6d	HOST	DEVICE

该消息应用于更改波特率情况下与主机通信的实际场景中。用户想更改波特率时，需要从 Host 发送带有先前波特率的第一个命令 **SetComPortParam**，然后使用带有新波特率的第二个命令 **SetComPortParamHandshake** 作为与 Host 的通信握手。一旦主机和 Saber 成功握手，就意味着波特率已成功改变。更多细节将在单独的程序备注中给出。

6.6.6 ComPortParamHandshakeAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x05	0x90	0			0x6d	DEVICE	HOST

此消息是对 **SetComPortParamHandshake** 的确认。

6.7 主机配置类

6.7.1 SaberDatapacket

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x81	Size of one packet			0x6d	HOST	DEVICE

当 Saber 处于 **MEASURE** 模式下时，它会自动发送数据包，有效载荷长度是根据用户配置的数据包类型而定。要了解有效载荷的细节，请参照“4.7Saber 数据包结构”。

6.7.2 SetDrdyConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x06	2			0x6d	HOST	DEVICE

当收到此消息，Saber 将打开或关闭数据准备(DRDY)功能。在 IIC 或 SPI 通信期间 DRDY 非常有用，它可以在 Saber 缓冲器准备好数据时通知主机。

有效载荷描述如下：

Bit 编号	规格说明
Bit 15	0:DRDY disable 1:DRDY enable
Bit 14:2	Reserve
Bit 1	OTYPE:Output type of DRDY pin 0:Push/pull 1:Open drain
Bit 0	POL:Polarity of DRDY signal 0:Active high 1:Active low

6.7.3 DrdyConfigAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x86	2			0x6d	DEVICE	HOST

此消息是对 **SetDrdyConfig** 的确认。

有效载荷描述如下：

Bit 编号	规格说明
Bit 15	0:DRDY disable 1:DRDY enable
Bit 14:2	Reserve
Bit 1	OTYPE:Output type of DRDY pin 0:Push/pull 1:Open drain

Bit 0	POL:Polarity of DRDY signal 0: Active high 1: Active low
-------	--

6.7.4 GetDrdyConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x07	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回 DRDY 配置。

6.7.5 RespDrdyConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x87	2			0x6d	DEVICE	HOST

对 **GetDrdyConfig** 命令的回应。

有效载荷描述如下：

Bit 编号	规格说明
Bit 15	0:DRDY disable 1:DRDY enable
Bit 14:2	Reserve
Bit 1	OTYPE:Output type of DRDY pin 0:Push/pull 1:Open drain
Bit 0	POL:Polarity of DRDY signal 0:Active high 1:Active low

6.7.6 SetGpioConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x08	2			0x6d	HOST	DEVICE

当收到此消息，Saber 将配置 GPIO 功能。Bit 0 用于选择 GPIO 模式。

如果选择 LED 模式，则 Bit 1 或 Bit 2 将决定相应的 LED 闪光灯功能。一旦呼吸灯启动，LED 将以每秒一次的模式 1 波形闪烁。在 LED 模式下，GPIO 被配置为输出 IO 口。

有效载荷描述如下：

Bit 编号	规格说明	模式
Bits 15:3	Reserved for future use	NA
Bit2	data transmission	Real time toggle
Bit1	breath shining	模式 1
Bit0	0: LED Mode	NA
	1: NA	NA



6.7.7 GpioConfigAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x88	2			0x6d	DEVICE	HOST

此消息是对 **SetGpioConfig** 的确认。

有效载荷描述如下：

Bit 编号	规格说明	模式
Bits 15:3	Reserved for future use	NA
Bit2	data transmission	Real time toggle
Bit1	breath shining	模式 1
Bit0	0: LED Mode	NA
	1: NA	NA



6.7.8 GetGpioConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x09	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将返回 GPIO 配置。

6.7.9 RespGpioConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x89	2			0x6d	DEVICE	HOST

对 **GetGpioConfig** 命令的回应。

有效载荷描述如下：

Bit Num	规格说明	模式
Bits 15:3	Reserved for future use	NA
Bit2	data transmission	Real time toggle
Bit1	breath shining	模式 1
Bit0	0: LED Mode	NA
	1:NA	NA



6.7.10 SetSyncOutConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x0C	8			0x6d	HOST	DEVICE

此消息用于配置同步输出功能。

有效载荷描述如下：

Offset(bytes)	Setting	模式
0	Reserved	bit0: Enable(1) or Disable(0) syncout function bit1-7:reserved
1	Polarity	The polarity of the sync pulse。 One of:Rising Edge(0x01),Falling Edge(0x02);Positive Pulse (0x03), Negative Pulse (0x04)
2-3	Offset	Offset from event to pulse generation (100μs units, range [-30000..+30000]).

4-5	Skip Factor	The number of events to skip after taking the action before taking action again.
6-7	PulseWidth	The width of the generated pulse in 100μs units.

6.7.11 SetSyncOutConfigAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x8C	8			0x6d	DEVICE	HOST

此消息是对 **SetSyncOutConfig** 的确认

关于有效载荷信息参考“6.7.10 SetSyncOutConfig”。

6.7.12 GetSyncOutConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x0D	8			0x6d	HOST	DEVICE

此消息是对返回同步输出配置

6.7.13 GetSyncOutConfigAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x8D	8			0x6d	DEVICE	HOST

此消息是对 **GetSyncOutConfig** 命令的回复

有效载荷信息为

Offset(bytes)	Setting	模式
0	Reserved	bit0: Enable(1) or Disable(0) syncout function bit1-7:reserved
1	Polarity	The polarity of the sync pulse。 One of:Rising Edge(0x01),Falling Edge(0x02);Positive Pulse (0x03), Negative Pulse (0x04)
2-3	Offset	Offset from event to pulse generation (100μs units, range [-30000..+30000]).
4-5	Skip Factor	The number of events to skip after taking the action before taking action again.
6-7	PulseWidth	The width of the generated pulse in 100μs

units.

6.7.14 SetDataPacketConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x0A	4*N			0x6d	HOST	DEVICE

有关有效载荷的详细信息，请参见“4.6 Saber 数据配置结构”一节。

6.7.15 DataPacketConfigAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x8A	4*N			0x6d	DEVICE	HOST

此消息是对 **SetDataPacketConfig** 的确认。

有关有效载荷的详细信息，请参见“4.6 Saber 数据配置结构”一节。

6.7.16 GetDataPacketConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x0B	0			0x6d	HOST	DEVICE

当收到此消息，Saber 将会返回数据包配置。

6.7.17 RespDataPacketConfig

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x06	0x8B	4*N			0x6d	DEVICE	HOST

对 **GetDataPacketConfig** 命令的回应。

有关有效载荷的详细信息，请参见“4.6 Saber 数据配置结构”一节。

6.8 固件更新类

6.8.1 FirmwareUpdateReq

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
------------------	-------	-------------	---------------	-------------------	---------	---------------------	---------------------	--------	-------------

0x41 0x78	0xFF or ID	0x0A	0x01	1			0x6d	HOST	DEVICE
-----------	------------	------	------	---	--	--	------	------	--------

当收到此消息，Saber 将开始或停止固件下载。

有效载荷描述如下：

1: Start to download firmware.

0: Stop to download firmware.

6.8.2 FirmwareUpdateReqAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x0A	0x81	1			0x6d	DEVICE	HOST

此消息是对 **FirmwareUpdateReq** 的确认。

有效载荷与请求消息相同。

6.8.3 FirmwareUpdatePacket

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x0A	0x02	64+2			0x6d	HOST	DEVICE

当收到此消息，Saber 将处理这个数据包。

Bin 文件是用于更新 Saber 固件的文件，并且始终为 64 个字节的整数倍。用户需要为每个要更新的数据包发送长度为 66 字节的 **FirmwareUpdatePacket**。其中的前 2 个字节是数据包编号，而 64 个字节是来自 bin 文件的数据。数据包从 0 开始计数。

有效载荷描述如下：

前两个字节是数据包编号，后面的 64 字节是固件的 bin 文件。

例如：“7.6.3FirmwareUpdatePacket”

6.8.4 FirmwareUpdatePacketAck

Preamble 'Ax'	MADDR	Class ID	Message ID	Payload Length	Payload	Message CheckSum	Footer (Sync)'m'	Source	Destination
0x41 0x78	0xFF or ID	0x0A	0x82	2			0x6d	DEVICE	HOST

此消息是对 **FirmwareUpdatePacket** 的确认。

有效载荷与 ***FirmwareUpdatePacket*** 的数据包编号相同。

7 示例代码

7.1 概述

在示例目录中可以找到如何使用Saber的示例。这些简单的程序告诉了用户如何轻松设置或者从Saber中读取数据。我们所支持的测试用例如下表所示：

SDK 例程名/ 环境	Windows		Linux		
	X86	X64	ARM	X86	X64
MeasureData Demo	√	√	NA	NA	NA
DownloadFirmware	√	√	√	√	√
AutoProtocol Demo	√	√	√	√	√

Table 13 – SDK 例程矩阵

Low_level 例程 名/环境	Windows		Linux	STM32- UART	STM32-IIC	STM32-SPI
	X86	X64				
MeasureData Demo	√	√	√	√	√	√
Low-Level DownLoad	√	√	√	√	NA	NA
AutoProtocol Demo	NA	NA	NA	NA	NA	NA

Table 14 – Low Level 例程矩阵

7.2 SDK AutoProtocolDemo

本节内容主要介绍的是主机如何调用 SDK 对 Saber 进行自动化协议测试。

Windows 例程基于 Visual Studio 2013，代码已被验证可运行。

Linux 例程执行其目录下 Makefile 生成可执行文件，代码已被验证可运行。

由于Windows例程、Linux例程的协议处理逻辑和数据包解析逻辑一致，故该说明将会对它们进行统一描述。

请在执行示例工程之前进行以下操作步骤：

- 查看设备COM口编号与波特率。
- 在对应示例工程目录下编辑Com.ini文件，填写COM口编号与波特率。
- 将对应运行环境下的库文件与头文件放置于示例工程目录下。

下面是代码结构：

main()函数：

1. SDK初始化，API:ATR_Init();
2. 打开串口，API: ATR_OpenCom ();
3. 注册回调函数，API: ATR_RegisterCallback ();
4. 设置模组为**CONFIG**模式，API: ATR_SwitchModeReq ();
5. 在上一条测试协议的回调函数中发送下一条协议测试；
6. 等待程序与模组所有测试测试完毕，程序退出；

程序退出后，请阅读生成的协议验证测试报告。

```

E:\Atom\Programs\Saber\RnD\Host\03_SDK\Common\SampleCodes\SDK\Windows_SDK_Demo\AutoProtocolDemo\Debug\AutoProtocolValidati...
Received GetPacketUpdateRateAck
Sending packet SetPacketUpdateRate...
Received SetPacketUpdateRateAck
Sending packet GetGyroscopeAutoCalibration...
Received GetGyroscopeAutoCalibrationAck
Sending packet SetGyroscopeAutoCalibration...
Received SetGyroscopeAutoCalibrationAck
Sending packet GetComPortParam...
Received GetComPortParamAck
Sending packet SetComPortParam...
Received SetComPortParamAck
Sending packet SetComPortParamHandshake...
Received SetComPortParamHandshakeAck
Sending packet GetAccelerometerScale...
Received GetAccelerometerScaleAck
Sending packet SetAccelerometerScale...
Received SetAccelerometerScaleAck
Sending packet GetGyroscopeScale...
Received GetGyroscopeScaleAck
Sending packet SetGyroscopeScale...
Received SetGyroscopeScaleAck
Sending packet GetMagnetometerScale...
Received GetGpioConfigAck
Sending packet SetGpioConfig...
Received SetGpioConfigAck
Sending packet GetGyroThreshold...
Received GetGyroThresholdAck
Sending packet SetGyroThreshold...
Received SetGyroThresholdAck
Sending packet SetDataPacketConfig...

```

Figure 14 – SDK AutoProtocolDemo 运行示例

7.3 SDK MeasureDataDemo

本节内容主要介绍的是主机如何调用 SDK 获取解析 Saber 测量数据通信，主要对模组的波特率进行了更改，配置了原始数据的包的加速度计和陀螺仪，解析数据并打印。

Windows 例程基于 Visual Studio 2013，代码已被验证可运行。

Linux 例程执行其目录下 Makefile 生成可执行文件，代码已被验证可运行。

由于Windows例程、Linux例程的协议处理逻辑和数据包解析逻辑一致，故该说明将会对它们进行统一描述。

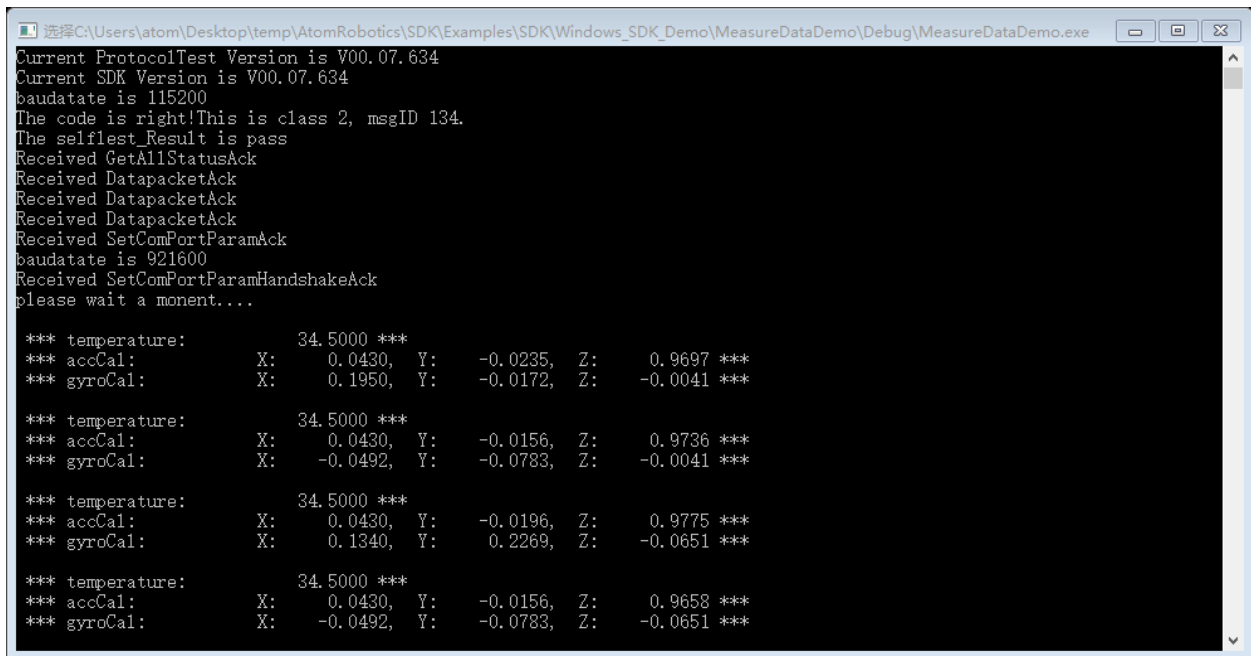
请在执行示例工程之前进行以下操作步骤：

- 查看设备COM口编号与波特率。
- 在对应示例工程目录下编辑Com.ini文件，填写COM口编号与波特率。
- 将对应运行环境下的库文件与头文件放置于示例工程目录下。

下面是代码结构：

main()函数:

1. SDK初始化, API: ATR_Init ();
2. 打开串口, API: ATR_OpenCom ();
3. 注册回调函数, API: ATR_RegisterCallback ();
4. 设置模组为**CONFIG**模式, API: ATR_SwitchModeReq ();
5. 发送加速度计与陀螺仪的配置包, API: ATR_SetDataPacketConfigReq ();
6. 发送更改波特率命令, 并确认握手, API: ATR_SetComPortParamReq ()、
ATR_SetComPortParamHandshakeReq();
7. 进入**MEASURE**模式采集数据, API: ATR_SwitchModeReq ();
8. 回调函数解析采集数据并打印在终端;



```

选择C:\Users\atom\Desktop\temp\AtomRobotics\SDK\Examples\SDK\Windows_SDK_Demo\MeasureDataDemo\Debug\MeasureDataDemo.exe
Current ProtocolTest Version is V00.07.634
Current SDK Version is V00.07.634
baudrate is 115200
The code is right!This is class 2, msgID 134.
The selftest_Result is pass
Received GetAllStatusAck
Received DatapacketAck
Received DatapacketAck
Received DatapacketAck
Received SetComPortParamAck
baudrate is 921600
Received SetComPortParamHandshakeAck
please wait a moment....

*** temperature:      34.5000 ***
*** accCal:           X:   0.0430, Y:  -0.0235, Z:   0.9697 ***
*** gyroCal:          X:   0.1950, Y:  -0.0172, Z:  -0.0041 ***

*** temperature:      34.5000 ***
*** accCal:           X:   0.0430, Y:  -0.0156, Z:   0.9736 ***
*** gyroCal:          X:  -0.0492, Y:  -0.0783, Z:  -0.0041 ***

*** temperature:      34.5000 ***
*** accCal:           X:   0.0430, Y:  -0.0196, Z:   0.9775 ***
*** gyroCal:          X:   0.1340, Y:   0.2269, Z:  -0.0651 ***

*** temperature:      34.5000 ***
*** accCal:           X:   0.0430, Y:  -0.0156, Z:   0.9658 ***
*** gyroCal:          X:  -0.0492, Y:  -0.0783, Z:  -0.0651 ***
  
```

Figure 15 – SDK MeasureDataDemo 运行示例

7.4 SDK DownloadFirmware

本节内容主要介绍的是主机为 Saber 模组进行 Firmware 的更新。

Windows 例程基于 Visual Studio 2013, 代码已被验证可运行。

Linux 例程执行其目录下 Makefile 生成可执行文件, 代码已被验证可运行。

请在执行示例工程之前进行以下操作步骤：

- 查看设备COM口编号，使用SaberXplore将波特率改为115200。
- 在对应示例工程目录下编辑Com.ini文件，填写COM口编号与波特率。
- 将对应运行环境下的库文件与头文件放置于示例工程目录下。
- 将用于升级的BIN文件放置于示例工程目录下，文件名称与示例源码中一致。

由于Windows例程、Linux例程的协议处理逻辑和数据包解析逻辑一致，故该说明将会对它们进行统一描述。

下面是例程代码结构：

main()函数：

1. SDK初始化，API: ATR_Init ();
2. 打开串口，API: ATR_OpenCom ();
3. 注册回调函数，API: ATR_RegisterCallback ();
4. 设置模组为**CONFIG**模式，API: ATR_SwitchModeReq ();
5. 发送更新Firmware命令，API: ATR_DownloadFirmware_new ();
6. 等待模组更新完成，程序退出；

```

选择C:\Users\atom\Desktop\temp\AtomRobotics\SDK\Examples\SDK\Windows_SDK_Demo\Download_Firmware\Debug\Download_Firmware.exe
Current ProtocolTest Version is V00.07.634
Current SDK Version is V00.07.634
baudrate is 115200
download start.....
The loop packet numbers is 4087
Received resetCallback
Event Signaled.
Received FirmwareUpdateReq
Received FirmwareUpdateReq
loop = 4087, frame= 0
Packet_Send: >> frameNo = 0
Received FirmwareUpdatePacketReq
Packet_Recv: 0
Event Signaled.
loop = 4086, frame= 1
Packet_Send: >> frameNo = 1
Received FirmwareUpdatePacketReq
Packet_Recv: 1
Event Signaled.
loop = 4085, frame= 2
Packet_Send: >> frameNo = 2
Received FirmwareUpdatePacketReq
Packet_Recv: 2
Event Signaled.
loop = 4084, frame= 3
Packet_Send: >> frameNo = 3
Received FirmwareUpdatePacketReq
Packet_Recv: 3
Event Signaled.
loop = 4083, frame= 4

```

Figure 16 – SDK DownloadFirmware 在 Windows 下运行示例

7.5 Low-Level MeasureDataDemo说明

本节内容主要介绍的是主机和 Saber 之间的底层数据通信。

由于Windows例程、Linux例程和STM32_Uart例程的协议处理逻辑和数据包解析逻辑一致，故该说明将会对它们进行统一描述。

7.5.1 Windows/STM32-Uart/Linux

以下示例中的字节均以十六进制表示。请确保应用程序能够以十六进制格式进行通信。

在更改配置之前，请保证设备处于**CONFIG**模式。

例程流程图如下“Figure 17 – UART例程流程图”所示，“Figure 18 – Wait Ack流程图”展示的是其中的WaitACK逻辑：

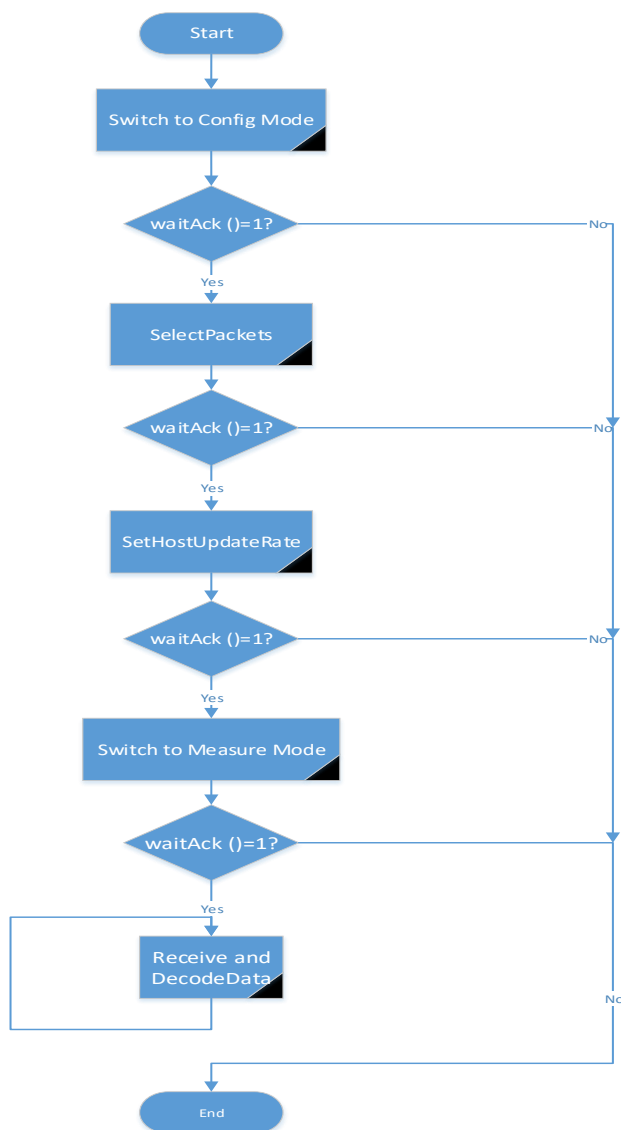


Figure 17 – UART 例程流程图

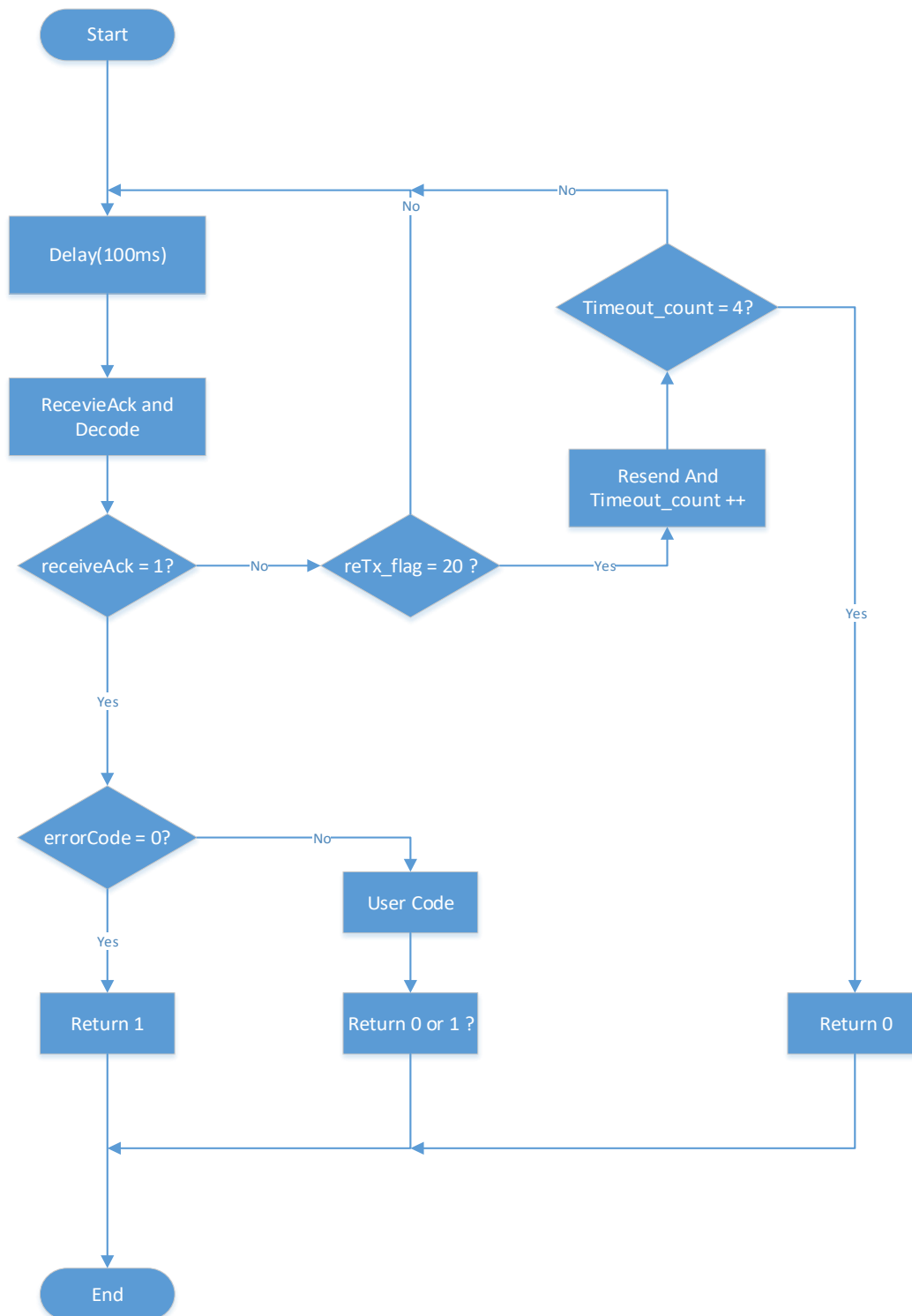


Figure 18 – Wait Ack 流程图

7.5.1.1 SwitchToConfigMode

Send: **SwitchToConfigMode**

TX: 41 78 FF 01 02 00 C5 6D

发送命令之后都会等待 Ack 并对 Ack 进行相应处理

Receive: **SwitchToConfigModeACK:**

RX: 41 78 FF 01 82 00 45 6D

7.5.1.2 SetDataPacketConfig

数据包设置列表如下:

- Enable packet Saber_Cal_Acc (PID = 0x8800)
- Enable packet Saber_Cal_Gyro (PID = 0x8C00)
- Enable packet Saber_Euler (PID = 0xB001)

Send : **SetDataPacketConfig**

TX: 41 78 FF 06 0A 18 FF FF 00 88 FF FF 00 8C FF FF 00 10 FF FF 01 B0 FF FF 00 30 FF FF 00 00 47 6D

发送命令之后都会等待 Ack 并对 Ack 进行相应处理

Receive: **DataPacketConfigACK:**

RX: 41 78 FF 06 8A 18 FF FF 00 88 FF FF 00 8C FF FF 00 10 FF FF 01 B0 FF FF 00 30 FF FF 00 00 C7 6D

7.5.1.3 SetHostUpdateRate

Send : **SetPacketUpdateRate**

TX: 41 78 FF 04 10 02 32 00 E2 6D

发送命令之后都会等待 Ack 并对 Ack 进行相应处理

Receive: **SetPacketUpdateRateACK** command:

RX: 41 78 FF 04 90 02 32 00 62 6D

7.5.1.4 Switch To Measure Mode

Send : **SwitchToMeasureMode**

TX: 41 78 FF 01 03 00 C4 6D

发送命令之后都会等待 Ack 并对 Ack 进行相应处理

Receive: **SwitchToMeasureModeACK**

RX: 41 78 FF 01 83 00 44 6D

7.5.1.5 SaberDataPacket

收到**SaberDataPacket**:

RX: 41 78 FF 06 81 31 00 88 0C 0D DD 6C BD F9 DE 5F 3C 9F 5C 83 3F 00 8C 0C 4A 9B D5 BC 56
5B 41 BE 41 3C BD BC 01 B0 10 FE 0D 50 40 FC 74 50 3F 35 52 4A 3B 00 00 00 00 CB 6D

该消息的构建如下:

41 78 FF 06 81 31	帧头
00 88	PID = 0x8800, 表示Saber_Cal_Acc
0C	该类型数据的有效载荷长度。
0D DD 6C BD F9 DE 5F 3C 9F 5C 83 3F	12个字节的校准ACC数据, 包括X轴, Y轴和Z轴三轴数据。
00 8C	PID = 0x8C00, 表示Saber_Cal_Gyro
0C	该类型数据的有效载荷长度。
4A 9B D5 BC 56 5B 41 BE 41 3C BD BC	12个字节的校准GYRO数据, 包括X轴, Y轴和Z轴三轴数据。

01 B0	PID = 0xB001, 表示 Saber_Euler
10	该类型数据的有效载荷长度。
FE 0D 50 40 FC 74 50 3F 35 52 4A 3B 00 00 00 00	16字节的Saber_Euler数据,12-15字节被保留。
CB 6D	Atom Bcc和尾字节。

若需要解析原始数据，首先需要将十六进制转换为short类型，然后经过计算得到float类型，计算方式如下：

加速度计原始数据解析方法例程，得到的x,y,z是加速度计float类型的Raw数据：

```
//scale 为当前模组的scale值,可通过ATR_GetAccelerometerScaleReq() 获取

typedef struct
{
    short accX;

    short accY;

    short accZ;
}Data_ACC_HandleType;

Data_ACC_HandleType accRawData

const float ACC_GAIN[4] = {0.9765625,1.953125, 3.90625, 7.8125};

float x = (float)accRawData.accX * ACC_GAIN[scale] /1000;

float y = (float)accRawData.accY * ACC_GAIN[scale] /1000;;

float z = (float)accRawData.accZ * ACC_GAIN[scale] /1000;
```

陀螺仪原始数据解析方法例程，得到的x,y,z是陀螺仪float类型的Raw数据：

```
//scale为当前模组的scale值，可通过ATR_GetGyroscopeScaleReq() 获取
```

```
typedef struct

{
    short gyroX;

    short gyroY;

    short gyroZ;

}Data_GYRO_HandleType;

Data_GYRO_HandleType gyroRawData

const float GYRO_GAIN[4] = {0.0076296273689993, 0.0152592547379986, 0.0305185094759972,
0.0610370189519944 };

float x = (float)gyroRawData.gyroX * GYRO_GAIN[scale];

float y = (float)gyroRawData.gyroY * GYRO_GAIN[scale];

float z = (float)gyroRawData.gyroZ * GYRO_GAIN[scale];
```

磁力计原始数据解析方法例程，得到的x,y,z是磁力计float类型的Raw数据：

```
typedef struct

{
    short magX;

    short magY;

    short magZ;

}Data_MAG_HandleType;

Data_MAG_HandleType magRawData;

float x = (float)magRawData.magX * (1.5);

float y = (float)magRawData.magY * (1.5);

float z = (float)magRawData.magZ * (1.5);
```

以下是解析校准ACC数据的例程，解析kalman数据与下面解析calibration数据的方式相同：

一帧数据: 41 78 FF 06 81 31 00 88 0C 0D DD 6C BD F9 DE 5F 3C 9F 5C 83 3F 00
8C 0C 4A 9B D5 BC 56 5B 41 BE 41 3C BD BC 01 B0 10 FE 0D 50 40 FC 74 50 3F
35 52 4A 3B 00 00 00 00 CB 6D

取其中加速度计calibration数据, PID: 0x88 0x00, PL:0x0C, Data: 0D DD 6C BD F9
DE 5F 3C 9F 5C 83 3F

解析加速度计calibration数据代码:

```
typedef union  
{  
    unsigned char u[4];  
  
    float f;  
}uTof;  
  
void function()  
{  
    uTof data;  
  
    float f;  
  
    data.u[0] = 0x0D;  
  
    data.u[1] = 0xDD;  
  
    data.u[2] = 0x6C;  
  
    data.u[3] = 0xBD;  
  
    f = data.f;  
  
    printf("Calibration ACC X: %f \n", f);  
  
    data.u[0] = 0xF9;
```

```
data.u[1] = 0xDE;

data.u[2] = 0x5F;

data.u[3] = 0x3C;

f = data.f;

printf("Calibration ACC Y: %f \n", f);

data.u[0] = 0x9F;

data.u[1] = 0x5C;

data.u[2] = 0x83;

data.u[3] = 0x3F;

f = data.f;

printf("Calibration ACC Z: %f \n", f);

}
```

下面是通过获取的数据包来解析原始数据的例程：

一帧数据： 41 78 FF 06 81 47 01 84 06 07 00 FB FF 0B 00 02 84 06 98 00 75 FE
C3 FF 01 88 0C 19 99 B1 3C F2 FB 59 3B 6C E9 7D 3F 00 B0 10 F0 FB 7F 3F CE
A2 D4 3A 52 82 34 BC 85 AC 38 39 01 B0 10 B5 9B A1 BF 58 1F 3E 3E 03 19 B6
3C 00 00 00 00 59 6D

取其中陀螺仪Raw数据，PID: 0x84 0x01, PL:0x06, Data: 07 00 FB FF 0B 00

解析陀螺仪原始数据代码：

```
typedef union
{

    char c[2];
```

```
short s;

}uTos;

const float GYRO_GAIN[4] = {0.0076296273689993, 0.0152592547379986,
0.0305185094759972, 0.0610370189519944 };

void function()

{

    uTos data;

    short s;

    data.c[0] = 0x07;

    data.c[1] = 0x00;

    s = data.s;

    printf("Raw GYRO X: %d \n", s);

    float x = (float)s * GYRO_GAIN[scale];

    printf("Raw GYRO X: %f \n", x);


    data.c[0] = 0xFB;

    data.c[1] = 0xFF;

    s = data.s;

    printf("Raw GYRO Y: %d \n", s);

    float y = (float)s * GYRO_GAIN[scale];

    printf("Raw GYRO Y: %f \n", y);


    data.c[0] = 0x0B;
```

```
data.c[1] = 0x00;

s = data.s;

printf("Raw GYRO Z: %d \n", s);

float z = (float)s * GYRO_GAIN[scale];

printf("Raw GYRO Z: %f \n", z);

}
```

7.5.2 STM32 IIC and SPI

本节内容主要介绍的是主机和 Saber 之间使用 IIC 和 SPI 的底层协议通信例程。

IIC相关例程分为两个，一个是Low-Level Protocol

Communication_STM32_IIC_without_DRDY，这个例程是不使用DRDY时的例程。另外一个
是Low-Level Protocol Communication_STM32_IIC_with_DRDY，这个例程是使用DRDY的例
程。如果想要保证数据同步性的话，推荐使用DRDY。

SPI相关例程只有一个， Low-Level Protocol Communication_STM32_IIC_with_DRDY，这个例
程是使用DRDY的例程。SPI接口必须使用DRDY。

“Figure 19– IIC and SPI例程流程图.”展示的是针对IIC和SPI例程所写的流程图，其中主要提
供了***SwitchToConfigMode***、***SetDataPacketConfig***、***SetPacketUpdateRate***、
GetDataPacketConfig、***SwitchToMeasureMode***五种message使用IIC和SPI协议设置的方法，
以及如何使用IIC和SPI协议获取**MEASURE**数据。

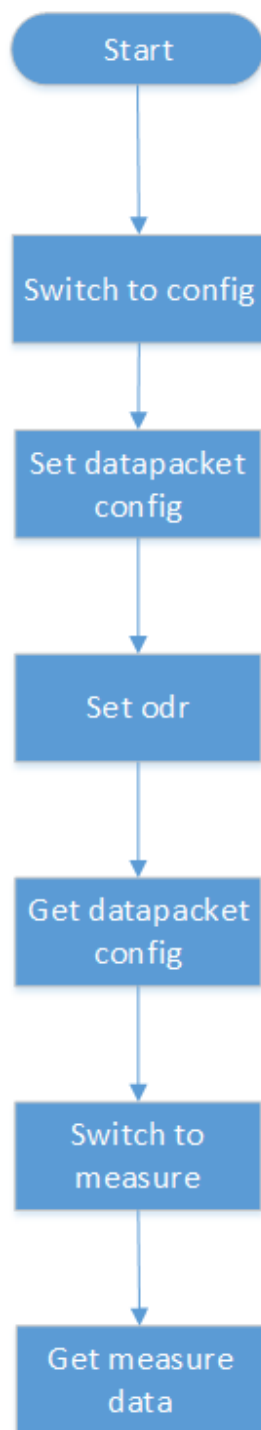


Figure 19 – IIC and SPI 例程流程图

7.5.2.1 Communication without DRDY

“Figure 20 – IIC发送Message流程”和“Figure 21 – IIC获取Measure数据流程”分别展示了使用IIC发送message以及获取测量数据的具体流程。

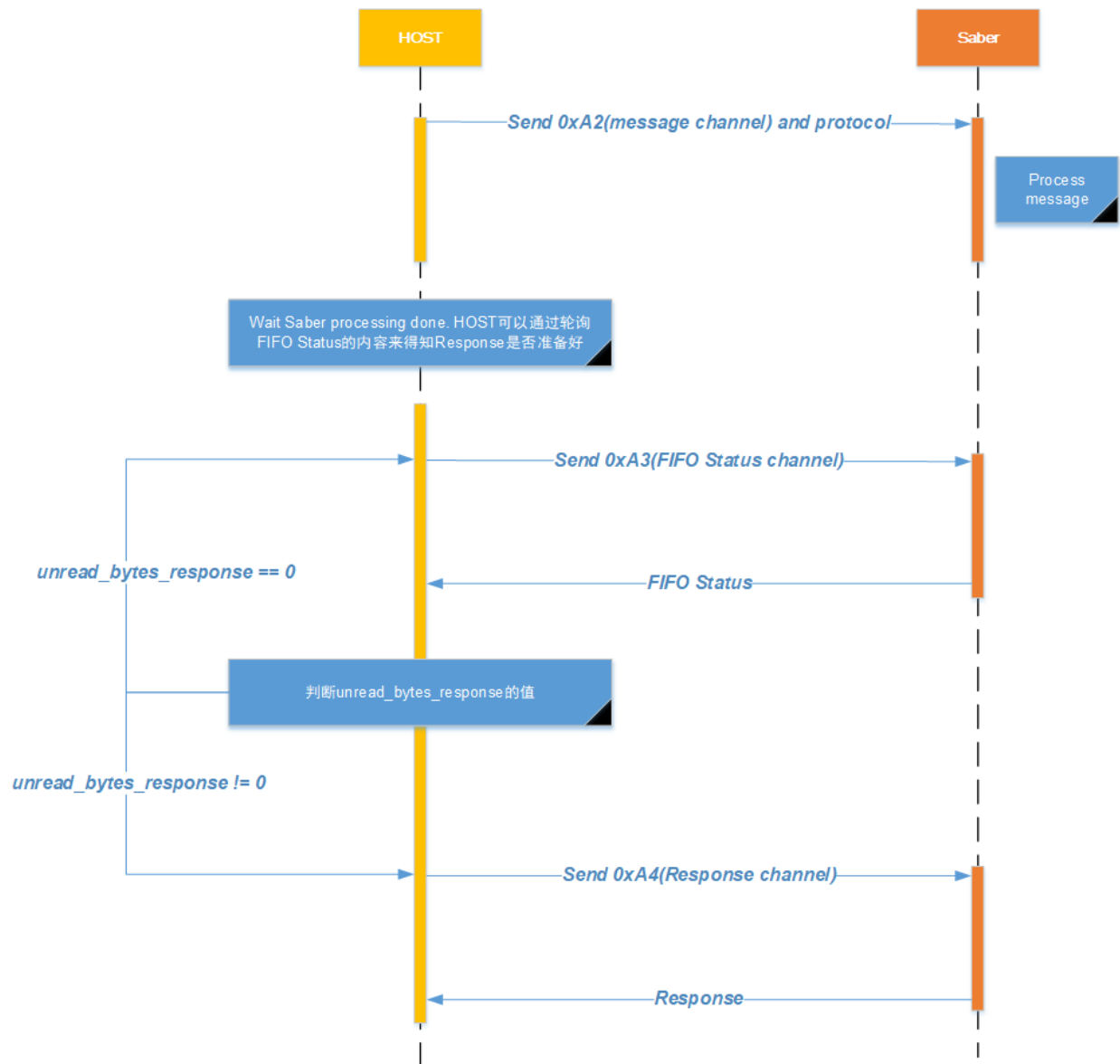


Figure 20 – IIC 发送 Message 流程

操作流程为：

①首先发送SSCP中的0xA2(Message channel)，然后发送message，详见“DS0001_Saber C1_C2_C3 Datasheet”中SSCP章节。

②如果在不使用DRDY的情况下，就需要使用轮询的方式，不断发送0xA3(FIFO_Status channel)以获取FIFO_Status的内容，当获取到FIFO_Status中的unread_bytes_response不为0时(代表Response_fifo中有值可读)。

③开始发送0xA4(Response channel)来获取响应。

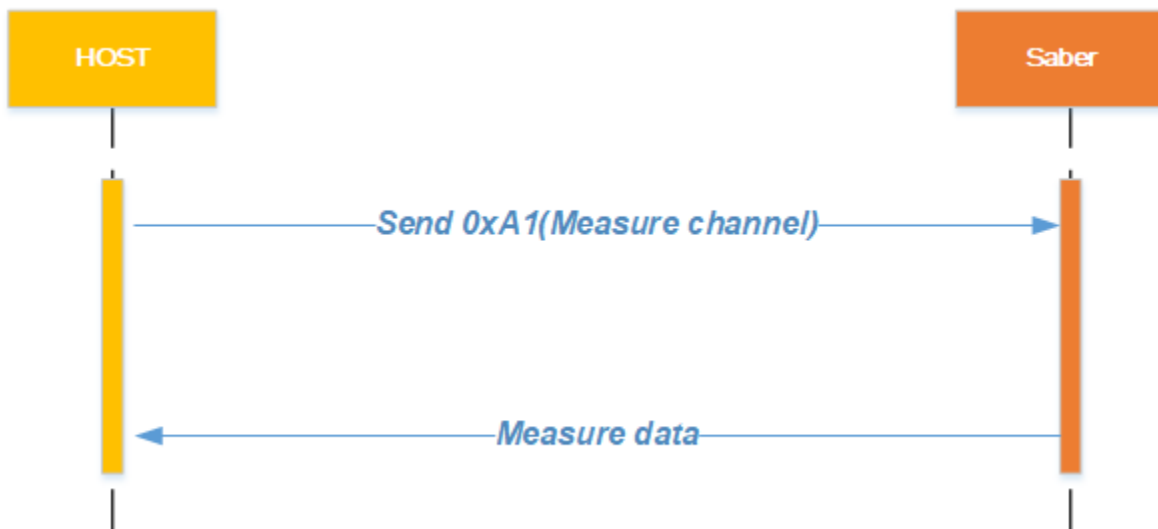


Figure 21 – IIC 获取 Measure 数据流程

当想要获取测量数据时，可以不断发送0xA1(measure channel)来获取测量数据

7.5.2.2 Communication with DRDY

本节对应的例程都是DRDY引脚配置为高有效的情况。

“Figure 22 – IIC and SPI with DRDY发送Message流程”和“Figure 23 – IIC and SPI with DRDY获取Measure数据流程”分别展示了使用DRDY发送message以及获取测量数据的具体流程。

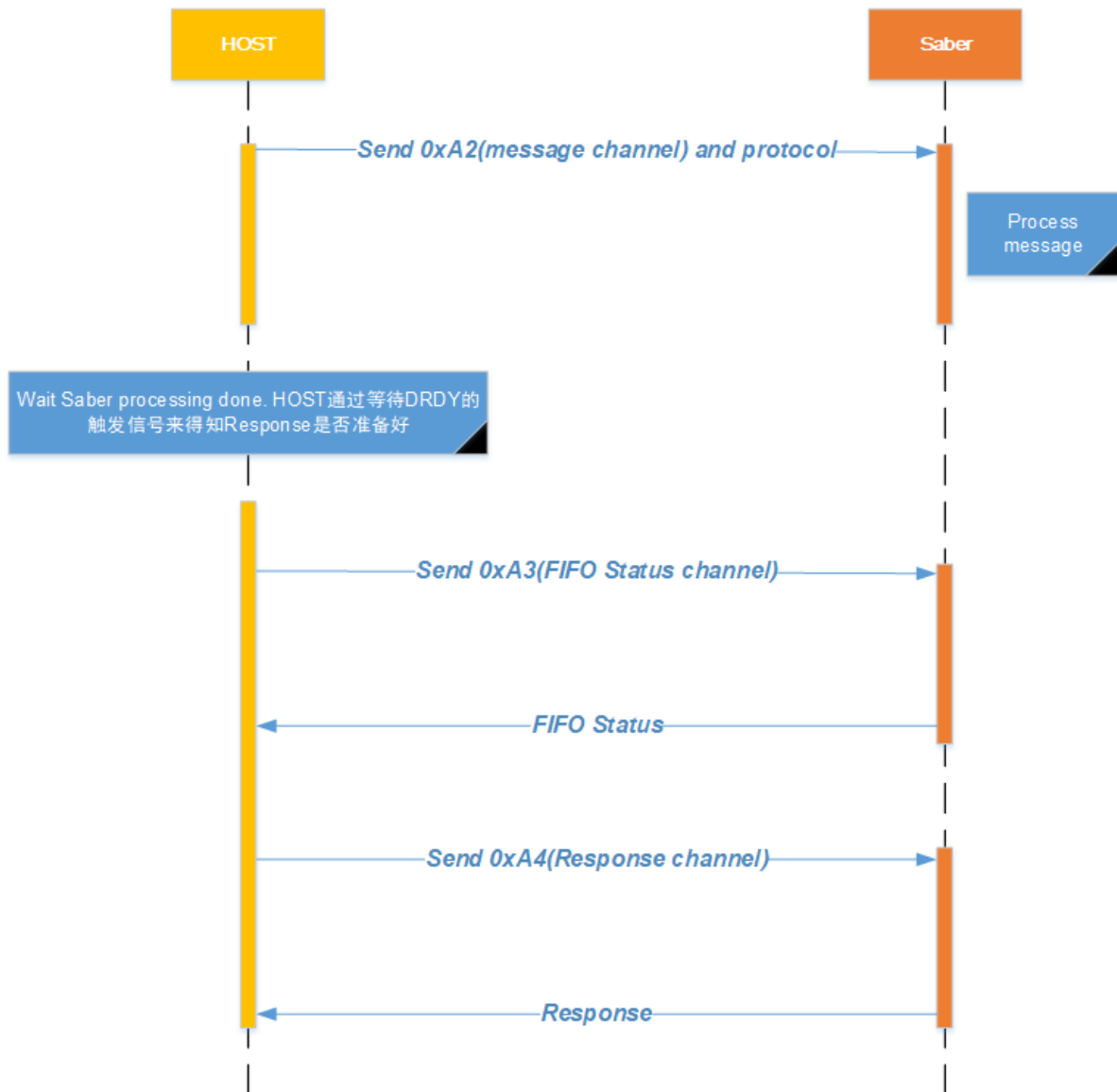


Figure 22 – IIC and SPI with DRDY 发送 Message 流程

①首先发送SSCP中的0xA2(Message channel)，然后发送message，详见“DS0001_Saber C1_C2_C3 Datasheet”中SSCP章节。

②如果在使用DRDY的情况下，可以等待收到DRDY为高时，发送0xA3(FIFO_Status channel)以获取FIFO_Status的内容，获取到的FIFO_Status中的前四个字节便是Response_fifo中的未读字节数。

③开始发送0xA4(response channel)来获取响应。

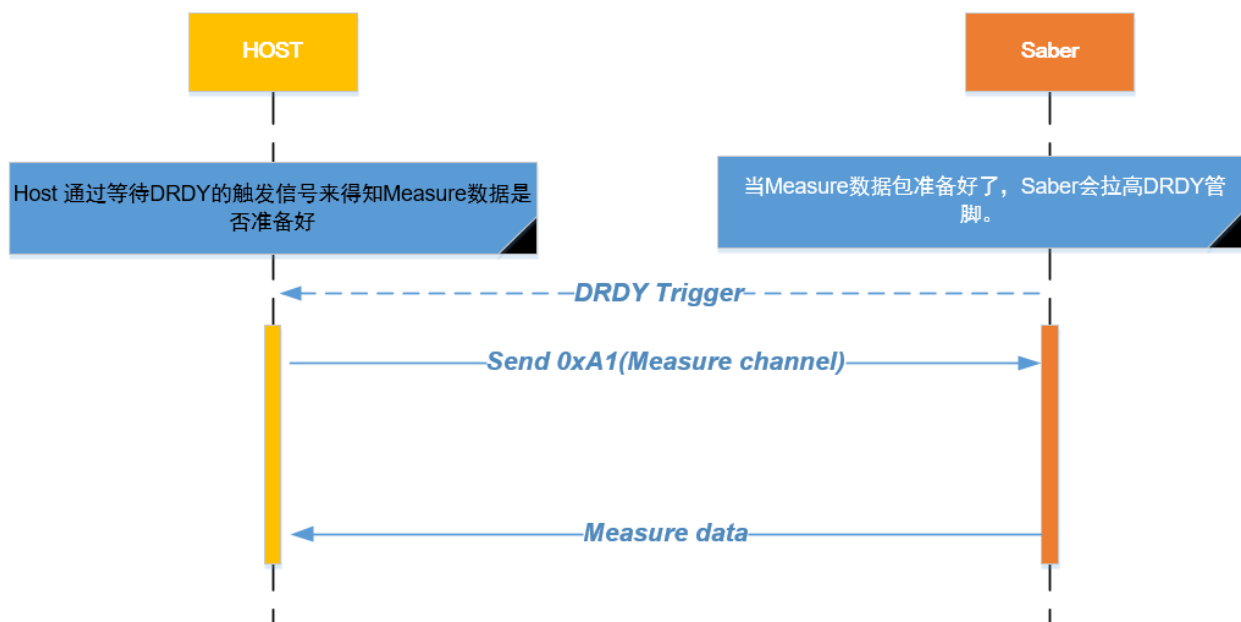


Figure 23 – IIC and SPI with DRDY 获取 Measure 数据流程

当想要获取测量数据时，可以在DRDY为高时发送0xA1(measure channel)来获取测量数据

7.6 Low-Level Download底层下载说明

本节内容主要介绍主机和Saber之间下载firmware.bin。

由于 Windows例程、Linux例程和STM32_Uart例程的协议处理逻辑和数据包解析逻辑一致，故该说明将例程进行统一描述。

以下示例中的字节均以十六进制表示。请确保应用程序能够以十六进制格式进行通信。

在更改配置之前，请保证设备处于**CONFIG**状态。

7.6.1 SwitchToConfigMode

Send **SwitchToConfigMode**:

TX: 41 78 FF 01 02 00 C5 6D(十六进制)

Receive **SwitchToConfigModeACK**:

RX: 41 78 FF 01 82 00 45 6D(十六进制)

7.6.2 FirmwareUpdateReq

Send **FirmwareUpdateReq**:

TX: 41 78 FF 0A 01 01 01 CD 6D(十六进制)

Receive **FirmwareUpdateReqACK**:

RX: 41 78 FF 0A 81 01 01 4D 6D(十六进制)

7.6.3 FirmwareUpdatePacket

Send **FirmwareUpdatePacket**:

TX: 41 78 FF 0A 02 42 01 00 xxxx 6D(十六进制)

本示例用来显示包号标记为1的第一个更新包:

- 前6个字节是Saber数据包头部;
- 红色字体(01 00)这2个字节是数据包编号, 它是0x0001。这表示数据包是第一个数据包;
- 64字节的绿色字体(xx xx)是来自bin文件的数据包。

Receive **FirmwareUpdatePacketACK**:

RX: 41 78 FF 0A 82 02 01 00 CD 6D(十六进制)

7.6.4 SystemReset

Send **SystemReset**:

TX: 41 78 FF 01 04 00 C3 6D(十六进制)

Receive **SystemResetACK**:

RX: 41 78 FF 01 84 00 43 6D(十六进制)

8 Saber 返回码

Saber 的大部分功能都有一个返回值。在本节，这些值的含义将会得到解释。

响应 ID	响应	描述
0x00	RETURN_CODE_OK	命令接收正确，操作完成
0x01	RETURN_CODE_ERROR_CLASS	CID 在类列表中是错误的或未找到的
0x02	RETURN_CODE_ERROR_CMD	MID 在命令列表中是错误的或未找到的
0x03	RETURN_CODE_ERROR_CRC	CRC 不符合从主机或设备转移的计算值和值之间的内联
0x04	RETURN_CODE_ERROR_FOOTER	Footer 不等于"0x6D"
0x05	RETURN_CODE_ERROR_PAYLOAD_LEN	Payload 长度与相应的命令定义不匹配
0x06	RETURN_CODE_ERROR_OPERATION	命令虽然被正确地接收和解析，但当设备操作相应的命令时发生错误
0x07	RETURN_CODE_ERROR_UNSupport	用户发送的消息所对应的功能暂时不支持
0x08	RETURN_CODE_ERROR_TIME_OUT	超时错误
0x09	RETURN_CODE_ERROR_FLASH_OPERATION	由于 flash 读写而发生错误。
0x0A	RETURN_CODE_ERROR_INVALID_PARA	Payload 中的参数包含无效值
0x0B	RETURN_CODE_ERROR_FIRMWARE_UPDATE	该错误信息将显示出主机固件更新不正确，固件更新程序必须重新运行。
0x0C	RETURN_CODE_ERROR_STATE	该错误信息表示用户发送的信息正确但是不属于当前的状态
0x0D	RETURN_CODE_ERROR_UART_BANDWIDTH	此错误信息表示串口带宽已超出
0x0F	RETURN_CODE_ERROR_SENSOR_INIT	此错误信息表示传感器初始化失败

Table 15 – Saber 返回码

历史版本

Revision Number	Date	Changes
0.1	16-May-2017	Initial release
0.2	15-Jul-2017	Update on following sections: - Revision history - Revise on section name
0.3	3-Aug-17	Add Low-Level Protocol Communication support Add Linux SDK Support.
0.4	5-Aug-17	Saber Packet table update Format adjustment
0.5	12-Aug-17	Add SDK sample for Arm Linux in section 7
0.6	30-Aug-17	Add command under low-level communication protocol section Remove SaberXplore related description Rename from ATR_setPacktUpdateRateReq to ATR_setPacketUpdateRateReq Add contents for explanation of SDK.
0.7	21-Oct-17	Modify Protocol, modify the relevant orders. Add SetAccelerometerCoeff GetAccelerometerCoeff SetGyroscopeCoeff GetGyroscopeCoeff SetMagnetometerCoeff GetMagnetometerCoeff
0.8	22-Nov-17	CRC is replaced with AtomBCC.
0.9	14-Dec-17	Update Sample Codes.
0.91	18-Jan-18	CRC is replaced with Atom BCC in the picture.
0.91.1	19-Jan-18	CRC is replaced with Atom BCC in the picture.
0.91.2	2-Feb-18	Add register return value, add low level download sample code.

0.92	7-Feb-18	Update API name and add LSB sample code.
0.93	15-Feb-18	Update on chapter 1, 3, 4, 5, 6 and 8.
0.94	2-Apr-18	Update on chapter 7.
0.95	4-May-18	Add code examples: Low Level Uart. Update code examples: SPI and IIC.
0.96	16-May-18	Update chapter 6. Update chapter 3. Update figure 15 -18
0.97	15-Oct-18	Add Rotation Matrix in Chapter 4 Add "SetDistanceOffSet", "SetVelocityOffSet", "SetStartYawReq" in chapter 6 Update selftest bit in section "6.3.8 RespAllStatus"
0.98	9-Nov-18	Add Sync out function Update BaudRate Macro Definition Add description of RAW/CAL/KAL data in Section "4.7.2 原始数据(Acc/Gyro/Mag)" Modify section 7.1 Modify section Magnetic distortion intensity Modify picture of GetAllStatus Ack.
0.99	24-Jan-19	Add section "2.2.3Windows 文件夹", Minor typo corrections
1.00	29-Apr-19	Add section "2.2.2Linux 文件夹", add the environment version description of the library file. Add section 7, modify demo routine description. Modify section "4.7Saber 数据包结构", add Packet description. Modify section "6.3.8RespAllStatus", update status word definition. Modify section "4.7.2 原始数据(Acc/Gyro/Mag)", update RAW/CAL/KAL description. Modify section "5.63ATR_GetScenarioReq()", add SetScenarioReq command. Add section "6.5.39" – "6.5.43", odometer command Support

1.01	9-Sep-19	Modify section “5.40” remark: ATR_SetPacketUpdateRateReq((u8 *)setRate,datalen)->ATR_SetPacketUpdateRateReq((u8 *)(&setRate),datalen) 2.Update Dataformat. 3.Update Data Packet PID in “4.7.2 原始数据 (Acc/Gyro/Mag” 4. Make corrections for modify section “5.63ATR_GetScenarioReq(” Synchronize versions for adding C3+ feature and function
1.02	18-Oct-19	Update with details and minor correction
1.03	23-Oct-19	Update with format and content Update 2.2 SDK 结构 Update figure1
1.04	2-Dec-19	Update with details and minor correction
1.05	4-Jan-20	Update chapter 7 Modify section “6.5.48 SetCoordinatesMode”
1.06	10-Feb-20	Remove Magnetic distortion intensity Remove unused PID Remove GyroscopeAutoCalibration related interface

Table 16 – 历史版本